

UNIVERZITET U BEOGRADU  
MATEMATIČKI FAKULTET



Aleksandra A. Trailović

UPOREDNA ANALIZA ANGULAR-A I  
VUE.JS-A

master rad

Beograd, 2026.

**Mentor:**

dr Vladimir FILIPOVIĆ, redovan profesor  
Univerzitet u Beogradu, Matematički fakultet

**Članovi komisije:**

dr Aleksandar KARTELJ, docent  
Univerzitet u Beogradu, Matematički fakultet

dr Mirko SPASIĆ, docent  
Univerzitet u Beogradu, Matematički fakultet

**Datum odbrane:** \_\_\_\_\_

**Naslov master rada:** Uporedna analiza Angular-a i Vue.js-a

**Rezime:** Razvoj modernih veb aplikacija doveo je do potrebe za sveobuhvatnim radnim okvirima (engl. *frameworks*) koji olakšavaju organizaciju koda, unapređuju održivost i poboljšavaju korisničko iskustvo. Među savremenim rešenjima za razvoj veb aplikacija nalaze se *Angular* i *Vue.js*, koji primenjuju različite pristupe organizaciji i razvoju aplikacija.

Ovaj master rad predstavlja uporednu analizu *Angular*-a i *Vue.js*-a, sa ciljem da se sagledaju njihove arhitektonske osobine, prednosti i ograničenja u razvoju modernih aplikacija. Pored teorijskog pregleda ključnih koncepata kao što su jednostranične aplikacije (engl. *Single Page Applications, SPA*), obrasci dizajna (engl. *design patterns*) i komponentni pristup (engl. *component-based approach*), rad obuhvata i praktičnu implementaciju istog zadatka u oba radna okvira. Teorijska analiza dopunjena je praktičnim poređenjem načina organizacije projekta i implementacije istih funkcionalnosti, kako bi se sagledale razlike koje mogu biti značajne pri izboru odgovarajućeg radnog okvira.

Zaključak rada daje preporuke za izbor tehnologije u zavisnosti od tipa projekta i potreba razvojnog tima, čime se rad pozicionira kao koristan vodič programerima i organizacijama koje planiraju razvoj savremenih veb aplikacija.

**Ključne reči:** *JavaScript, Angular, Vue.js*, radni okviri (engl. *frameworks*), SPA, komponentni pristup (engl. *component-based approach*), frontend razvoj (engl. *frontend development*)

# Sadržaj

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Uvod</b>                                                              | <b>1</b>  |
| <b>2</b> | <b>Teorijski okvir</b>                                                   | <b>4</b>  |
| 2.1      | Osnove veb tehnologija . . . . .                                         | 4         |
| 2.2      | Biblioteke i prelaz ka radnim okvirima . . . . .                         | 5         |
| 2.3      | Frontend radni okviri . . . . .                                          | 6         |
|          | Jednostranične aplikacije . . . . .                                      | 6         |
|          | Odabrani obrasci relevantni za organizaciju korisničkog interfejsa . . . | 8         |
|          | Komponentni pristup . . . . .                                            | 10        |
| 2.4      | Zaključna razmatranja . . . . .                                          | 12        |
| <b>3</b> | <b>Ključni koncepti i poređenje</b>                                      | <b>13</b> |
| 3.1      | Arhitektura i filozofija radnih okvira . . . . .                         | 14        |
| 3.2      | Složenost usvajanja . . . . .                                            | 17        |
| 3.3      | Sintaksa i rad sa šablonima . . . . .                                    | 18        |
| 3.4      | Performanse i renderovanje . . . . .                                     | 21        |
| 3.5      | Upravljanje stanjem . . . . .                                            | 25        |
| 3.6      | Ekosistem i alati . . . . .                                              | 31        |
| 3.7      | Pogodnost u različitim projektnim kontekstima . . . . .                  | 40        |
| 3.8      | Održavanje i dugoročna stabilnost . . . . .                              | 47        |
| 3.9      | Zaključna razmatranja poglavlja . . . . .                                | 52        |
| <b>4</b> | <b>Praktična implementacija</b>                                          | <b>54</b> |
| 4.1      | Postavljanje projekata . . . . .                                         | 55        |
| 4.2      | Implementacija funkcionalnosti i poređenje . . . . .                     | 57        |
|          | Rutiranje i navigacija . . . . .                                         | 58        |
|          | Autentifikacija i zaštita ruta . . . . .                                 | 62        |

|                                                            |           |
|------------------------------------------------------------|-----------|
| Upravljanje stanjem i podacima o zadacima . . . . .        | 67        |
| Prikaz liste zadataka i komponentna organizacija . . . . . | 71        |
| Pretraga, filtriranje i sortiranje . . . . .               | 76        |
| Dodavanje i izmena zadataka i validacija formi . . . . .   | 81        |
| Prikaz detalja i brisanje zadataka . . . . .               | 87        |
| 4.3 Završna razmatranja . . . . .                          | 91        |
| <b>5 Zaključak</b>                                         | <b>94</b> |
| <b>Bibliografija</b>                                       | <b>97</b> |

# Glava 1

## Uvod

U današnje vreme Internet zauzima centralno mesto u svakodnevnom životu. Veliki broj aktivnosti, od svakodnevni obaveza do poslovnih procesa, povezan je sa vebom. Putem veb aplikacija plaćamo račune, kupujemo proizvode, gledamo filmove i serije, komuniciramo sa prijateljima i kolegama, učestvujemo u onlajn nastavi ili radimo od kuće. Veb aplikacije su na taj način postale važan deo savremenog digitalnog okruženja.

Od modernih veb aplikacija danas se očekuje mnogo više od pukog prikazivanja informacija. One treba da budu interaktivne, pouzdane, jednostavne za upotrebu i prilagodljive različitim uređajima. Istovremeno, sa povećanjem broja funkcionalnosti raste i složenost razvoja, zbog čega organizacija kôda, mogućnost održavanja i izbor odgovarajućih tehnologija postaju sve značajniji [46].

Razvoj veb aplikacija pratio je prelazak od pretežno statičnih stranica ka sve složenijim i interaktivnijim sistemima. Značajnu ulogu u tom razvoju imao je programski jezik *JavaScript*, koji je omogućio dinamičko menjanje sadržaja i reagovanje na korisničke akcije u pregledaču [48]. Sa povećanjem količine kôda i složenosti korisničkih interfejsa javila se potreba za jednostavnijim načinima realizacije često ponavljanih operacija.

Jedno od rešenja predstavljale su *JavaScript* biblioteke, poput *jQuery*-ja, koje su pojednostavile rad sa *DOM*-om, obradu događaja i druge česte zadatke u razvoju veb aplikacija [61]. Međutim, biblioteke namenjene pojedinačnim funkcionalnostima same po sebi ne određuju način organizacije aplikacije u celini.

Sa rastom obima i složenosti aplikacija povećavao se značaj strukturisanijeg pristupa njihovom razvoju. Tome je doprinela šira primena radnih okvira (engl. *frameworks*), koji u većoj meri određuju organizaciju aplikacije i način povezivanja njenih

delova i pružaju mehanizme za rešavanje čestih zahteva savremenog razvoja [46].

Predmet ovog rada su radni okviri *Angular* i *Vue.js*. Iako oba omogućavaju razvoj složenih komponentno zasnovanih veb aplikacija, razlikuju se u arhitekturi, načinu organizacije komponenti, upravljanju stanjem, ekosistemu i pristupu razvoju [23, 76].

Izbor radnog okvira za razvoj savremenih veb aplikacija zavisi od većeg broja faktora, među kojima su performanse, složenost, lakoća učenja, modularnost, mogućnost proširivanja, podrška zajednice i pogodnost za konkretan projekat [62]. Zbog toga poređenje radnih okvira zahteva razmatranje više tehničkih i projektnih kriterijuma, kao i načina na koji njihove karakteristike odgovaraju zahtevima konkretnog projekta i organizaciji razvojnog tima.

U literaturi postoje empirijska poređenja radnih okvira *Angular* i *Vue.js*, zasnovana na implementaciji funkcionalno sličnih aplikacija i analizi njihovih karakteristika [43]. U ovom radu poređenje je sprovedeno na teorijskom i praktičnom nivou. Teorijska analiza usmerena je na poređenje ključnih koncepata i razvojnih pristupa dva radna okvira, dok praktični deo omogućava poređenje načina na koji se iste funkcionalnosti implementiraju u oba okruženja.

## Ciljevi i doprinos rada

Glavni cilj rada je da izvrši uporednu analizu radnih okvira *Angular* i *Vue.js* kroz teorijsko razmatranje njihovih ključnih koncepata i praktičnu implementaciju dve međusobno funkcionalno ekvivalentne aplikacije.

Teorijski deo rada obuhvata poređenje arhitekture i razvojne filozofije, složenosti usvajanja, sintakse i rada sa šablonima, performansi i renderovanja, upravljanja stanjem, ekosistema i razvojnih alata, pogodnosti u različitim projektnim kontekstima, kao i održavanja i dugoročne stabilnosti. Praktični deo zasniva se na implementaciji dve međusobno funkcionalno ekvivalentne aplikacije, čime se omogućava neposredno poređenje načina organizacije projekta i implementacije istih funkcionalnosti.

Na osnovu teorijske analize i praktične implementacije razmatraju se prednosti i ograničenja oba pristupa, kao i okolnosti u kojima njihove različite karakteristike mogu biti značajne pri izboru tehnologije. Doprinos rada ogleda se u povezivanju teorijskog poređenja sa praktičnom implementacijom istih funkcionalnosti i izdvajanju kriterijuma koji mogu poslužiti kao orijentir pri izboru između radnih okvira

*Angular* i *Vue.js* u zavisnosti od zahteva konkretnog projekta i potreba razvojnog tima.<sup>1</sup>

### Struktura rada

Rad je organizovan na sledeći način: u drugom poglavlju predstavljen je teorijski okvir sa osnovnim tehnologijama i konceptima relevantnim za razumevanje savremenih veb radnih okvira. Treće poglavlje daje pregled radnih okvira *Angular* i *Vue.js* i sadrži njihovu teorijsku uporednu analizu kroz arhitekturu, složenost usvajanja, sintaksu i rad sa šablonima, performanse i renderovanje, upravljanje stanjem, ekosistem i alate, pogodnost u različitim projektnim kontekstima, kao i održavanje i dugoročnu stabilnost. U četvrtom poglavlju prikazana je praktična implementacija istih funkcionalnosti u oba radna okvira i sprovedena analiza dobijenih rešenja na osnovu strukture projekta i primera kôda. Peto poglavlje donosi završna razmatranja i zaključke, koji sumiraju rezultate analize i ukazuju na smernice za praktičnu primenu.

Na taj način rad povezuje teorijske osnove i praktičnu primenu u okviru uporedne analize *Angular*-a i *Vue.js*-a.

---

<sup>1</sup>Tokom izrade rada korišćen je *ChatGPT* kao pomoćni alat prvenstveno za organizovanje i strukturisanje pojedinih delova rukopisa, jezičko i stilsko doterivanje, preformulisanje i jasnije izražavanje tehničkog sadržaja, kao i za povremenu diskusiju pojedinih delova praktične implementacije. Autorka poseduje višegodišnje praktično iskustvo u razvoju veb aplikacija, prvenstveno u radu sa *Vue.js*-om, kao i iskustvo u radu sa *Angular*-om. Tehničko razumevanje obrađenih koncepata, izbor aspekata za poređenje i konačne odluke u vezi sa sadržajem i programskom implementacijom zasnovani su na znanju i iskustvu autorke, uz proveru relevantnih karakteristika korišćenih verzija prema zvaničnoj dokumentaciji i navedenim izvorima.



# Glava 2

## Teorijski okvir

Cilj ovog poglavlja je da predstavi teorijske koncepte potrebne za razumevanje uporedne analize radnih okvira *Angular* i *Vue.js*.

Najpre su predstavljene osnovne veb tehnologije na kojima počiva frontend razvoj (engl. *frontend development*), među kojima su *HTML*, *CSS* i *JavaScript*, a zatim i *TypeScript*, koji ima značajnu ulogu u savremenom frontend razvoju.

Zatim je prikazan prelaz od biblioteka ka radnim okvirima i objašnjeni su razlozi zbog kojih se javila potreba za strukturisanim pristupom razvoju veb aplikacija.

Posebna pažnja posvećena je jednostraničnim aplikacijama, odabranim obrascima relevantnim za razumevanje organizacije korisničkog interfejsa i reagovanja na promene, kao i komponentnom pristupu.

Ovi koncepti pružaju osnovu za razumevanje načina na koji savremeni radni okviri organizuju aplikaciju, povezuju podatke i prikaz i reaguju na promene stanja, čime se uspostavlja teorijski okvir za poređenje sprovedeno u nastavku rada.

### 2.1 Osnove veb tehnologija

Osnovne tehnologije na kojima se zasniva frontend razvoj obuhvataju *HTML*, *CSS* i *JavaScript*. One zajedno omogućavaju definisanje strukture, izgleda i ponašanja veb stranica i aplikacija i predstavljaju osnovu savremenog frontend razvoja.

*HTML* definiše strukturu i sadržaj veb dokumenta [53], dok *CSS* omogućava kontrolu njegove prezentacije, uključujući stilizovanje i raspored elemenata [44]. *JavaScript* omogućava interaktivnost i dinamičko menjanje sadržaja veb stranice, između ostalog i kroz rad sa objektnim modelom dokumenta (engl. *Document Object Model*, *DOM*) [49].

Pored navedenih tehnologija, značajnu ulogu u savremenom frontend razvoju ima i *TypeScript*. *TypeScript* proširuje *JavaScript* sistemom statičkih tipova, čime omogućava proveru tipova pre izvršavanja programa. *TypeScript* kôd se prevodi u *JavaScript*, koji se zatim izvršava u odgovarajućem *JavaScript* okruženju [70].

*Angular* se standardno koristi sa *TypeScript*-om, dok *Vue.js* pruža podršku za njegovu upotrebu, ali je ne zahteva [8, 99]. Zbog toga je *TypeScript* značajan u kontekstu oba radna okvira, iako se način i stepen njegove primene razlikuju.

## 2.2 Biblioteke i prelaz ka radnim okvirima

Sa povećanjem složenosti veb stranica i aplikacija rasla je i količina *JavaScript* kôda potrebnog za realizaciju interaktivnosti korisničkog interfejsa. Jedan od izazova predstavljao je direktan rad sa *DOM*-om. *DOM* predstavlja veb dokument kao hijerarhijsku strukturu čvorova, pri čemu pojedini čvorovi predstavljaju elemente, tekstualni sadržaj i druge delove dokumenta. Pomoću *DOM* metoda moguće je pristupiti postojećim elementima, menjati njihov sadržaj i svojstva, kao i kreirati i dodavati nove elemente [47].

```
const list = document.querySelector('#todo');           // referenca na UL
const li   = document.createElement('li');             // kreiranje novog LI elementa
li.textContent = 'Novi zadatak';                       // dodavanje sadržaja u LI element
li.classList.add('todo-item');                         // dodavanje CSS klase elementu
list.appendChild(li);                                  // umetanje elementa u DOM
```

Kôd 2.1: Dodavanje `<li>` elementa pomoću čistog JavaScript-a

U kôdu 2.1 prikazano je kako se novi element eksplicitno kreira, zatim mu se dodeljuju sadržaj i *CSS* klasa, nakon čega se umeće u postojeću *DOM* strukturu. Kod složenijih korisničkih interfejsa, veliki broj ovakvih operacija može povećati količinu kôda i otežati njegovu organizaciju [46].

Za pojednostavljivanje često ponavljanih operacija počele su da se koriste *JavaScript* biblioteke (engl. *libraries*). Biblioteke predstavljaju skupove unapred pripremljenih funkcija i drugih programskih elemenata koje programer može da koristi za realizaciju određenih zadataka, čime se omogućava ponovna upotreba kôda i pojednostavljuje implementacija čestih funkcionalnosti [46].

Među značajnim primerima *JavaScript* biblioteka izdvaja se *jQuery*. Ova biblioteka pojednostavljuje rad sa *DOM* elementima i ublažava razlike u ponašanju između različitih pregledača [61].

```
$('#todo').append('<li class="todo-item">Novi zadatak</li>');
```

Kôd 2.2: Dodavanje `<li>` elementa pomoću jQuery-a

Kôd 2.2 pokazuje kako se isti rezultat može postići kraćim zapisom pomoću biblioteke *jQuery*, bez eksplicitnog pozivanja više pojedinačnih *DOM* metoda.

Pored rada sa *DOM*-om, *jQuery* pruža i metode za asinhronu komunikaciju sa serverom, čime je moguće preuzeti podatke i ažurirati deo stranice bez njenog potpunog ponovnog učitavanja [50, 61]. Asinhrona komunikacija detaljnije je razmotrena u kontekstu jednostraničnih aplikacija.

Iako su biblioteke poput *jQuery*-ja značajno pojednostavile realizaciju brojnih čestih zadataka, njihova upotreba sama po sebi ne određuje arhitekturu aplikacije u celini. Sa povećanjem obima aplikacije i broja međusobno povezanih funkcionalnosti, nedovoljno strukturisan kôd može postati težak za razumevanje i održavanje i doprineti pojavi tzv. *špageti kôda* [54].

Potreba za strukturisanim načinom razvoja doprinela je široj primeni radnih okvira (engl. *frameworks*). Radni okvir predstavlja strukturisano okruženje za razvoj koje u većoj meri određuje organizaciju aplikacije i način povezivanja njenih delova [46]. Za razliku od biblioteka, čije funkcije programer poziva prema potrebi, radni okviri u većoj meri usmeravaju organizaciju aplikacije kroz određenu strukturu, pravila i konvencije razvoja [46].

## 2.3 Frontend radni okviri

Savremeni frontend radni okviri često organizuju korisnički interfejs kroz komponente i pružaju različite mehanizme za razvoj i organizaciju složenijih interaktivnih veb aplikacija [46].

Za razumevanje načina na koji radni okviri *Angular* i *Vue.js* organizuju razvoj aplikacija, u nastavku su predstavljeni koncepti jednostraničnih aplikacija, odabrani principi organizacije korisničkog interfejsa i reagovanja na promene, kao i komponentni pristup.

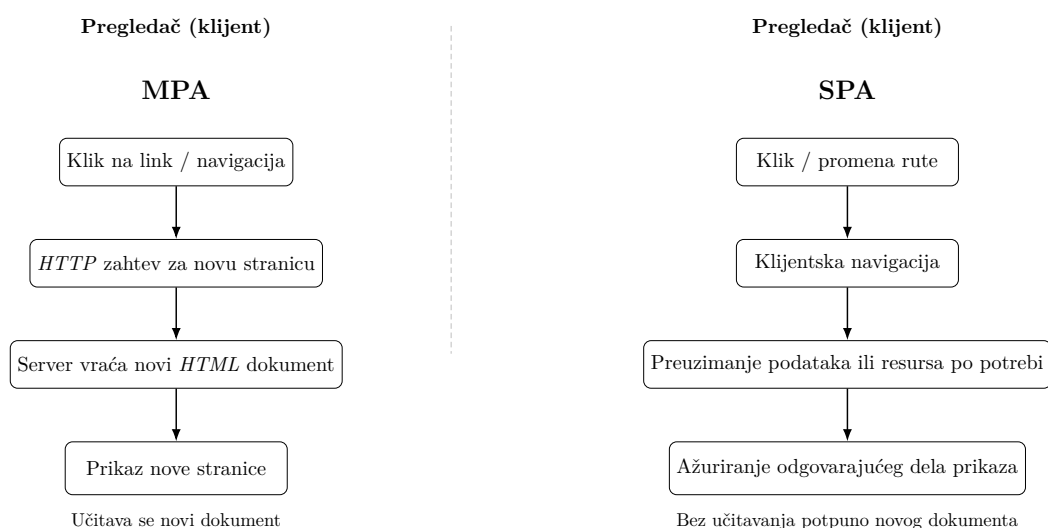
### Jednostranične aplikacije

Višestranične aplikacije (engl. *Multi-Page Applications*, *MPA*) predstavljaju pristup kod kojeg navigacija između različitih stranica najčešće podrazumeva slanje

zahteva serveru i učitavanje novog *HTML* dokumenta. Pri takvom prelasku pregledač prikazuje novu stranicu, čak i kada pojedini delovi korisničkog interfejsa, poput navigacije ili zaglavlja, ostaju sadržajno isti [46].

Jednostranične aplikacije (engl. *Single-Page Applications*, *SPA*) zasnivaju se na drugačijem pristupu. Kod tipične *SPA* aplikacije početni *HTML* dokument učitava se jednom, dok se naknadne promene prikaza obavljaju dinamički pomoću *JavaScript*-a, bez učitavanja potpuno novog dokumenta [46]. Kada su potrebni dodatni podaci, aplikacija ih može asinhrono preuzeti sa servera, često u formatu *JSON* (engl. *JavaScript Object Notation*), i na osnovu njih ažurirati odgovarajuće delove *DOM*-a [46, 55]. Na taj način moguće je ostvariti kontinuitet interakcije bez potpunog ponovnog učitavanja stranice.

Na slici 2.1 prikazan je pojednostavljen uporedni tok navigacije kod višestrančnih i jednostraničnih aplikacija. Kod *MPA* pristupa navigacija između stranica najčešće podrazumeva zahtev serveru i učitavanje novog *HTML* dokumenta, dok kod tipične *SPA* aplikacije klijentska navigacija omogućava ažuriranje odgovarajućeg dela korisničkog interfejsa bez učitavanja potpuno novog dokumenta.



Slika 2.1: *MPA* i *SPA* — pojednostavljen prikaz toka navigacije

Asinhrona komunikacija sa serverom predstavlja jedan od važnih mehanizama u razvoju jednostraničnih aplikacija. Tehnika *AJAX* omogućava slanje asinhronih *HTTP* zahteva i ažuriranje odgovarajućih delova stranice bez njenog potpunog ponovnog učitavanja [50]. Za realizaciju ovakvih zahteva tradicionalno se koristio interfejs *XMLHttpRequest*, dok *Fetch API* predstavlja savremeniji i fleksibilniji interfejs za preuzimanje resursa preko mreže [45].

Drugi važan element *SPA* aplikacija jeste klijentsko rutiranje. Pri klijentskoj navigaciji prelazak između ruta omogućava promenu prikazanog sadržaja bez učitavanja potpuno novog veb dokumenta. *History API* omogućava upravljanje istorijom sesije i promenu URL-a bez neposrednog učitavanja novog dokumenta [51].

Ipak, *SPA* pristup donosi i određene izazove. Oni mogu obuhvatiti optimizaciju za pretraživače (engl. *Search Engine Optimization, SEO*), upravljanje stanjem aplikacije, implementaciju navigacije i praćenje performansi [46, 40]. U zavisnosti od zahteva konkretne aplikacije, ovi aspekti moraju se uzeti u obzir prilikom izbora odgovarajućeg pristupa razvoju.

### **Odabrani obrasci relevantni za organizaciju korisničkog interfejsa**

Pri razvoju složenijih softverskih sistema često se javljaju slični problemi povezani sa organizacijom kôda, razdvajanjem odgovornosti, komunikacijom između delova sistema i upravljanjem promenama. Za njihovo rešavanje mogu se primenjivati različiti arhitektonski obrasci i obrasci dizajna.

Obrasci dizajna (engl. *design patterns*) ne predstavljaju gotove implementacije, već opisuju opšta i ponovo primenljiva rešenja za probleme koji se pojavljuju u različitim softverskim sistemima. Njihova upotreba omogućava programerima da koriste zajedničku terminologiju i postojeća iskustva pri projektovanju softvera [39].

U nastavku su predstavljeni arhitektonski obrasci *MVC* i *MVVM*, relevantni za razumevanje organizacije korisničkog interfejsa i razdvajanja odgovornosti, kao i obrazac dizajna posmatrač (engl. *Observer*). Njihovo razmatranje pruža teorijski kontekst za kasniju analizu organizacije aplikacije, povezivanja podataka i prikaza i reakcije sistema na promene.

#### **Model–prikaz–kontroler**

Obrazac model–prikaz–kontroler (engl. *Model–View–Controller, MVC*) zasniva se na razdvajanju odgovornosti između modela, prikaza i kontrolera. Model upravlja podacima i poslovnim logikom, prikaz predstavlja korisnički interfejs, dok kontroler obrađuje korisničke akcije i povezuje odgovarajuće promene modela i prikaza [66, 52].

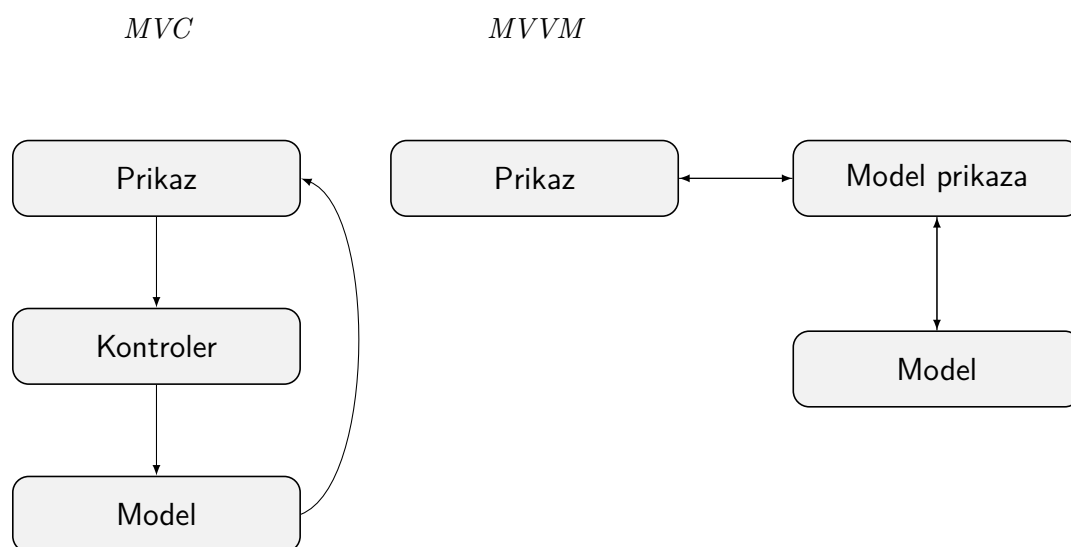
Osnovna ideja ovog pristupa jeste razdvajanje podataka, korisničkog interfejsa i logike koja upravlja njihovom međusobnom interakcijom. Takva podela pruža teorijski kontekst za razumevanje organizacije odgovornosti u složenijim aplikacijama.

**Model–prikaz–model prikaza**

Obrazac model–prikaz–model prikaza (engl. *Model–View–ViewModel*, *MVVM*) uvodi sloj modela prikaza (engl. *ViewModel*) između modela i prikaza. Model obuhvata podatke i poslovnu logiku, prikaz predstavlja korisnički interfejs, dok model prikaza sadrži stanje i logiku potrebnu za prikaz i posreduje između modela i korisničkog interfejsa [41, 68].

Važan princip povezan sa ovim obrascem jeste vezivanje podataka (engl. *data binding*), kojim se omogućava povezivanje vrednosti između prikaza i modela prikaza. Poseban oblik predstavlja dvostrano vezivanje (engl. *two-way binding*), kod kojeg se promene podataka mogu odraziti na korisnički interfejs, dok se korisničke izmene mogu preneti na odgovarajuće podatke [68].

Ovi principi pružaju teorijski kontekst za razumevanje načina na koji savremeni frontend radni okviri povezuju stanje aplikacije sa prikazom, bez potrebe da se konkretni radni okviri neposredno klasifikuju kao *MVVM* sistemi.



Slika 2.2: Obrasci MVC i MVVM – relacije između delova

Razlike između obrazaca *MVC* i *MVVM* prikazane su na slici 2.2. Kod obrasca *MVC* kontroler posreduje između korisničkih akcija, modela i prikaza, dok u obrascu *MVVM* sloj modela prikaza posreduje između prikaza i podataka i omogućava njihovo povezivanje putem mehanizma vezivanja podataka.

### Posmatrač

Obrazac posmatrač (engl. *Observer*) zasniva se na tome da se objekti koji zavise od određenog stanja obaveštavaju kada dođe do njegove promene. Subjekat (engl. *Subject*) održava skup svojih posmatrača (engl. *Observers*) i obaveštava ih kada nastupi odgovarajuća promena [39].

Ovaj princip pruža teorijski kontekst za razumevanje mehanizama kod kojih promena stanja dovodi do ažuriranja zavisnih delova korisničkog interfejsa. Savremeni radni okviri takvo ponašanje ostvaruju sopstvenim mehanizmima reaktivnosti i praćenja zavisnosti, pa se oni ne moraju posmatrati kao direktne implementacije obrasca *Observer* [3, 80].

Prednost ovog pristupa ogleda se u smanjenoj zavisnosti između subjekta i objekata koji reaguju na njegove promene. Ipak, kod većeg broja međusobnih zavisnosti tok promena može postati teži za praćenje i razumevanje [39].

Navedeni obrasci pružaju teorijski kontekst za razumevanje razdvajanja odgovornosti, povezivanja podataka i prikaza i reagovanja delova sistema na promene stanja. Ovi principi značajni su za razumevanje načina na koji savremeni radni okviri organizuju korisnički interfejs i upravljaju njegovim promenama.

Važnu ulogu u organizaciji savremenih veb aplikacija ima i komponentni pristup, koji je predstavljen u narednom delu.

### Komponentni pristup

Komponentni pristup (engl. *component-based approach*) zasniva se na organizovanju korisničkog interfejsa kao skupa manjih, međusobno povezanih i ponovo upotrebljivih celina – komponenti. Svaka komponenta predstavlja određeni deo korisničkog interfejsa, dok se složeniji interfejs formira povezivanjem više komponenti [2, 93].

Komponentni pristup predstavlja osnovni princip organizacije korisničkog interfejsa u radnim okvirima *Angular* i *Vue.js* [2, 93].

### Osnovne osobine komponenti

Jedna od osnovnih osobina komponenti jeste modularnost. Korisnički interfejs razlaže se na manje celine koje imaju određenu odgovornost i koje se mogu po-

vezivati sa drugim komponentama. Na taj način složeniji delovi interfejsa nastaju kombinovanjem jednostavnijih komponenti [2, 93].

Važna osobina komponentnog pristupa jeste i ponovna upotreba. Jednom definisana komponenta može se koristiti na više mesta u aplikaciji, čime se smanjuje potreba za ponavljanjem iste strukture i ponašanja [2, 93].

Komponente se najčešće organizuju hijerarhijski, tako da pojedine komponente sadrže druge komponente i zajedno formiraju stablo komponenti. Ovakva organizacija omogućava da se složen korisnički interfejs posmatra kao skup međusobno povezanih manjih celina [2, 93].

Još jedna važna osobina komponentnog pristupa jeste enkapsulacija. Komponenta objedinjuje funkcionalnost potrebnu za realizaciju određene odgovornosti i izlaže definisan interfejs preko kojeg ostvaruje komunikaciju sa drugim delovima sistema. Na taj način pojedini detalji njene unutrašnje implementacije ostaju sakriveni od delova sistema koji je koriste [69].

### **Komunikacija između komponenti**

Iako predstavljaju odvojene celine, komponente moraju međusobno da razmenjuju podatke i informacije o događajima kako bi aplikacija funkcionisala kao celina. Kod hijerarhijski organizovanih komponenti čest je odnos roditeljske komponente (engl. *parent component*) i dete-komponente (engl. *child component*), pri čemu roditeljska komponenta može da prosleđuje podatke dete-komponenti, dok dete-komponenta može da obavesti roditeljsku komponentu o nastaloj promeni ili korisničkoj akciji [2, 93].

Konkretni mehanizmi kojima se ovakva komunikacija ostvaruje zavise od korišćenog radnog okvira i razmatraju se u kasnijem teorijskom i praktičnom poređenju.

### **Prednosti i izazovi**

Podela sistema na manje celine sa definisanim odgovornostima i interfejsima može doprineti preglednijoj organizaciji i jednostavnijem održavanju softvera [69].

Sa druge strane, način podele sistema na komponente zahteva pažljivo projektovanje. Neodgovarajuća podela sistema na komponente može povećati složenost njihovih međusobnih zavisnosti i otežati praćenje komunikacije između njih. Zbog toga je pri projektovanju komponentne strukture potrebno pronaći odgovarajuću ravnotežu između razdvajanja odgovornosti i složenosti međusobnih zavisnosti.



Komponentni pristup predstavlja važnu teorijsku osnovu za razumevanje načina na koji radni okviri *Angular* i *Vue.js* organizuju korisnički interfejs i komunikaciju između njegovih delova.

### 2.4 Zaključna razmatranja

U ovom poglavlju postavljen je teorijski okvir potreban za razumevanje koncepta relevantnih za analizu savremenih frontend radnih okvira. Najpre su predstavljene osnovne veb tehnologije – *HTML*, *CSS* i *JavaScript* – kao i značaj *TypeScript*-a u savremenom frontend razvoju, a zatim razvoj od biblioteka ka radnim okvirima i potreba za strukturisanim pristupom organizaciji složenijih veb aplikacija.

Posebna pažnja posvećena je jednostraničnim aplikacijama, njihovom načinu funkcionisanja, asinhronoj komunikaciji sa serverom i klijentskom rutiranju. Zatim su predstavljeni odabrani principi organizacije korisničkog interfejsa, povezivanja podataka i prikaza i reagovanja na promene stanja, kroz arhitektonske obrasce *MVC* i *MVVM* i obrazac dizajna *Observer*. Na kraju je razmotren komponentni pristup, sa naglaskom na modularnost, ponovnu upotrebu, enkapsulaciju i komunikaciju između komponenti.

Predstavljeni koncepti čine teorijsku osnovu za uporednu analizu radnih okvira *Angular* i *Vue.js*. U narednom poglavlju radni okviri se uporedno analiziraju kroz definisane tehničke i projektne kriterijume, sa ciljem sagledavanja njihovih sličnosti, razlika i karakteristika značajnih u različitim projektnim kontekstima.

## Glava 3

# Ključni koncepti i poređenje

Cilj ovog poglavlja je da, na osnovu unapred definisanih kriterijuma, uporedi radne okvire *Angular* i *Vue.js* i prikaže razlike koje mogu biti značajne pri izboru tehnologije. Analiza je organizovana tematski i obuhvata sledeće oblasti:

1. arhitekturu i razvojnu filozofiju,
2. složenost usvajanja,
3. sintaksu i rad sa šablonima,
4. performanse i renderovanje,
5. upravljanje stanjem,
6. ekosistem i alate,
7. pogodnost u različitim projektnim kontekstima,
8. održavanje i dugoročnu stabilnost.

U okviru svakog kriterijuma razmatraju se relevantni aspekti oba radna okvira, uz neposredno poređenje njihovih ključnih razlika i praktičnih implikacija.

## Metodološka napomena

Izbor kriterijuma za poređenje zasniva se na relevantnoj literaturi o izboru i evaluaciji frontend radnih okvira. Pano i saradnici (2018) među faktorima koji utiču na njihov izbor izdvajaju performanse, lakoću učenja, složenost, razumljivost, podršku

zajednice, pogodnost za konkretne potrebe, modularnost i mogućnost proširivanja [62]. Olila i saradnici (2022) analiziraju strategije renderovanja savremenih veb radnih okvira i njihov uticaj na performanse [60], dok Lipski i saradnici (2021) porede radne okvire *Angular* i *Vue.js* na osnovu funkcionalno sličnih implementacija [43].

Kriterijumi korišćeni u ovom radu nisu preuzeti kao jedinstven skup iz jednog izvora, već su formirani objedinjavanjem aspekata relevantnih za tehničke karakteristike radnih okvira, njihov razvojni ekosistem i primenu u različitim projektnim kontekstima.

Poređenje je prvenstveno usmereno na *Angular* 22 i *Vue.js* 3.5, koji predstavljaju aktuelne stabilne verzije u trenutku izrade rada [14, 100]. Karakteristike ranijih verzija navode se kada su relevantne za razumevanje razvoja pojedinih mehanizama ili kada se razmatraju rezultati prethodnih istraživanja.

Tehničke karakteristike konkretnih mehanizama radnih okvira proveravaju se prvenstveno na osnovu njihove zvanične dokumentacije, dok se naučna literatura koristi za definisanje kriterijuma poređenja i razmatranje rezultata prethodnih istraživanja. Rezultati istraživanja zasnovanih na ranijim verzijama posmatraju se u okviru verzija koje su u tim istraživanjima analizirane i ne prenose se neposredno na savremene verzije radnih okvira.

Detalji implementacije koji nisu neposredno relevantni za navedene kriterijume izostavljeni su iz teorijskog poređenja i, kada je potrebno, obrađeni su u praktičnom delu rada.

### 3.1 Arhitektura i filozofija radnih okvira

Arhitektura radnog okvira određuje način na koji se organizuju komponente, podaci i zajedničke funkcionalnosti aplikacije, dok njegova razvojna filozofija utiče na stepen strukturisanosti i slobode koji se pruža programeru. *Angular* i *Vue.js* zasnivaju razvoj veb aplikacija na komponentnom pristupu, ali se razlikuju u načinu organizacije aplikacije i razvojnoj filozofiji.

#### Razvojna filozofija

*Angular* predstavlja radni okvir koji pruža širok skup međusobno povezanih alata, biblioteka i ugrađenih mehanizama za razvoj veb aplikacija. Njegov ekosistem obuhvata, između ostalog, komponentni model, sistem ubrizgavanja zavisnosti, ru-

tiranje, rad sa formama i zvanične razvojne alate [23]. Ovakav pristup omogućava da se česti zahtevi u razvoju aplikacija rešavaju pomoću mehanizama i konvencija koje pruža sam radni okvir [23].

*Vue.js* se u zvaničnoj dokumentaciji opisuje kao progresivni radni okvir, odnosno okvir projektovan tako da može postepeno da se uvodi i prilagođava različitim potrebama projekta [76]. Može se koristiti za dodavanje interaktivnosti postojećim veb stranicama, ali i za razvoj jednostraničnih aplikacija, kao i aplikacija koje koriste renderovanje na strani servera (engl. *server-side rendering, SSR*) [76]. Dodatne funkcionalnosti i prateći alati mogu se uključivati u skladu sa zahtevima konkretne aplikacije.

Osnovna razlika u razvojnoj filozofiji ogleda se u stepenu unapred definisane strukture. *Angular* pruža integrisaniji skup mehanizama i konvencija, dok *Vue.js* naglašava postepeno usvajanje i fleksibilnost pri izboru načina organizacije aplikacije [23, 76].

### Arhitektonska organizacija

U *Angular*-u komponente predstavljaju osnovne gradivne jedinice korisničkog interfejsa i organizovane su u hijerarhijsku strukturu. Svaka komponenta obuhvata *TypeScript* klasu koja definiše njeno ponašanje i šablon koji određuje prikaz. Klasa se označava dekoratorom `@Component`, kojim se definišu metapodaci komponente, kao što su njen selektor, šablon i druge konfiguracione osobine [2].

U savremenom *Angular*-u komponente su podrazumevano samostalne (engl. *standalone*), što znači da mogu neposredno da uvoze komponente, direktive i cevi (engl. *pipes*), koje se koriste za transformaciju vrednosti u šablonima.

U ranijem modelu organizacije *Angular* aplikacija značajnu ulogu imao je *NgModule*, klasa označena dekoratorom `@NgModule`, koja služi za grupisanje i povezivanje komponenti, direktiva i cevi, kao i za konfiguraciju pojedinih zavisnosti. Iako je ovaj pristup i dalje podržan zbog postojećih aplikacija, za novi kôd preporučuje se upotreba samostalnih komponenti [2, 28].

Važan deo arhitekture predstavlja sistem ubrizgavanja zavisnosti (engl. *dependency injection, DI*). Ubrižgavanje zavisnosti predstavlja obrazac prema kojem se zavisnosti potrebne određenoj klasi obezbeđuju spolja, umesto da ih klasa neposredno kreira. Na taj način se olakšavaju razdvajanje odgovornosti, ponovna upotreba funkcionalnosti i testiranje pojedinačnih delova aplikacije [8]. U *Angular*-u se za-

jedničke funkcionalnosti, poslovna logika ili podaci često izdvajaju u servise koji se zatim mogu ubrizgavati u komponente i druge delove aplikacije.

Za *Vue.js* projekte koji koriste proces izgradnje zvanična dokumentacija preporučuje upotrebu komponenti u jednoj datoteci (engl. *Single-File Components, SFC*) [97]. Reč je o datotekama sa ekstenzijom `.vue` u kojima su šablon, logika i stil komponente objedinjeni u jasno razdvojene celine [97]. Ovakav format omogućava da međusobno povezani delovi jedne komponente budu organizovani na jednom mestu.

Za organizaciju logike *Vue.js* pruža *Composition API*, ugrađeni skup *API*-ja za rad sa reaktivnim stanjem, životnim ciklusom komponenti i drugim funkcionalnostima okvira [75]. Logika koja se koristi u više komponenti može se izdvojiti u kompozicione funkcije (engl. *composables*), odnosno funkcije koje koriste *Composition API* kako bi se zajednička reaktivna logika organizovala i ponovo koristila u različitim komponentama [74].

Važan deo komponentnog modela *Vue.js*-a predstavlja i ugrađeni sistem reaktivnosti, čiji se mehanizmi i uticaj na ažuriranje korisničkog interfejsa detaljnije razmatraju u odeljku posvećenom performansama i renderovanju [80].

Posmatrano na arhitektonskom nivou, oba radna okvira polaze od komponentnog pristupa, ali se razlikuju u načinu organizovanja pratećih mehanizama. *Angular* komponentni model povezuje sa ugrađenim sistemom ubrizgavanja zavisnosti i izraženijim konvencijama za organizaciju aplikacije, dok *Vue.js* koristi *SFC* format i reaktivni sistem, uz mogućnost organizovanja i ponovne upotrebe logike pomoću *Composition API*-ja i kompozicionih funkcija.

### Zaključna razmatranja

Analiza razvojne filozofije i arhitektonske organizacije pokazuje da se jedna od osnovnih razlika između dva radna okvira ogleda u stepenu unapred definisane strukture. *Angular* se oslanja na integrisaniji skup međusobno povezanih mehanizama i konvencija, dok *Vue.js* primenjuje progresivniji pristup i ostavlja više prostora za postepeno uključivanje i organizovanje potrebnih funkcionalnosti.

Ovakve razlike utiču i na način na koji se osnovni koncepti radnih okvira usvajaju i primenjuju. U narednom odeljku zato se razmatra složenost njihovog usvajanja.

## 3.2 Složenost usvajanja

Složenost usvajanja predstavlja jedan od relevantnih kriterijuma pri izboru radnog okvira [62]. U ovom odeljku razmatraju se koncepti sa kojima programer treba da se upozna na početku rada sa *Angular*-om i *Vue.js*-om, kao i mogućnost njihovog postepenog uvođenja.

### Početni koncepti

Za razvoj aplikacija u *Angular*-u potrebno je upoznavanje sa većim brojem međusobno povezanih koncepata. Standardni razvoj zasniva se na *TypeScript*-u, komponentama i šablonima, dok važnu ulogu imaju i ubrizgavanje zavisnosti i dekoratori kojima se definišu metapodaci pojedinih elemenata aplikacije [23, 2].

U savremenom *Angular*-u samostalne komponente predstavljaju podrazumevani pristup, čime je smanjena potreba za poznavanjem *NgModule*-a pri razvoju novog kôda [2, 28]. Pored toga, *Angular* poseduje sistem signala za rad sa reaktivnim stanjem, odnosno vrednostima čije se promene automatski prate, dok se *RxJS* i *Observable* tokovi koriste za obradu asinhronih podataka i događaja. Radni okvir pruža i mehanizme za povezivanje ova dva pristupa [3, 31].

Zbog toga početak rada sa *Angular*-om podrazumeva usvajanje više različitih mehanizama i njihove međusobne povezanosti. Sa druge strane, unapred definisane konvencije i integrisani mehanizmi pružaju jasnu strukturu za dalji razvoj aplikacije.

Za početak rada sa *Vue.js*-om potrebno je poznavanje osnovnih veb tehnologija i komponentnog modela, dok se dodatni mehanizmi mogu uvoditi postepeno u skladu sa potrebama aplikacije. Ovakav pristup proizlazi iz progresivne prirode radnog okvira, koji može da se koristi od dodavanja interaktivnosti postojećim stranicama do razvoja jednostraničnih aplikacija [76].

Pri razvoju aplikacija koje koriste proces izgradnje, zvanična dokumentacija preporučuje upotrebu *SFC* formata [97]. Za organizaciju logike komponenti *Vue.js* podržava dva stila: *Options API* i *Composition API*. *Options API* organizuje logiku komponente kroz unapred definisane opcije, dok *Composition API* omogućava njeno organizovanje pomoću funkcija i reaktivnih *API*-ja [75].

Oba pristupa su podržana u savremenom *Vue.js*-u, a korišćenje *Composition API*-ja nije uslov za početak rada sa radnim okvirom. Mogućnost izbora načina organizovanja komponentne logike i postepenog uključivanja dodatnih funkcionalnosti

omogućava programeru da nove koncepte uvodi u skladu sa složenošću i potrebama aplikacije [76, 75].

### Zaključna razmatranja

Razlike između *Angular*-a i *Vue.js*-a u ovom kriterijumu prvenstveno se ogledaju u broju koncepata sa kojima se programer susreće na početku rada i mogućnosti njihovog postepenog uvođenja. *Angular* već u početnoj fazi uvodi širi skup međusobno povezanih mehanizama i konvencija, dok *Vue.js* omogućava postepeno uključivanje dodatnih funkcionalnosti i različite načine organizovanja komponentne logike.

Ova razlika ne znači da je jedan radni okvir jednostavan, a drugi složen u apsolutnom smislu, već da se složenost tokom procesa njihovog usvajanja raspoređuje na različite načine.

### 3.3 Sintaksa i rad sa šablonima

Sintaksa i način rada sa šablonima predstavljaju važan aspekt poređenja radnih okvira *Angular* i *Vue.js*, jer određuju način na koji se definišu struktura korisničkog interfejsa, prikaz podataka i obrada korisničkih događaja.

Pod šablonom se u ovom radu podrazumeva deklarativni opis strukture korisničkog interfejsa, zasnovan na *HTML*-u i proširen sintaksom radnog okvira. Ta sintaksa omogućava povezivanje podataka komponente sa prikazom, kao i reagovanje na događaje u korisničkom interfejsu, poput klika ili promene vrednosti unosa [5, 98].

Iako oba radna okvira koriste deklarativne šablone zasnovane na *HTML*-u, razlikuju se u sintaksi i načinu njihove organizacije. U ovom odeljku porede se organizacija komponente, kontrola toka u šablonima, vezivanje podataka i obrada događaja, kao i pravila za korišćenje izraza u šablonima.

#### Organizacija komponente

U *Angular*-u komponenta se definiše pomoću *TypeScript* klase i šablona koji određuje njen prikaz. Šablon može biti definisan neposredno u metapodacima komponente ili izdvojen u zasebnu `.html` datoteku, dok se stilovi na sličan način mogu navesti unutar komponente ili u zasebnoj datoteci [2]. Na taj način *Angular* omogućava da šablon i stilovi budu definisani neposredno uz logiku komponente ili

izdvojeni u zasebne datoteke.

U savremenim *Vue.js* projektima preporučuje se format komponenti u jednoj datoteci (engl. *Single-File Components, SFC*) [97]. U okviru datoteke sa ekstenzijom `.vue` šablon, logika i stilovi komponente organizovani su u zasebne celine: `<template>`, `<script>` i `<style>`. Logika komponente može biti pisana u *JavaScript*-u ili *TypeScript*-u, a pri korišćenju *Composition API*-ja može se koristiti i sintaksa `<script setup>` [97, 96].

Posmatrano sa stanovišta organizacije komponente, oba radna okvira omogućavaju povezivanje logike, šablona i stilova, ali primenjuju različite pristupe njihovom raspoređivanju. *Angular* omogućava da šablon i stilovi budu definisani neposredno uz logiku komponente ili izdvojeni u zasebne datoteke, dok *Vue.js* kroz *SFC* format ove delove organizuje u okviru jedne `.vue` datoteke.

### Kontrola toka u šablonima

Kontrola toka u šablonima određuje koji će se elementi prikazati i koji će se delovi šablona ponavljati u zavisnosti od uslova i podataka aplikacije.

U savremenom *Angular*-u kontrola toka u šablonima ostvaruje se blokovima `@if`, `@for` i `@switch`, koji omogućavaju uslovno prikazivanje elemenata, iteraciju kroz kolekcije i izbor između više grana prikaza [4]. Ova sintaksa dostupna je od verzije *Angular 17* i predstavlja savremeni pristup kontroli toka u šablonima.

U *Vue.js*-u kontrola toka ostvaruje se direktivama, odnosno posebnim atributima u šablonu koji radnom okviru zadaju određeno ponašanje. Za uslovno prikazivanje koriste se direktive `v-if`, `v-else-if` i `v-else`, dok se iteracija kroz kolekcije ostvaruje direktivom `v-for` [98]. *Vue.js* pruža i direktivu `v-show`, koja, za razliku od `v-if`, ne uklanja element iz *DOM*-a, već menja njegovu vidljivost pomoću *CSS* svojstva `display` [98].

Oba radna okvira na deklarativan način omogućavaju kontrolu prikaza neposredno u šablonu, ali koriste različite sintaksičke konvencije. *Angular* koristi posebne blokove konstrukcije sa prefiksom `@`, dok *Vue.js* kontrolu toka izražava direktivama koje se navode kao atributi *HTML* elemenata.



## Vezivanje podataka i obrada događaja

Vezivanje podataka predstavlja mehanizam kojim se vrednosti iz logike komponente povezuju sa elementima njenog šablona. Obrada događaja omogućava povezivanje događaja u korisničkom interfejsu, kao što su klik ili promena vrednosti unosa, sa odgovarajućom logikom komponente.

U *Angular*-u se dinamičke tekstualne vrednosti u šablonu prikazuju interpolacijom pomoću dvostrukih vitičastih zagrada `{{ }}`. Vezivanje vrednosti za svojstva elemenata ostvaruje se sintaksom `[property]`, dok se za obradu događaja koristi sintaksa `(event)` [5]. Za dvosmerno vezivanje koristi se kombinovana sintaksa `[()]`. Kod elemenata za unos podataka, kao što su tekstualna polja, *Angular* omogućava korišćenje direktive `ngModel`, uz uključivanje modula `FormsModule` [35].

U *Vue.js*-u se za tekstualnu interpolaciju takođe koriste dvostruke vitičaste zagrade. Vezivanje atributa i svojstava ostvaruje se direktivom `v-bind`, za koju postoji skraćeni oblik `:`, dok se događaji obrađuju pomoću direktive `v-on` ili njenog skraćenog oblika `@` [98]. Za dvosmerno vezivanje vrednosti elemenata za unos podataka, kao što su tekstualna polja, koristi se direktiva `v-model` [94].

Oba radna okvira omogućavaju osnovne mehanizme povezivanja podataka i korisničkog interfejsa, ali koriste različite sintaksičke konvencije. *Angular* razlikuje vezivanje svojstava i događaja pomoću uglastih i okruglih zagrada, dok *Vue.js* koristi direktive sa prefiksom `v-` i njihove skraćene oblike.

## Izrazi u šablonima

Pod izrazom se podrazumeva deo koda čijim se izvršavanjem dobija određena vrednost, na primer pristup svojstvu, aritmetičke operacije ili poređenja.

*Angular* omogućava korišćenje izraza unutar šablona, pri čemu je njihova sintaksa zasnovana na *JavaScript*-u, ali uz određena ograničenja. Podržani su, između ostalog, pristup svojstvima, aritmetički i logički operatori, poređenja i uslovni izrazi, dok deklaracije funkcija, klasa i promenljivih nisu dozvoljene u okviru šablonskih izraza [20].

*Vue.js* takođe omogućava korišćenje *JavaScript* izraza unutar šablonskih vezivanja. Izrazi se mogu koristiti u interpolaciji i vrednostima direktiva, pri čemu svako vezivanje prihvata jedan izraz, dok naredbe, kao što su deklaracije promenljivih ili uslovne naredbe, nisu dozvoljene [98].

Oba radna okvira, prema tome, omogućavaju korišćenje izraza unutar deklarativnih šablona, ali primenjuju sopstvena pravila i ograničenja njihove upotrebe. Razlike se ogledaju u podržanoj sintaksi i načinu na koji se izrazi koriste u okviru šablonskih vezivanja.

### Zaključna razmatranja

Analiza sintakse i rada sa šablonima pokazuje da *Angular* i *Vue.js* pružaju uporedive mogućnosti za definisanje prikaza, povezivanje podataka, obradu događaja i kontrolu toka, ali ih izražavaju različitim sintaksičkim konvencijama. *Angular* se oslanja na blokovske konstrukcije i posebne oznake za različite vrste vezivanja, dok *Vue.js* u većoj meri koristi direktive i njihove skraćene oblike.

Razlike se ogledaju i u organizaciji komponente, posebno u načinu na koji su šablon, logika i stilovi raspoređeni. Sa stanovišta ovog kriterijuma, ključna razlika zato nije u dostupnosti osnovnih mogućnosti, već u načinu na koji su one sintaksički izražene i organizovane.

## 3.4 Performanse i renderovanje

Pod performansama se u ovom odeljku podrazumeva efikasnost izvršavanja veb aplikacije, posebno u pogledu brzine odziva korisničkog interfejsa i količine rada potrebne za njegovo ažuriranje. Renderovanje predstavlja proces kojim se stanje aplikacije odražava na prikaz korisničkog interfejsa i kojim se taj prikaz ažurira kada dođe do promene stanja [60].

Olila i saradnici (2022) ukazuju na to da analizirani veb radni okviri primenjuju različite strategije renderovanja kako bi promene stanja aplikacije preneli na korisnički interfejs, pri čemu izbor strategije može uticati na performanse [60]. U ovom odeljku zato se porede mehanizmi koje *Angular* i *Vue.js* koriste za praćenje promena i ažuriranje prikaza, kao i načini na koje smanjuju nepotreban rad tokom renderovanja.

## Praćenje promena i renderovanje

Praćenje promena obuhvata mehanizme kojima radni okvir utvrđuje da se stanje aplikacije promenilo i određuje koje delove korisničkog interfejsa je potrebno ponovo proveriti ili ažurirati.

Kada se stanje aplikacije promeni, *Angular* mora da utvrdi koje delove korisničkog interfejsa je potrebno proveriti i eventualno ažurirati. Za to koristi mehanizam detekcije promena (engl. *change detection*), kojim se vrednosti povezane sa šablonom proveravaju i usklađuju sa prikazom [1].

Od verzije *Angular 22*, *OnPush* predstavlja podrazumevanu strategiju detekcije promena, čime se omogućava preskakanje komponenti i njihovih potomaka kada ne postoji razlog za novu proveru [1].

Važnu ulogu imaju i signali, odnosno reaktivne vrednosti čije promene *Angular* može da prati. Kada se promeni signal korišćen u šablonu, odgovarajuća komponenta označava se za proveru prilikom narednog ciklusa detekcije promena [3].

*Angular 22* podrazumevano koristi i model bez biblioteke *ZoneJS* (engl. *zone-less*). Umesto da veliki broj asinhronih aktivnosti posmatra kao mogući razlog za proveru prikaza, radni okvir se oslanja na konkretnija obaveštenja o promenama, kao što su promena ulazne vrednosti komponente, događaj iz šablona ili promena signala [37].

Kada se stanje aplikacije promeni, *Vue.js* koristi ugrađeni sistem reaktivnosti kako bi pratio zavisnosti između podataka i prikaza koji te podatke koristi. Prilikom renderovanja beleže se reaktivne vrednosti od kojih prikaz komponente zavisi, tako da njihova kasnija promena može da pokrene odgovarajuće ažuriranje korisničkog interfejsa [80].

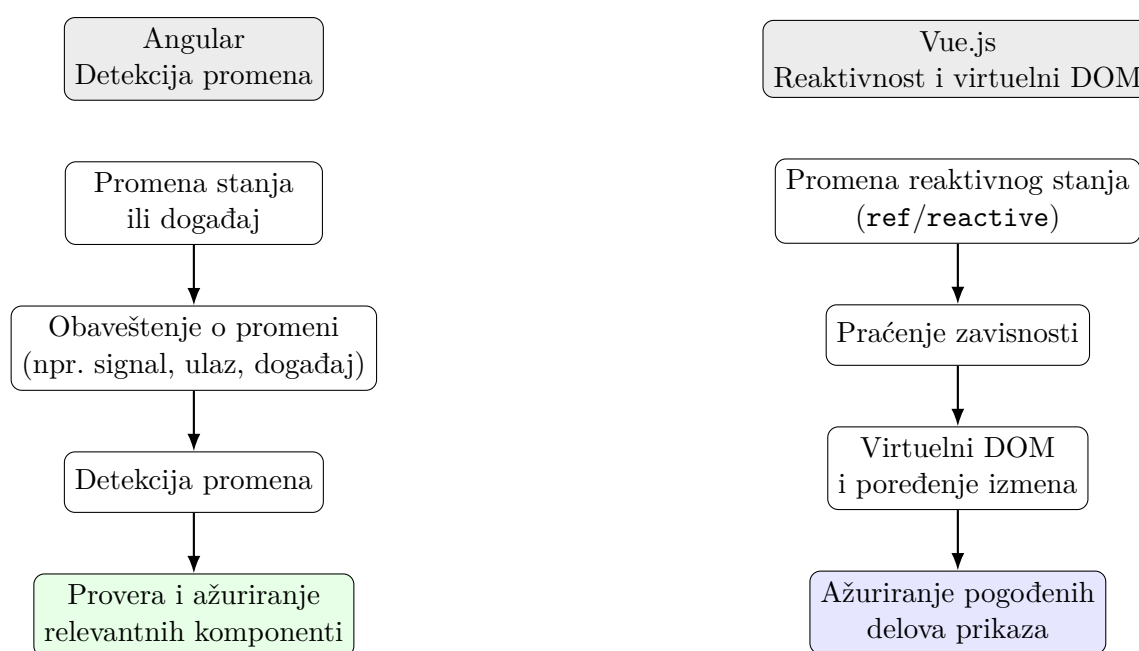
U verziji *Vue 3* reaktivni objekti kreirani pomoću funkcije `reactive()` zasnivaju se na mehanizmu *Proxy*, koji omogućava praćenje pristupa svojstvima objekta i njihovih promena, dok se kod vrednosti kreiranih pomoću funkcije `ref()` pristup i promene prate pomoću *getter/setter* mehanizma [80]. Na taj način *Vue.js* može da utvrdi od kojih reaktivnih podataka određeni deo prikaza zavisi.

Šabloni *Vue.js*-a prevode se u funkcije za renderovanje koje proizvode virtuelnu reprezentaciju korisničkog interfejsa, odnosno virtuelni *DOM* (engl. *Virtual DOM*, *VDOM*). Kada se promeni reaktivna zavisnost, formira se nova virtuelna reprezentacija koja se poredi sa prethodnom, nakon čega se potrebne izmene primenjuju na

stvarni *DOM* [82].

Pri tome *Vue.js* koristi pristup koji zvanična dokumentacija naziva virtuelnim *DOM*-om vođenim informacijama dobijenim tokom kompajliranja (engl. *compiler-informed Virtual DOM*). Analizom šablona kompajler može da prepozna statičke i dinamičke delove prikaza i tako omogućiti izbegavanje dela nepotrebnog rada tokom ažuriranja [82].

Na slici 3.1 prikazan je pojednostavljen pregled načina na koji *Angular* i *Vue.js* prate promene stanja i ažuriraju korisnički interfejs.



Ključna razlika: *Angular* koristi detekciju promena za proveru relevantnih komponenti, dok *Vue.js* prati reaktivne zavisnosti i ažurira pogođene delove prikaza.

Slika 3.1: Pojednostavljen prikaz praćenja promena i ažuriranja prikaza u radnim okvirima *Angular* i *Vue.js*

Poređenje pokazuje da se nepotreban rad pri ažuriranju korisničkog interfejsa ograničava različitim mehanizmima. *Angular* se oslanja na detekciju promena i konkretna obaveštenja o promenama, dok *Vue.js* prati reaktivne zavisnosti i koristi virtuelni *DOM* uz informacije dobijene tokom kompajliranja šablona.

Poseban slučaj predstavlja renderovanje listi, kod kojeg je važno očuvati identitet pojedinačnih elemenata pri promenama kolekcije. U *Angular*-u blok `@for` koristi izraz `track`, kojim se određuje vrednost na osnovu koje se prati identitet svakog ele-

menta. Stabilna i jedinstvena vrednost, poput identifikatora elementa, omogućava povezivanje postojećih prikaza sa odgovarajućim podacima i smanjuje potrebu za njihovim nepotrebnim uklanjanjem i ponovnim kreiranjem [4].

U *Vue.js*-u se liste renderuju pomoću direktive `v-for`, dok atribut `key` omogućava praćenje identiteta pojedinačnih elemenata. Navođenjem jedinstvene i stabilne vrednosti atributa `key`, na primer identifikatora elementa, postojeći *DOM* elementi mogu se pravilno povezati sa odgovarajućim podacima i, kada je potrebno, ponovo upotrebiti ili promeniti redosled [77].

U ovom aspektu pristupi su veoma slični: *Angular* koristi izraz `track`, a *Vue.js* atribut `key`, pri čemu oba mehanizma služe očuvanju identiteta elemenata i smanjenju nepotrebnih izmena prikaza.

## Rezultati prethodnog istraživanja

Olila i saradnici (2022) uporedili su strategije renderovanja radnih okvira *Angular*, *React*, *Vue.js*, *Svelte* i *Blazor* pomoću skupa eksperimenata kojima su ispitivali kako se troškovi renderovanja menjaju sa povećanjem složenosti aplikacije. Rezultati su pokazali značajne razlike u skaliranju troškova između analiziranih strategija, pri čemu relativne performanse zavise od količine podataka koju je potrebno obraditi tokom renderovanja [60].

Autori posebno izdvajaju pristupe koji smanjuju obim rada tokom renderovanja, između ostalog primenom optimizacija u vreme kompajliranja i reaktivnih modela programiranja [60]. Ovi nalazi potvrđuju značaj mehanizama kojima se ograničava nepotreban rad tokom ažuriranja korisničkog interfejsa, što je relevantno i za prethodno opisane pristupe u *Angular*-u i *Vue.js*-u.

## Zaključna razmatranja

Analiza pokazuje da *Angular* i *Vue.js* različitim mehanizmima nastoje da ograniče nepotreban rad pri ažuriranju korisničkog interfejsa. Razlike u načinu praćenja promena i renderovanja utiču na to kako radni okviri određuju delove prikaza koje je potrebno proveriti ili ažurirati.

Na osnovu analiziranih mehanizama i rezultata prethodnog istraživanja ne može se izvesti zaključak o univerzalnoj prednosti jednog radnog okvira u pogledu efi-

kasnosti renderovanja. Performanse zavise i od karakteristika konkretne aplikacije, količine podataka koja se obrađuje i načina njene implementacije. Zbog toga se ovaj kriterijum ne može posmatrati samo kroz pojedinačne mehanizme renderovanja, već u kontekstu konkretnih zahteva aplikacije.

### 3.5 Upravljanje stanjem

Pod stanjem aplikacije podrazumeva se skup podataka čije trenutne vrednosti utiču na ponašanje aplikacije i prikaz korisničkog interfejsa. Stanje može obuhvatati, na primer, podatke unete u obrazac, izabrane opcije, rezultate preuzete sa servera ili informacije koje koristi više komponenti. Upravljanje stanjem obuhvata načine na koje se takvi podaci čuvaju, menjaju, dele i održavaju usklađenim sa korisničkim interfejsom.

U jednostavnijim slučajevima stanje može ostati lokalno u okviru jedne komponente, dok se sa povećanjem broja komponenti i potrebe za korišćenjem istih podataka u različitim delovima aplikacije može javiti potreba za njihovim deljenjem i organizovanjem izvan pojedinačne komponente [84]. Poseban izazov predstavljaju podaci koji potiču iz asinhronih izvora, kao što su odgovori servera ili drugi događaji koji nastaju tokom izvršavanja aplikacije.

U ovom odeljku porede se pristupi koje *Angular* i *Vue.js* koriste za lokalno stanje, deljenje stanja između komponenti, rad sa asinhronim i reaktivnim podacima i centralizovano upravljanje stanjem.

#### Lokalno stanje

Pod lokalnim stanjem podrazumevaju se podaci koji pripadaju konkretnoj komponenti i čije se promene prvenstveno odnose na njeno ponašanje i prikaz, bez potrebe da budu organizovani kao zajedničko stanje aplikacije. Takvo stanje može obuhvatati, na primer, trenutno izabranu vrednost, podatke obrasca ili stanje pojedinih elemenata korisničkog interfejsa.

U *Angular*-u se reaktivno lokalno stanje može organizovati pomoću signala. Promenljivi signal kreira se funkcijom `signal()`, a njegova vrednost menja se metodama `set()` i `update()`. Za vrednosti izvedene iz postojećeg stanja koristi se `computed()`, kojim se definiše signal samo za čitanje čija vrednost zavisi od drugih signala [3].

Na ovaj način lokalno stanje i vrednosti izvedene iz njega mogu ostati neposredno povezani sa komponentom kojoj pripadaju, bez potrebe za uvođenjem posebnog servisa ili centralizovanog skladišta stanja.

U *Vue.js*-u se lokalno reaktivno stanje pri korišćenju *Composition API*-ja može definisati pomoću funkcija `ref()` i `reactive()`. Funkcija `ref()` kreira reaktivnu vrednost kojoj se u programskom kodu pristupa i koja se menja preko svojstva `value`, dok `reactive()` omogućava kreiranje reaktivnog objekta čija se svojstva čitaju i menjaju na uobičajen način [80]. Za vrednosti izvedene iz postojećeg reaktivnog stanja koristi se funkcija `computed()`, koja omogućava definisanje izračunate vrednosti čije se zavisnosti reaktivno prate [80].

Sa stanovišta lokalnog stanja, razlika se prvenstveno ogleda u programskom interfejsu koji se koristi za definisanje reaktivnih vrednosti: *Angular* koristi signale, dok *Vue.js* u okviru *Composition API*-ja koristi funkcije `ref()` i `reactive()`. Sam koncept lokalnog stanja pri tome ostaje sličan, jer su podaci organizovani u okviru komponente kojoj pripadaju.

## Deljenje stanja između komponenti

Kada je iste podatke potrebno koristiti u više komponenti, javlja se potreba za mehanizmima koji omogućavaju njihovo deljenje izvan granica pojedinačne komponente. Deljeno stanje predstavlja podatke kojima pristupa više komponenti, pri čemu je važno odrediti gde se ono čuva i kojim delovima aplikacije je dostupno.

U *Angular*-u se deljenje stanja može organizovati pomoću servisa i ugrađenog sistema za ubrizgavanje zavisnosti (engl. *Dependency Injection*, DI). Servis predstavlja izdvojenu klasu koja može da sadrži podatke i logiku povezanu sa njima, dok komponente kojima je servis potreban dobijaju njegovu instancu preko DI mehanizma [19].

Obim dostupnosti servisa zavisi od mesta na kojem je definisan njegov provajder, odnosno konfiguracija kojom se servis stavlja na raspolaganje DI sistemu. Servis dostupan na nivou aplikacije mogu koristiti različiti delovi aplikacije, dok servis obezbeđen na nivou određene komponente može biti ograničen na tu komponentu i odgovarajući deo hijerarhije njenih potomaka [21].

U *Vue.js*-u se stanje između komponente i njenih potomaka može deliti pomoću mehanizma `provide/inject`. Funkcija `provide()` omogućava komponenti da određenu vrednost učini dostupnom svojim potomcima, dok oni toj vrednosti pristupaju pomoću funkcije `inject()`, bez potrebe da se ona prosleđuje kroz svaki nivo hijerarhije komponenti [95].

Prosleđena vrednost može biti reaktivna, na primer vrednost kreirana funkcijom `ref()`, čime se zadržava reaktivna veza između komponente koja pruža stanje i komponenti koje ga koriste [95]. Jednostavno deljeno reaktivno stanje može se izdvojiti i u zaseban modul, odakle ga može koristiti više komponenti, neposredno ili kroz kompozabilnu funkciju [84].

Za složenije potrebe deljeno stanje može se organizovati pomoću namenskog rešenja kao što je *Pinia*, koje će biti razmotreno u nastavku.

Sa stanovišta deljenja stanja, ključna razlika ogleda se u načinu njegove organizacije. *Angular* deljeno stanje povezuje sa servisima i hijerarhijskim DI sistemom, dok *Vue.js* omogućava njegovo deljenje pomoću mehanizma `provide/inject` ili izdvajanja reaktivnog stanja izvan pojedinačnih komponenti.

## Rad sa asinhronim i reaktivnim podacima

Stanje aplikacije ne mora da se menja samo kao neposredna posledica korisničke interakcije, već može zavisiti i od asinhronih izvora podataka. Asinhroni podaci nisu dostupni odmah, već mogu stići naknadno, na primer kao odgovor servera ili kao rezultat događaja koji nastaju tokom izvršavanja aplikacije. U takvim slučajevima potrebno je organizovati tok promena podataka i obezbediti da komponente reaguju na njihove nove vrednosti.

U *Angular* ekosistemu važnu ulogu u radu sa asinhronim i reaktivnim podacima ima biblioteka *RxJS*. Jedan od osnovnih koncepata biblioteke *RxJS* jeste **Observable**, koji predstavlja tok vrednosti koje mogu pristizati tokom vremena. Komponente ili drugi delovi programa mogu da se pretplate na takav tok, odnosno da registruju logiku koja će reagovati kada **Observable** emituje novu vrednost [67].

U šablonima se vrednosti iz **Observable** tokova mogu koristiti pomoću `async` cevi (engl. *pipe*). Ona upravlja pretplatom na tok, prikazuje njegovu poslednju emitovanu vrednost i automatski prekida pretplatu kada se komponenta uništi [16].



*Angular* omogućava i povezivanje *RxJS* tokova sa signalima. Funkcija `toSignal()` omogućava da se vrednosti koje pristižu iz `Observable` toka koriste kao signal, dok `toObservable()` omogućava suprotan smer, odnosno pretvaranje signala u `Observable` [31]. Na taj način asinhroni podaci mogu se, u zavisnosti od potreba aplikacije, organizovati kroz *RxJS* tokove, signalni model ili njihovo međusobno povezivanje.

Pored toga, *Angular 22* pruža funkciju `resource()` za uključivanje asinhrono učitanih podataka u model zasnovan na signalima. Ona omogućava da se zajedno sa dobijenom vrednošću prate i stanje učitavanja i eventualna greška, što olakšava prikaz različitih stanja asinhrono operacije u korisničkom interfejsu [15].

U *Vue.js*-u asinhroni podaci mogu se povezati sa ugrađenim sistemom reaktivnosti korišćenjem reaktivnih vrednosti, kao što su `ref()` i `reactive()`. Podaci povezani sa asinhronom operacijom, kao što su rezultat zahteva, stanje učitavanja ili informacija o grešci, mogu se čuvati kao reaktivne vrednosti i menjati tokom izvršavanja operacije [74].

Kada je potrebno izvršiti dodatnu logiku kao reakciju na promenu reaktivnog stanja, *Vue.js* pruža funkcije `watch()` i `watchEffect()`. Funkcija `watch()` prati eksplicitno naveden izvor, dok `watchEffect()` automatski prati reaktivne zavisnosti korišćene tokom izvršavanja funkcije. Oba mehanizma mogu se koristiti za pokretanje dodatnih operacija kao reakcije na promenu stanja, na primer slanje zahteva serveru ili ažuriranje drugih podataka [101].

Logika koja obuhvata reaktivno stanje i povezane asinhrono operacije može se izdvojiti u kompozabilnu funkciju. Na taj način ista logika može da se organizuje izvan pojedinačne komponente i ponovo koristi u različitim delovima aplikacije [74].

U *Angular* ekosistemu za rad sa tokovima podataka koriste se *RxJS* tokovi predstavljeni pomoću `Observable` objekata, uz mogućnost njihovog povezivanja sa signalima, dok funkcija `resource()` omogućava organizovanje asinhronih podataka neposredno u signalnom modelu. U *Vue.js*-u asinhrono stanje može se organizovati kroz sistem reaktivnosti, funkcije `watch()` i `watchEffect()` i kompozabilne funkcije.

Razlika se, prema tome, ne odnosi na mogućnost rada sa asinhronim podacima, već na mehanizme kojima se oni organizuju i povezuju sa promenama stanja u svakom radnom okviru.

## Centralizovano upravljanje stanjem

Kada stanje koristi veći broj međusobno povezanih delova aplikacije, može se javiti potreba da se ono izdvoji iz pojedinačnih komponenti i organizuje kroz namenski sloj za upravljanje stanjem. Centralizovano upravljanje stanjem podrazumeva da se deljeno stanje i logika njegovih promena organizuju izvan pojedinačnih komponenti, čime se omogućava jasnije praćenje toka podataka kroz aplikaciju.

U *Angular* ekosistemu jedno od rešenja za upravljanje deljenim stanjem na nivou aplikacije predstavlja biblioteka *NgRx*. Njen *Store* predstavlja centralizovano skladište stanja iz kojeg različiti delovi aplikacije mogu da koriste potrebne podatke, dok se promene stanja organizuju kroz unapred definisan tok [57].

Osnovni elementi klasičnog *NgRx Store* modela su akcije (engl. *actions*), redukeri (engl. *reducers*) i selektori (engl. *selectors*). Akcije opisuju događaje koji nastaju u aplikaciji. Na osnovu trenutnog stanja i primljene akcije, reduker određuje novo stanje, dok selektori omogućavaju komponentama da izdvoje podatke koji su im potrebni iz zajedničkog stanja [57].

Na taj način komponenta ne menja zajedničko stanje neposredno, već šalje akciju, nakon čega se odgovarajuća promena obrađuje kroz definisani tok. Za operacije koje uključuju komunikaciju sa spoljnim sistemima, kao što je slanje zahteva serveru, *NgRx* pruža efekte (engl. *Effects*). Oni reaguju na akcije i omogućavaju da se takve operacije izdvoje iz redukera [56].

*NgRx* pruža i *SignalStore*, rešenje zasnovano na *Angular* signalima, u kojem se stanje, izvedene vrednosti i operacije nad stanjem mogu organizovati unutar posebnog *store*-a. Time je, pored klasičnog modela zasnovanog na akcijama i redukerima, dostupan i pristup upravljanju stanjem neposrednije povezan sa sistemom signala [58].

U *Vue.js* ekosistemu preporučeno rešenje za upravljanje deljenim stanjem na nivou aplikacije predstavlja biblioteka *Pinia*. Ona omogućava da se zajedničko stanje i logika povezana sa njegovim promenama izdvoje iz pojedinačnih komponenti i organizuju u posebne jedinice, odnosno *store*-ove. Takav *store* predstavlja zajedničko mesto za čuvanje i upravljanje podacima kojima mogu pristupati različiti delovi aplikacije [64].

*Pinia store* organizuje se oko tri osnovna elementa: stanja (engl. *state*), izvedenih vrednosti (engl. *getters*) i akcija (engl. *actions*). *State* predstavlja podatke koje *store*

čuva, *getters* omogućavaju izračunavanje vrednosti na osnovu postojećeg stanja, dok *actions* sadrže logiku za rad sa stanjem i mogu da menjaju njegove vrednosti ili izvršavaju asinhronne operacije [63].

*Pinia* omogućava definisanje *store*-a pomoću dva pristupa: *Option Store* koristi svojstva *state*, *getters* i *actions*, dok *Setup Store* koristi reaktivne vrednosti, izračunate vrednosti i funkcije [64].

Za razliku od ranijeg rešenja *Vuex*, *Pinia* ne uvodi poseban koncept mutacija. Zvanična dokumentacija preporučuje *Pinia*-u za nove aplikacije i ističe jednostavniji programski interfejs i bolju podršku za rad sa *TypeScript*-om [64].

*NgRx* i *Pinia* imaju sličnu osnovnu svrhu: izdvajanje zajedničkog stanja i logike povezane sa njegovim promenama iz pojedinačnih komponenti i njihovo organizovanje kroz namenske *store*-ove. Razlika se prvenstveno ogleda u stepenu formalizacije toka promena stanja.

Klasični *NgRx Store* model koristi eksplicitan tok zasnovan na akcijama, redukcima, selektorima i efektima, čime se promene stanja organizuju kroz jasno definisane korake. *Pinia* koristi jednostavniji model u kojem *store* objedinjuje stanje, izvedene vrednosti i akcije, bez posebnog koncepta mutacija.

*SignalStore* predstavlja alternativni *NgRx* pristup koji upravljanje stanjem neposrednije povezuje sa sistemom *Angular* signala.

### Zaključna razmatranja

Analiza upravljanja stanjem pokazuje da su razlike između *Angular*-a i *Vue.js*-a manje izražene kod lokalnog stanja, dok postaju uočljivije sa povećanjem obima deljenja stanja i složenosti tokova podataka. Oba radna okvira omogućavaju organizovanje reaktivnog stanja na nivou komponente, ali za deljenje i obradu složenijih tokova koriste različite programske apstrakcije.

*Angular* upravljanje stanjem povezuje sa signalima, servisima i DI sistemom, dok se za tokove podataka može koristiti *RxJS*, a za formalizovanje upravljanje stanjem rešenja iz *NgRx* ekosistema. *Vue.js* se oslanja na sopstveni sistem reaktivnosti, mehanizme za deljenje reaktivnih vrednosti i kompozabilne funkcije, dok *Pinia* pruža namensku organizaciju zajedničkog stanja kroz *store*-ove.

Ključna razlika zato se ne ogleda u tome da li radni okviri omogućavaju upravljanje različitim vrstama stanja, već u načinu na koji su mehanizmi za njegovo organizovanje povezani i u stepenu formalizacije toka promena stanja.

## 3.6 Ekosistem i alati

Pored karakteristika samog radnog okvira, na razvoj veb aplikacije utiču i alati i prateće biblioteke koji se koriste tokom njenog razvoja, testiranja, izgradnje i održavanja. U tom smislu, ekosistem radnog okvira obuhvata razvojne alate, biblioteke i proširenja koja dopunjuju njegove osnovne mogućnosti i podržavaju različite faze razvojnog procesa.

*Angular* ekosistem, pored osnovnih mehanizama radnog okvira, obuhvata zvanične alate i biblioteke namenjene različitim aspektima razvoja aplikacija [23]. *Vue.js* se u zvaničnoj dokumentaciji opisuje kao radni okvir i ekosistem projektovan tako da podrži različite načine upotrebe i postepeno uključivanje dodatnih mogućnosti u zavisnosti od potreba aplikacije [76].

U ovom odeljku pored se alati za inicijalizaciju i izgradnju projekata, razvoj i otklanjanje grešaka, testiranje i kontrolu kvaliteta koda, rutiranje, kao i prateća rešenja za izgradnju korisničkog interfejsa, internacionalizaciju i serversko i hibridno renderovanje. Poređenje je usmereno na rešenja koja neposredno utiču na tipičan razvojni tok i pokazuju razlike u načinu organizovanja *Angular* i *Vue.js* ekosistema, a ne na iscrpan pregled svih dostupnih biblioteka.

### Inicijalizacija projekta i proces izgradnje

Proces inicijalizacije projekta i alati za njegovu izgradnju određuju način na koji programer započinje razvoj aplikacije, upravlja njenom konfiguracijom i priprema je za izvršavanje u razvojnom i produkcionom okruženju. *Angular* i *Vue.js* u tu svrhu koriste različito organizovane razvojne alate, pa se u nastavku pored načini kreiranja projekta, rad sa komandnom linijom i savremeni proces izgradnje aplikacije.

Centralno mesto u razvojnom toku *Angular* aplikacija zauzima *Angular CLI*, zvanični alat komandne linije koji objedinjuje inicijalizaciju projekta, generisanje programskih elemenata, pokretanje razvojnog servera i izgradnju aplikacije. Novi projekat kreira se naredbom `ng new`, dok se za generisanje i izmenu elemenata aplikacije koristi naredba `ng generate`, zasnovana na sistemu *schematics*, odnosno unapred definisanim pravilima za automatsko generisanje i izmenu strukture projekta [27, 25].

Proces izgradnje i pokretanja aplikacije organizovan je kroz *builder*-e, odnosno unapred definisane postupke kojima *Angular CLI* izvršava zadatke kao što su iz-

gradnja ili pokretanje aplikacije. Kod novih projekata kreiranih naredbom `ng new`, podrazumevani `application builder` koristi `esbuild`, alat za brzo prevođenje i objedinjavanje programskog koda, pri izgradnji i optimizaciji aplikacije [17].

Tokom razvoja, *Angular CLI* pokreće razvojni server zasnovan na alatu *Vite*. *Angular* sistem za izgradnju generiše razvojnu verziju aplikacije, koja se zatim prosleđuje *Vite*-u kako bi bila dostupna u pregledaču.

Na taj način *Angular CLI* predstavlja zajednički interfejs za inicijalizaciju projekta, pokretanje razvojnog servera i izgradnju aplikacije, dok se u pozadini za pojedine zadatke koriste alati kao što su *esbuild* i *Vite* [33, 24].

Za inicijalizaciju novih *Vue.js* projekata preporučuje se `create-vue`, zvanični alat za kreiranje početne strukture projekta zasnovanog na *Vite*-u. Tokom inicijalizacije moguće je izabrati dodatne mogućnosti projekta, nakon čega se razvoj i izgradnja aplikacije odvijaju korišćenjem *Vite*-a [79].

Ranije korišćeni *Vue CLI* nalazi se u režimu održavanja, a zvanična dokumentacija za nove projekte preporučuje korišćenje `create-vue` i *Vite* [88].

*Vite* objedinjuje razvojni server i proces izgradnje produkcione verzije aplikacije. Razvojni server podržava brzu zamenu modula (engl. *Hot Module Replacement*, *HMR*), odnosno ažuriranje izmenjenih delova aplikacije bez potpunog ponovnog učitavanja stranice. Za produkcionu izgradnju *Vite* koristi *Rolldown*, alat za objedinjavanje modula aplikacije [71, 72].

Ključna razlika ogleda se u organizaciji razvojnog alatnog lanca. Kod *Angular*-a razvojni tok je u većoj meri objedinjen kroz *Angular CLI*, koji predstavlja zajednički interfejs za inicijalizaciju, razvoj i izgradnju projekta. Kod *Vue.js*-a `create-vue` služi za kreiranje početnog projekta, dok se dalji razvoj i izgradnja neposredno oslanjaju na *Vite*.

## Razvojni alati i otklanjanje grešaka

Pored alata za inicijalizaciju i izgradnju projekta, razvojni ekosistem obuhvata i alate namenjene inspekciji strukture aplikacije, praćenju njenog izvršavanja i otkrivanju problema tokom razvoja. *Angular* i *Vue.js* za te potrebe pružaju namenske razvojne alate integrisane sa alatima pregledača.

*Angular DevTools* je zvanična ekstenzija za razvojne alate pregledača namenjena pregledu strukture i analizi performansi *Angular* aplikacija. Omogućava pregled hijerarhije komponenti i direktiva, kao i pregled njihovih svojstava i stanja tokom izvršavanja aplikacije [7].

Pored pregleda komponenti, alat sadrži funkcionalnost *Profiler*, namenjenu analizi performansi aplikacije tokom izvršavanja. Njime se mogu analizirati ciklusi detekcije promena i vreme utrošeno na obradu pojedinačnih komponenti i direktiva. Na taj način moguće je utvrditi delove aplikacije koji učestvuju u konkretnom ciklusu detekcije promena i analizirati potencijalna uska grla pri ažuriranju korisničkog interfejsa [7, 29].

Savremeni *Angular DevTools* omogućava i pregled hijerarhije *injector*-a, odnosno elemenata DI sistema koji obezbeđuju zavisnosti komponentama i drugim delovima aplikacije, kao i dostupnih provajdera i konfigurisanog stabla ruta. Ove mogućnosti odgovaraju mehanizmima ubrizgavanja zavisnosti i rutiranja koji čine deo arhitekture *Angular* aplikacija [22, 30].

Za inspekciju i otklanjanje grešaka u *Vue.js* aplikacijama dostupan je zvanični alat *Vue DevTools*. Može se koristiti kao ekstenzija pregledača, dodatak za *Vite* ili kao samostalna aplikacija [89].

*Vue DevTools* omogućava pregled hijerarhije komponenti i njihovog stanja tokom izvršavanja aplikacije. U okviru prikaza komponenti moguće je analizirati njihove podatke i odnose u stablu komponenti, kao i neposredno pratiti promene stanja tokom razvoja [90].

Alat pruža i dodatne prikaze povezane sa strukturom aplikacije, uključujući pregled ruta, dok se njegove mogućnosti mogu proširivati integracijama sa bibliotekama iz *Vue.js* ekosistema [90].

*Angular DevTools* izlaže informacije o mehanizmima karakterističnim za *Angular*, kao što su detekcija promena, hijerarhija *injector*-a i struktura rutiranja. *Vue DevTools* je usmeren na pregled stabla komponenti, njihovog stanja i drugih elemenata *Vue.js* aplikacije, uz mogućnost integracije sa dodatnim delovima ekosistema.

## Testiranje i kvalitet koda

Alati za testiranje i statičku analizu koda predstavljaju deo razvojnog ekosistema koji omogućava proveru ponašanja aplikacije i rano otkrivanje potencijalnih

problema. Statička analiza podrazumeva proveru programskog koda bez njegovog izvršavanja, sa ciljem otkrivanja mogućih grešaka i odstupanja od definisanih pravila.

*Angular* i *Vue.js* pružaju podršku za ove aktivnosti, ali se razlikuju u načinu na koji su odgovarajući alati uključeni u njihov razvojni tok.

Novi projekti kreirani pomoću *Angular CLI*-ja podrazumevano koriste *Vitest* kao alat za izvršavanje jediničnih testova, dok se biblioteka *jsdom* koristi za simulaciju *DOM* okruženja pregledača bez pokretanja samog pregledača. Testovi se pokreću naredbom `ng test`, koja pri radu u interaktivnom terminalu podrazumevano prati promene datoteka i nakon izmene ponovo izvršava odgovarajuće testove [12].

Pored samog pokretača testova, *Angular* pruža sopstvene pomoćne mehanizme za testiranje delova aplikacije. *TestBed* omogućava kreiranje testnog okruženja u kojem se mogu konfigurisati komponente, servisi i njihove zavisnosti, dok *ComponentFixture* omogućava pristup instanci testirane komponente i njenom prikazu u testnom okruženju [34].

Statička analiza koda nije vezana za jedan podrazumevani alat instaliran u svakom novom *Angular CLI* projektu. Komanda `ng lint` postaje dostupna kada se projektu doda odgovarajući paket za statičku analizu, koji se zatim može uključiti u standardni *CLI* tok rada [26].

Za jedinično testiranje *Vue.js* aplikacija zasnovanih na *Vite*-u zvanična dokumentacija preporučuje *Vitest*. Pošto je prilagođen radu sa *Vite*-om i može da koristi njegovu konfiguraciju, *Vitest* se jednostavno uključuje u razvojno okruženje *Vue.js* projekta [85, 73].

Za testiranje komponenti dostupan je *Vue Test Utils*, zvanična biblioteka koja omogućava kreiranje instance komponente u testnom okruženju i interakciju sa njom. Na taj način moguće je proveravati renderovani sadržaj, prosleđena svojstva i reakcije komponente na korisničke interakcije [92].

Za statičku analizu *Vue.js* koda može se koristiti *ESLint* sa dodatkom *eslint-plugin-vue*, koji omogućava proveru `<template>` i `<script>` delova *Single-File Components* datoteka i primenu pravila specifičnih za *Vue.js* [38].

Kod jediničnog testiranja razlika se prvenstveno ogleda u pomoćnim mehanizmima koji prate sam radni okvir. *Angular* pruža sopstvene alate za konfigurisanje

testnog okruženja i rad sa komponentama i zavisnostima, dok *Vue.js* za testiranje komponenti koristi posebnu biblioteku *Vue Test Utils*.

Razlika postoji i u načinu uključivanja statičke analize u razvojni tok. Kod *Angular*-a ona se može povezati sa *Angular CLI*-jem dodavanjem odgovarajućeg paketa, dok se u *Vue.js* projektima može koristiti *ESLint* sa dodatkom *eslint-plugin-vue* za primenu pravila specifičnih za *Vue.js*.

## Prateće biblioteke i proširenje ekosistema

Pored osnovnih razvojnih alata, ekosistemi radnih okvira obuhvataju i biblioteke i proširenja namenjena funkcionalnostima koje su česte u savremenim veb aplikacijama. U nastavku se porede rešenja za rutiranje, izgradnju korisničkog interfejsa, internacionalizaciju i serversko i hibridno renderovanje, jer predstavljaju značajne delove pratećeg ekosistema savremenih veb aplikacija.

### Rutiranje

Rutiranje omogućava povezivanje *URL* adresa sa odgovarajućim prikazima aplikacije i upravljanje navigacijom bez ponovnog učitavanja cele stranice. Oba radna okvira imaju namenska rešenja za ovu funkcionalnost, ali se ona razlikuju u načinu uključivanja u osnovni razvojni ekosistem.

Za upravljanje navigacijom *Angular* koristi zvaničnu biblioteku *Angular Router*, dostupnu kroz paket `@angular/router`. Ona predstavlja osnovni deo radnog okvira i podrazumevano je uključena u projekte kreirane pomoću *Angular CLI*-ja [11].

Rutiranje se zasniva na konfiguraciji ruta koje povezuju *URL* putanje sa komponentama aplikacije, dok `RouterOutlet` određuje mesto u šablonu na kojem se prikazuje komponenta povezana sa trenutno aktivnom rutom.

Biblioteka podržava ugnježdene i dinamičke rute, kao i parametre ruta. Ugnježdene rute omogućavaju organizovanje ruta u hijerarhiju, dok dinamičke rute omogućavaju prikaz sadržaja na osnovu promenljivog dela *URL* putanje. Parametri ruta predstavljaju vrednosti koje se izdvajaju iz same *URL* adrese. Pored toga, podržani su programska navigacija i mehanizmi za kontrolu navigacije, kao što su zaštitnici ruta (engl. *route guards*), koji određuju da li korisnik može da pristupi određenoj ruti ili je napusti [11].



Za rutiranje u *Vue.js* aplikacijama koristi se *Vue Router*, zvanično rešenje za rutiranje na strani klijenta. Biblioteka se može dodati postojećem projektu kao poseban paket, dok alat `create-vue` omogućava da se podrška za rutiranje uključi već tokom inicijalizacije novog projekta [91].

Konfiguracija se zasniva na povezivanju *URL* putanja sa komponentama koje treba prikazati za odgovarajuću rutu. Komponenta `RouterView` određuje mesto u šablonu na kojem se prikazuje komponenta povezana sa trenutno aktivnom rutom, dok `RouterLink` omogućava kreiranje veza za navigaciju između ruta bez ponovnog učitavanja cele stranice.

*Vue Router* podržava i dinamičke i ugnježdene rute, parametre ruta, programsku navigaciju i zaštitnike navigacije (*navigation guards*), koji omogućavaju kontrolu toga da li se određena navigacija može izvršiti [91].

Osnovne mogućnosti *Angular Router*-a i *Vue Router*-a u velikoj meri se podudaraju i obuhvataju deklarativnu i programsku navigaciju, dinamičke i ugnježdene rute, parametre ruta i mehanizme za kontrolu toka navigacije.

Glavna razlika ogleda se u načinu njihovog uključivanja u ekosistem. Kod *Angular*-a rutiranje je deo osnovnog radnog okvira i njegovog razvojnog toka, dok se kod *Vue.js*-a *Vue Router* koristi kao poseban zvanični paket koji se uključuje prema potrebama projekta.

#### **Biblioteke za izgradnju korisničkog interfejsa**

Biblioteke komponenti omogućavaju korišćenje unapred implementiranih elemenata korisničkog interfejsa i zajedničkih obrazaca interakcije. U ekosistemima *Angular*-a i *Vue.js*-a ovakva rešenja postoje, ali se razlikuju po stepenu povezanosti sa samim radnim okvirom.

U okviru *Angular* ekosistema dostupni su *Angular Material* i *Component Dev Kit (CDK)*. *Angular Material* predstavlja biblioteku gotovih komponenti korisničkog interfejsa, dok *CDK* obezbeđuje osnovne funkcionalnosti i pomoćne mehanizme za izgradnju sopstvenih komponenti, bez unapred definisanog vizuelnog izgleda [10, 6].

*CDK* obuhvata mehanizme za česte obrasce ponašanja korisničkog interfejsa, među kojima su pristupačnost, prikaz sadržaja preko postojećeg interfejsa (engl. *overlay*), prevlačenje elemenata, upravljanje fokusom i prilagođavanje različitim veličinama prikaza. Zbog toga se *CDK* može koristiti i nezavisno od gotovih *Angular*

*Material* komponenti, pri razvoju prilagođenih komponenti aplikacije [6].

Za razliku od *Angular*-a, *Vue.js* nema jednu zvaničnu biblioteku komponenti korisničkog interfejsa koja predstavlja standardni deo njegovog ekosistema. Umesto toga, dostupno je više nezavisnih biblioteka koje se mogu uključiti u projekat u zavisnosti od njegovih zahteva i izabranog pristupa dizajnu.

Među takvim rešenjima nalaze se, na primer, *Vuetify* i *PrimeVue*. Obe biblioteke pružaju skup gotovih komponenti korisničkog interfejsa, ali se razvijaju kao zasebni projekti izvan osnovnog *Vue.js* radnog okvira [102, 65]. Izbor konkretne biblioteke zato predstavlja odluku na nivou projekta, a ne unapred određen deo *Vue.js* razvojnog toka.

Glavna razlika ogleda se u stepenu zvanične integracije rešenja za izgradnju korisničkog interfejsa u ekosistem radnog okvira. *Angular* raspolaže zvaničnim bibliotekama *Angular Material* i *CDK*, namenjenim gotovim komponentama i izgradnji prilagođenih elemenata korisničkog interfejsa.

Kod *Vue.js*-a ne postoji jedno zvanično *UI* rešenje sa istim položajem u okviru ekosistema, već programer bira između više nezavisnih biblioteka, kao što su *Vuetify* i *PrimeVue*, u skladu sa zahtevima konkretnog projekta.

#### Internacionalizacija i lokalizacija

Internacionalizacija omogućava pripremu aplikacije za različite jezike i regionalne formate, dok lokalizacija predstavlja prilagođavanje aplikacije konkretnom jeziku i regionu. *Angular* i *Vue.js* pružaju rešenja za ove potrebe, ali ih organizuju na različite načine.

*Angular* pruža zvaničnu podršku za internacionalizaciju kroz sopstveni sistem *i18n* (skraćeno od engl. *internationalization*) i paket `@angular/localize`. Tekst namenjen prevođenju može se označiti u šablonima ili programskom kodu, a označene poruke mogu se izdvojiti u datoteke za prevođenje pomoću alata *Angular CLI* [9].

Pored prevođenja tekstualnog sadržaja, *Angular* podržava prilagođavanje prikaza podataka izabranom jeziku i regionu. Ugrađeni mehanizmi omogućavaju odgovarajuće formatiranje datuma, brojeva, procenata i valuta, dok se lokalizovane verzije

aplikacije mogu generisati u okviru procesa izgradnje projekta [9].

U *Vue.js* aplikacijama internacionalizacija i lokalizacija mogu se realizovati pomoću biblioteke *Vue I18n*. Ona se instalira kao poseban paket i registruje u aplikaciji kao dodatak (engl. *plugin*), čime njene funkcionalnosti postaju dostupne komponentama aplikacije [42].

*Vue I18n* omogućava definisanje poruka za različite jezike i regione, promenu aktivnog jezika i organizovanje lokalizacije na globalnom nivou ili na nivou pojedinačnih komponenti. Pored tekstualnih poruka, podržano je i odgovarajuće formatiranje datuma, brojeva, procenata i valuta [42].

Glavna razlika ogleda se u stepenu integracije internacionalizacije i lokalizacije sa samim radnim okvirom. Kod *Angular*-a ove mogućnosti pripadaju zvaničnom ekosistemu i povezane su sa paketom `@angular/localize` i procesom izgradnje aplikacije.

Kod *Vue.js*-a internacionalizacija i lokalizacija se tipično dodaju pomoću zasebnog dodatka *Vue I18n*, koji se uključuje u projekat prema njegovim potrebama.

#### Serversko i hibridno renderovanje

Veb aplikacije mogu generisati sadržaj na različitim mestima i u različitim trenucima. Kod renderovanja na strani klijenta (engl. *Client-Side Rendering*, CSR) sadržaj se generiše u pregledaču, dok se kod renderovanja na strani servera (engl. *Server-Side Rendering*, SSR) *HTML* sadržaj generiše na serveru i zatim šalje pregledaču. Statičko generisanje (engl. *Static Site Generation*, SSG) podrazumeva da se *HTML* stranice generišu unapred, tokom procesa izgradnje aplikacije.

Hibridno renderovanje omogućava kombinovanje ovih pristupa u okviru iste aplikacije, u zavisnosti od potreba pojedinačnih delova sistema.

*Angular* pruža zvaničnu podršku za serversko i hibridno renderovanje kroz paket `@angular/ssr`. Ova podrška može se uključiti prilikom kreiranja novog projekta pomoću opcije `-ssr` ili naknadno dodati postojećoj aplikaciji pomoću *Angular CLI*-ja [32].

Hibridni model omogućava da se za pojedinačne rute odredi način renderovanja. Tako jedna ruta može koristiti *CSR*, druga *SSR*, dok se određene rute mogu unapred generisati korišćenjem *SSG*-a. Na taj način različiti pristupi mogu se kom-

binovati unutar iste aplikacije u skladu sa zahtevima njenih pojedinačnih delova [32].

*Vue.js* neposredno podržava renderovanje na strani servera kroz API za kreiranje serverski renderovane aplikacije i paket `vue/server-renderer`. Nakon što se *HTML* sadržaj generiše na serveru i pošalje pregledaču, na strani klijenta izvršava se hidracija, odnosno povezivanje postojećeg *HTML* sadržaja sa *Vue.js* logikom i događajima kako bi aplikacija postala interaktivna [83].

Izgradnja produkcione *SSR* aplikacije zahteva usklađivanje serverskog i klijentskog procesa izgradnje, rutiranja, pribavljanja podataka i upravljanja stanjem. Zbog toga zvanična *Vue.js* dokumentacija za ovakve scenarije preporučuje korišćenje radnih okvira višeg nivoa koji objedinjuju potrebne mehanizme za serversko renderovanje [83].

Jedno od takvih rešenja je *Nuxt*. Aktuelna generacija *Nuxt 4* podrazumevano koristi univerzalno renderovanje, pri kojem se početni *HTML* sadržaj generiše na serveru, a aplikacija zatim nastavlja da radi interaktivno u pregledaču. Pored toga, podržani su i klijentsko i hibridno renderovanje.

Pravila renderovanja mogu se definisati na nivou ruta, čime se pojedini delovi aplikacije mogu unapred generisati, renderovati na serveru ili izvršavati na strani klijenta u skladu sa zahtevima aplikacije [59].

Ključna razlika ogleda se u načinu organizovanja podrške za različite modele renderovanja. Kod *Angular*-a serversko, klijentsko i statičko renderovanje objedinjeni su u okviru zvaničnog ekosistema i mogu se kombinovati na nivou pojedinačnih ruta.

*Vue.js* pruža sopstvenu podršku za *SSR*, dok se za integrisano produkciono i hibridno renderovanje može koristiti radni okvir višeg nivoa, kao što je *Nuxt*, koji objedinjuje različite modele renderovanja i prateće mehanizme potrebne za njihov razvoj.

## Zaključna razmatranja

Analiza ekosistema i razvojnih alata pokazuje da *Angular* i *Vue.js* obezbeđuju rešenja za ključne faze razvoja savremenih veb aplikacija, dok se glavne razlike odnose na način na koji su ta rešenja organizovana i povezana sa samim radnim okvirom.

*Angular* se oslanja na integrisaniji razvojni tok, u kojem *Angular CLI* i zvanične biblioteke objedinjuju veliki deo funkcionalnosti potrebnih tokom razvoja. Nasuprot

tome, *Vue.js* primenjuje modularniji pristup, pri kojem se osnovni radni okvir dopunjuje zvaničnim i nezavisnim rešenjima u skladu sa potrebama konkretnog projekta.

Razlika između ova dva pristupa zato se ne ogleda prvenstveno u dostupnosti potrebnih funkcionalnosti, već u stepenu njihove integracije i slobodi pri izboru pratećih alata. Takva organizacija ekosistema može uticati na način formiranja tehnološkog skupa projekta, zbog čega se njen praktični značaj dodatno razmatra u narednom odeljku, u okviru pogodnosti radnih okvira za različite projektne kontekste.

### 3.7 Pogodnost u različitim projektnim kontekstima

Pogodnost radnog okvira za konkretan projekat ne može se proceniti samo na osnovu veličine aplikacije. Na izbor utiču i složenost zahteva, iskustvo razvojnog tima, lakoća usvajanja tehnologije, potreba za standardizacijom, modularnost i mogućnost proširivanja sistema u skladu sa zahtevima projekta [62].

Razlike između *Angular*-a i *Vue.js*-a analizirane u prethodnim odeljcima imaju različit značaj u zavisnosti od projektnog konteksta. Viši stepen unapred definisane strukture i integracije može biti koristan kada je potrebno uskladiti rad većeg broja programera i održati konzistentan razvojni tok, dok modularniji i progresivniji pristup može imati prednost u okruženjima u kojima su važni brz početak, prilagodljivost i postepeno uključivanje dodatnih mogućnosti.

Zbog toga se u ovom odeljku pogodnost dva radna okvira razmatra kroz nekoliko tipičnih projektnih konteksta: projekte sa prioritetom brzog razvoja i isporuke, poslovne aplikacije srednje složenosti, velike sisteme u kojima radi više razvojnih timova i postepenu integraciju u postojeće aplikacije. Cilj nije da se bilo koji radni okvir unapred veže za određenu vrstu ili veličinu projekta, već da se utvrdi u kojim okolnostima njegove prethodno analizirane karakteristike mogu predstavljati prednost ili ograničenje.

#### Projekti sa prioritetom brzog razvoja i isporuke

U pojedinim projektima osnovni cilj predstavlja što brže formiranje početne verzije aplikacije i njeno prilagođavanje na osnovu promenljivih zahteva. Takav kontekst je karakterističan za prototipe, minimalno održive proizvode (engl. *minimum viable*

*product*, *MVP*) i aplikacije kod kojih se deo zahteva definiše ili menja tokom razvoja. U ovim okolnostima poseban značaj imaju brzina usvajanja tehnologije, količina početne konfiguracije i mogućnost brzog menjanja strukture aplikacije.

Kod *Angular*-a veći broj međusobno povezanih koncepata i unapred definisanih obrazaca može povećati početne zahteve za tim koji prethodno nema iskustva sa ovim radnim okvirom. Arhitektura, ubrizgavanje zavisnosti, reaktivni mehanizmi i organizacija aplikacije uvode se već u početnim fazama razvoja, pa je potrebno ranije usvojiti veći broj koncepata i pravila rada.

Istovremeno, *Angular CLI* automatizuje inicijalizaciju projekta i generisanje njegovih elemenata [27, 25]. U kombinaciji sa unapred definisanim strukturom, to može smanjiti broj odluka koje je potrebno donositi pri uspostavljanju osnovne organizacije aplikacije. Zbog toga viši početni nivo strukture ne mora predstavljati ograničenje kada tim već poznaje *Angular* ili kada se očekuje da početna verzija aplikacije preraste u sistem koji zahteva dosledne razvojne obrasce.

U projektima u kojima su važni jednostavan početak i često eksperimentisanje sa strukturom aplikacije, ovakav nivo unapred definisane organizacije može predstavljati dodatno opterećenje. Njegov značaj se, međutim, menja kada su standardizacija i predvidljiva organizacija projekta važni već od početka razvoja.

*Vue.js* je projektovan kao progresivan radni okvir koji se može prilagoditi različitim nivoima složenosti aplikacije. Razvoj može započeti korišćenjem osnovnih mogućnosti radnog okvira, dok se dodatni alati i funkcionalnosti uključuju u skladu sa rastom zahteva projekta [76]. Ovakav pristup može biti značajan u projektima u kojima se početna struktura često menja i u kojima nije potrebno unapred definisati sve elemente buduće arhitekture.

Kod novih aplikacija, **create-vue** omogućava brzo formiranje projekta zasnovanog na alatu *Vite*, pri čemu se tokom inicijalizacije mogu izabrati potrebne dodatne mogućnosti projekta [79]. Time se početno razvojno okruženje može prilagoditi konkretnim zahtevima, bez potrebe da se svi delovi ekosistema uključe od samog početka razvoja.

U kontekstu prototipa i *MVP* aplikacija, ovakav modularni pristup omogućava da se početna implementacija zadrži relativno jednostavnom, a da se struktura i dodatni alati proširuju kako zahtevi postaju jasniji. Istovremeno, veća sloboda pri organizaciji projekta znači da tim mora samostalnije da donosi odluke o konvenci-

jama i izboru dodatnih rešenja kako aplikacija raste.

U projektima u kojima su prioritet brzo formiranje početne verzije, eksperimentisanje i prilagođavanje promenljivim zahtevima, modularniji pristup *Vue.js*-a može olakšati početak razvoja. *Angular*, sa druge strane, zahteva ranije upoznavanje sa sopstvenim obrascima, ali istovremeno od početka uspostavlja standardizovan način organizacije projekta.

Pogodnost jednog ili drugog pristupa zato u velikoj meri zavisi od iskustva tima i očekivanog razvoja projekta. *Vue.js* može imati prednost kada su važni postepeno usvajanje tehnologije i fleksibilno oblikovanje početne strukture, dok *Angular* može biti podjednako pogodan za brzu isporuku kada tim već poznaje njegove obrasce ili želi unapred definisanu organizaciju.

### Poslovne aplikacije i projekti srednje složenosti

Kako aplikacija raste, pored brzine razvoja sve veći značaj dobijaju jasna organizacija koda, doslednost razvojnih obrazaca i mogućnost proširivanja sistema uz očuvanje jasne i dosledne strukture. Ovakvi zahtevi mogu biti posebno relevantni kod internih poslovnih aplikacija, informacionih sistema i drugih projekata koji se razvijaju tokom dužeg perioda i postepeno dobijaju nove funkcionalnosti.

U ovakvom projektnom kontekstu viši stepen standardizacije koji pruža *Angular* može predstavljati prednost. Unapred definisana struktura aplikacije, sistem ubrizgavanja zavisnosti i razvojni tok organizovan kroz *Angular CLI* omogućavaju da različiti delovi aplikacije budu razvijani prema zajedničkim pravilima [19, 27].

Kako broj funkcionalnosti i programera raste, ovakav pristup može smanjiti broj arhitektonskih odluka koje se donose pojedinačno i olakšati održavanje ujednačene strukture projekta. Integrisaniji ekosistem dodatno omogućava da se rutiranje, testiranje, internacionalizacija i izgradnja korisničkog interfejsa organizuju pomoću međusobno povezanih alata iz istog *Angular* ekosistema.

U poslovnim aplikacijama srednje složenosti *Vue.js* omogućava postepeno proširivanje strukture projekta u skladu sa rastom njegovih zahteva. Osnovni radni okvir može se dopuniti zvaničnim rešenjima za rutiranje i upravljanje stanjem, kao i dodatnim bibliotekama iz ekosistema, pri čemu tim zadržava veću slobodu u izboru konkretnih alata i načina organizacije aplikacije.

Ovakva modularnost može biti korisna kada projekat zahteva veću prilagodljivost i kada nije potrebno unapred definisati sva pravila organizacije aplikacije. Istovremeno, veća sloboda izbora znači da tim mora jasnije definisati sopstvene konvencije kako bi se tokom rasta aplikacije zadržala dosledna organizacija koda.

Kod poslovnih aplikacija srednje složenosti razlika se prvenstveno ogleda u stepenu unapred definisane organizacije. *Angular* pruža standardizovaniji razvojni tok i veći broj međusobno povezanih mehanizama, dok *Vue.js* ostavlja više prostora za izbor strukture i dodatnih rešenja prema potrebama konkretnog projekta.

Izbor zato zavisi od toga da li tim veći značaj pridaje unapred definisanim pravilima i doslednosti razvojnog toka ili većoj prilagodljivosti pri organizaciji projekta.

## Veliki sistemi i rad više razvojnih timova

Kod velikih sistema tehnička složenost aplikacije često je praćena i organizacionom složenošću razvoja. Kada na istoj kodnoj bazi radi veći broj programera ili više razvojnih timova, poseban značaj dobijaju zajedničke konvencije, predvidljiva struktura projekta i mogućnost da različiti delovi sistema budu razvijani na dosledan način [62].

U ovakvom okruženju viši stepen standardizacije koji karakteriše *Angular* može olakšati usklađivanje rada većeg broja programera. Unapred definisana struktura aplikacije, sistem ubrizgavanja zavisnosti i razvojni tok kroz *Angular CLI* pružaju zajedničku osnovu na kojoj timovi mogu organizovati različite delove sistema.

Prednost ovakvog pristupa nije u tome što *Angular* sam po sebi garantuje kvalitet ili održivost velike aplikacije, već u tome što smanjuje razlike u načinu organizovanja sličnih delova sistema. Kada se razvoj odvija paralelno u više timova, unapred definisana pravila mogu olakšati održavanje doslednosti između različitih delova aplikacije.

*Vue.js* se može koristiti i u velikim aplikacijama, ali njegova veća fleksibilnost znači da je potrebno više pravila organizacije definisati na nivou projekta ili organizacije. Kako raste broj timova, postaje važnije uspostaviti zajedničke konvencije za organizaciju komponenti, upravljanje stanjem, izbor dodatnih biblioteka i strukturu projekta.



Ovakav pristup omogućava da se razvojno okruženje prilagodi specifičnim potrebama sistema, ali istovremeno zahteva jasnije definisana interna pravila i dogovore između timova. Zbog toga pogodnost *Vue.js*-a u velikim sistemima u većoj meri zavisi od toga koliko dosledno organizacija definiše i primenjuje zajednička pravila razvoja.

U velikim sistemima u kojima radi više razvojnih timova razlika između dva pristupa prvenstveno se ogleda u tome gde se nalazi odgovornost za standardizaciju. *Angular* veći deo zajedničke strukture i pravila organizacije pruža kroz sam radni okvir i njegov ekosistem, dok *Vue.js* ostavlja više prostora da se ta pravila definišu na nivou konkretnog projekta ili organizacije.

To ne znači da je jedan radni okvir ograničen na određenu veličinu aplikacije. Veći stepen unapred definisane strukture može biti prednost kada je cilj uskladiti rad više timova, dok veća fleksibilnost može biti korisna kada organizacija želi da razvojno okruženje prilagodi sopstvenim pravilima i načinu rada.

## Postepena integracija u postojeće aplikacije

Izbor radnog okvira značajan je i kada se nova tehnologija uvodi u već postojeću aplikaciju. U takvom kontekstu važni su mogućnost postepene integracije i obim promena koje je potrebno napraviti u postojećoj strukturi sistema.

*Angular* pruža unapred definisanu strukturu i skup povezanih razvojnih mehanizama. Takav pristup odgovara situacijama u kojima se novi deo postojećeg sistema može razvijati kao zaokružena celina prema *Angular* pravilima i organizaciji.

Pored toga, paket `@angular/elements` omogućava da se *Angular* komponenta pretvori u prilagođeni *HTML* element (engl. *custom element*). Nakon registracije, takva komponenta može se koristiti na stranici slično standardnom *HTML* elementu, što omogućava uključivanje pojedinačnih *Angular* komponenti i u aplikacije koje nisu u celini razvijene pomoću *Angular*-a [18].

Kada se *Angular* uvodi kao osnova za širu funkcionalnu celinu, njegova unapred definisana struktura može zahtevati veće prilagođavanje postojećeg sistema. Ako se nova celina može razvijati relativno nezavisno, ista struktura može olakšati njen dalji razvoj.

Progresivni pristup *Vue.js*-a posebno dolazi do izražaja kada se radni okvir uvo-  
di u postojeću aplikaciju. Njegove mogućnosti mogu se uključivati postepeno, od  
pojedinačnih komponenti do šireg dela aplikacije, bez potrebe da se čitav postojeći  
sistem odmah prilagodi novom razvojnom pristupu [76].

*Vue.js* se zato može najpre primeniti na ograničenom delu korisničkog interfejsa,  
dok se dodatni mehanizmi i biblioteke uključuju kako se njegova uloga u aplikaci-  
ji proširuje. Time se smanjuje potreba za istovremenom promenom cele postojeće  
aplikacije.

U kontekstu postepene integracije glavna razlika ogleda se u načinu uključivanja  
radnog okvira u postojeći sistem. *Vue.js* omogućava da se njegova uloga postepeno  
proširuje od pojedinačnih delova interfejsa ka većem delu aplikacije, dok *Angular*  
omogućava i uključivanje pojedinačnih komponenti pomoću prilagođenih *HTML*  
elemenata.

Kada se uvodi šira funkcionalna celina, *Angular* češće podrazumeva organizova-  
nje tog dela sistema prema sopstvenoj strukturi, dok *Vue.js* ostavlja više prostora  
za postepeno prilagođavanje postojećeg okruženja.

### Sažeti pregled razlika relevantnih za izbor

Prethodno razmatrani projektni konteksti pokazuju da pogodnost radnog okvira  
zavisí od zahteva projekta i karakteristika razvojnog tima. Ključne razlike relevantne  
pri izboru sažete su u tabeli 3.1.

Tabela 3.1: Pregled kriterijuma relevantnih pri izboru između Angular-a i Vue.js-a

| Kriterijum                            | Angular                                                                                                                                                                        | Vue.js                                                                                                                |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Početak razvoja                       | Veći broj unapred definisanih koncepata i obrazaca, uz standardizovan razvojni tok.                                                                                            | Postepenije uvođenje koncepata i mogućnost naknadnog uključivanja dodatnih rešenja.                                   |
| Standardizacija                       | Viši stepen unapred definisane strukture i zajedničkih razvojnih obrazaca.                                                                                                     | Veća sloboda u definisanju strukture i projektnih konvencija.                                                         |
| Fleksibilnost izbora alata            | Veći deo tipičnog razvojnog toka organizovan je kroz povezana rešenja iz ekosistema.                                                                                           | Modularniji pristup omogućava izbor dodatnih biblioteka prema zahtevima projekta.                                     |
| Rad više razvojnih timova             | Unapred definisani obrasci mogu olakšati održavanje konzistentnosti između timova.                                                                                             | Veća fleksibilnost povećava potrebu za jasnim definisanjem zajedničkih konvencija na nivou projekta ili organizacije. |
| Postepena integracija                 | Omogućava uključivanje pojedinačnih komponenti putem prilagođenih HTML elemenata, kao i razvoj širih funkcionalnih celina prema sopstvenoj strukturi i pravilima organizacije. | Progresivni model omogućava postepeno uključivanje i proširivanje njegove uloge u postojećoj aplikaciji.              |
| Prilagođavanje promenljivim zahtevima | Promene se realizuju unutar standardizovane strukture i unapred definisanih obrazaca.                                                                                          | Modularniji pristup omogućava prilagođavanje strukture i dodatnih rešenja prema zahtevima projekta.                   |

## Zaključna razmatranja

Analiza različitih projektnih konteksta pokazuje da se pogodnost *Angular*-a i *Vue.js*-a ne može odrediti samo na osnovu veličine aplikacije. Značaj pojedinih karakteristika radnog okvira menja se u zavisnosti od zahteva projekta, iskustva razvojnog tima, potrebe za standardizacijom i stepena fleksibilnosti koji je potreban tokom razvoja.

*Angular* može imati prednost u okruženjima u kojima su važni unapred definisana organizacija, zajednička pravila razvoja i veći stepen integracije pratećih mehanizama. Takve karakteristike mogu olakšati usklađivanje rada više programera i

održavanje dosledne strukture projekta. *Vue.js*, sa druge strane, pruža veću slobodu pri oblikovanju strukture i izboru dodatnih rešenja, što može biti značajno kada su važni postepeno uvođenje tehnologije, prilagođavanje promenljivim zahtevima i integracija u postojeće aplikacije.

Nijedna od ovih karakteristika sama po sebi ne određuje da je jedan radni okvir pogodniji za određenu veličinu ili vrstu projekta. Njihov praktični značaj zavisi od toga koliko se način organizacije konkretnog radnog okvira poklapa sa zahtevima projekta i načinom rada razvojnog tima. Pored početnog izbora tehnologije, značajan kriterijum predstavlja i način na koji se aplikacija može održavati i razvijati tokom dužeg vremenskog perioda, što se razmatra u narednom odeljku.

### 3.8 Održavanje i dugoročna stabilnost

Dugoročno održavanje aplikacije ne zavisi samo od stabilnosti programskih interfejsa radnog okvira, već i od načina na koji se objavljuju njegove nove verzije, uvode promene i podržavaju nadogradnje postojećih projekata. Poseban značaj imaju predvidljivost razvoja radnog okvira, dostupnost migracionih mehanizama i kompatibilnost sa pratećim alatima i bibliotekama.

Razlike između *Angular*-a i *Vue.js*-a u ovom kriterijumu proizlaze iz njihove politike izdanja i podrške, načina uvođenja promena i organizacije pratećeg ekosistema. Ovi faktori utiču na to kako razvojni timovi planiraju nadogradnje i upravljaju verzijama i zavisnostima tokom životnog ciklusa projekta.

Zbog toga se dugoročna stabilnost u ovom odeljku razmatra kroz politiku izdanja i podrške, nadogradnje i migracije, stabilnost jezgra i kompatibilnost pratećeg ekosistema, odnosno kroz njihov uticaj na održavanje projekta tokom dužeg vremenskog perioda.

#### Politika izdanja i podrške

*Angular* primenjuje semantičko verzionisanje, kod kojeg brojevi glavne, manje i zakrpe verzije ukazuju na vrstu promena koje izdanje donosi. Od verzije 22 predviđen je ciklus u kojem se nova glavna verzija objavljuje približno jednom godišnje, uz više manjih izdanja tokom njenog životnog ciklusa [14].

Glavne verzije se uobičajeno podržavaju ukupno 24 meseca. Prvih 12 meseci predstavlja period aktivne podrške, tokom kojeg se objavljuju redovna ažuriranja i

ispravke, dok narednih 12 meseci predstavlja period dugoročne podrške (engl. *Long-Term Support, LTS*), u kojem se objavljuju kritične ispravke i bezbednosne zakrpe [14]. Ovako definisan raspored omogućava razvojnim timovima da unapred znaju koliko dugo će određena glavna verzija biti podržana i da u skladu sa tim planiraju prelazak na naredne verzije.

*Vue.js* nema unapred definisan ciklus objavljivanja novih verzija. Ispravke se objavljuju prema potrebi, dok se manje verzije sa novim funkcionalnostima tipično objavljuju u razmacima od tri do šest meseci. Glavne verzije najavljuju se unapred i prolaze kroz faze razvoja i preprodukcijских izdanja pre objavljivanja stabilne verzije [81].

Izdanja *Vue.js*-a prate semantičko verzionisanje, uz određene dokumentovane izuzetke, među kojima su promene definicija tipova za *TypeScript*. Funkcionalnosti označene kao zastarele nastavljaju da budu podržane tokom iste glavne verzije, a njihovo uklanjanje predviđa se tek u narednoj glavnoj verziji [81].

Za razliku od *Angular*-a, *Vue.js* nema unapred određen period aktivne i dugoročne podrške za svaku glavnu verziju. Predvidljivost se zato u većoj meri zasniva na pravilima semantičkog verzionisanja, prethodnom najavljivanju većih promena i preprodukcijским fazama izdanja.

Ključna razlika ogleda se u predvidljivosti rasporeda izdanja i podrške. *Angular* unapred definiše ritam glavnih izdanja i trajanje aktivne i dugoročne podrške, što timovima olakšava planiranje nadogradnji u određenom vremenskom okviru.

*Vue.js*, sa druge strane, predvidljivost zasniva na semantičkom verzionisanju, najavljivanju većih promena i zadržavanju zastarelih funkcionalnosti do naredne glavne verzije. Pošto vreme podrške nije unapred određeno za svaku glavnu verziju, razvojni timovi moraju redovnije da prate objavljene promene i u skladu sa njima planiraju nadogradnje.

## Nadogradnje, zastarevanje *API*-ja i migracije

Za nadogradnju postojećih projekata *Angular* pruža zvanični mehanizam kroz komandu `ng update` i vodič *Angular Update Guide*. Komanda `ng update` ažurira odgovarajuće pakete i može automatski da primeni potrebne izmene u postojećem kodu radi prilagođavanja novijoj verziji, dok vodič pruža uputstva prilagođena početnoj i ciljnoj verziji projekta [13].

Promene koje narušavaju kompatibilnost uvode se u skladu sa definisanom politikom zastarevanja. Kada se programski interfejs ili funkcionalnost označi kao zastarela, ona ostaje dostupna najmanje tokom naredne glavne verzije, odnosno najmanje približno godinu dana, a njeno uklanjanje može se izvršiti tek u glavnom izdanju [14]. Time se timovima ostavlja period u kojem postojeći kod mogu postepeno da prilagode preporučenom načinu korišćenja.

Pri prelasku preko više glavnih verzija preporučuje se nadogradnja redom, jednu glavnu verziju po jednu, čime se zadržava strukturisan put prelaska između verzija [14].

Za prelazak između glavnih verzija *Vue.js* pruža migracione vodiče u kojima su dokumentovane promene koje narušavaju kompatibilnost i preporučeni načini prilagođavanja postojećeg koda. Posebno detaljan migracioni put obezbeđen je za prelazak sa *Vue.js* 2 na *Vue.js* 3, za koji dokumentacija obuhvata pregled promenjenih i uklonjenih programskih interfejsa i preporuke za njihovu zamenu [87].

Za isti prelazak razvijen je i paket *@vue/compat*, odnosno *Migration Build*. Reč je o posebnoj verziji *Vue.js* 3 koja može da obezbedi ponašanje kompatibilno sa *Vue.js* 2 i da upozori na korišćenje funkcionalnosti koje su u novoj verziji promenjene ili zastarele. Kompatibilnost pojedinih funkcionalnosti može se podešavati i na nivou komponente, što omogućava da se prilagođavanje postojećeg projekta obavlja postepeno [78].

Kod *Vue.js*-a migracioni proces se u većoj meri oslanja na migracione vodiče i mehanizme predviđene za konkretne prelaze između glavnih verzija, umesto na jednu centralizovanu *CLI* komandu za primenu migracija.

Ključna razlika ogleda se u stepenu automatizacije i formalizacije migracionog procesa. *Angular* pruža centralizovaniji put nadogradnje kroz `ng update` i zvanični vodič, dok se *Vue.js* u većoj meri oslanja na migracione vodiče i mehanizme prilagođene konkretnim prelascima između glavnih verzija.

Zbog toga je kod *Angular*-a proces nadogradnje više objedinjen kroz standardni razvojni tok, dok kod *Vue.js*-a u većoj meri zavisi od prirode promena između konkretnih verzija.

## Stabilnost jezgra i kompatibilnost ekosistema

Stabilnost *Angular*-a zasniva se i na pravilima kompatibilnosti javnih program-skih interfejsa između izdanja. Njihove promene uvode se u skladu sa politikom semantičkog verzionisanja i zastarevanja, dok su manje verzije kompatibilne sa prethodnim izdanjima iste glavne verzije [14].

Pored kompatibilnosti sopstvenih paketa, *Angular* objavljuje zvanične zahteve za verzije važnih zavisnosti, kao što su *Node.js*, *TypeScript* i *RxJS*. Za svaku podržanu verziju radnog okvira dokumentovane su kompatibilne verzije ovih zavisnosti, čime se timu olakšava usklađivanje razvojnog okruženja prilikom nadogradnje projekta [36].

Ovakva politika ne uklanja potrebu da se proverí usklađenost svih biblioteka koje projekat koristi, naročito nezavisnih paketa, ali jasno definisane podržane verzije osnovnih zavisnosti smanjuju neizvesnost prilikom održavanja *Angular* razvojnog okruženja.

Kod *Vue.js*-a kompatibilnost jezgra prati pravila semantičkog verzionisanja, uz određene dokumentovane izuzetke. Jedan od njih odnosi se na definicije tipova za *TypeScript*, koje u manjim verzijama *Vue.js*-a mogu zahtevati prilagođavanje zbog promena u *TypeScript*-u ili korišćenja njegovih novijih mogućnosti [81].

Kompatibilnost je značajna i između pojedinih delova samog *Vue.js* okruženja. Zvanična dokumentacija navodi da novija manja verzija kompajlera, odnosno dela koji obrađuje i priprema kôd aplikacije, može generisati rezultat koji nije kompatibilan sa starijom manjom verzijom izvršnog dela radnog okvira.

U standardnim aplikacijama ove verzije su usklađene, dok je ova razlika posebno relevantna pri razvoju i distribuciji unapred prevedenih biblioteka komponenti [81].

Pored jezgra, *Vue.js* koristi i zasebno verzionisane alate i biblioteke iz svog ekosistema. Zbog toga je tokom održavanja projekta potrebno pratiti usklađenost ne samo verzije samog radnog okvira, već i pratećih rešenja koja projekat koristi.

Kod pojedinih zvaničnih alata verzije se usklađuju sa jezgrom. Na primer, `@vue/compiler-sfc` objavljuje se u istoj verziji kao glavni `vue` paket [86].

Ključna razlika ogleda se u načinu upravljanja kompatibilnošću razvojnog okruženja. *Angular* preciznije definiše podržane verzije ključnih zavisnosti, čime deo odluka o njihovom usklađivanju unapred standardizuje.

Kod *Vue.js*-a kompatibilnost jezgra prati semantičko verzionisanje, dok modu-

larniji ekosistem zahteva pažljivije praćenje usklađenosti alata i biblioteka koje konkretan projekat koristi.

### Uticaj na dugoročno održavanje projekta

Dugoročno održavanje projekta ne zavisi samo od politike izdanja radnog okvira, već i od načina na koji su organizovani aplikacija, razvojni alati i prateće zavisnosti. Prethodno analizirane razlike u politici podrške, migracijama i organizaciji ekosistema zato neposredno utiču na način održavanja *Angular* i *Vue.js* projekata tokom vremena.

Kod *Angular*-a viši stepen standardizacije, povezanost zvaničnih alata i definisan put nadogradnje mogu olakšati održavanje u projektima u kojima je važno da se promene sprovede prema zajedničkim pravilima. Centralizovaniji pristup smanjuje broj odluka koje tim mora samostalno da donosi pri usklađivanju razvojnog okruženja, ali zahteva redovno praćenje podržanih verzija i pravovremeno planiranje nadogradnji.

Kod *Vue.js*-a modularniji pristup omogućava da se pojedini delovi razvojnog okruženja biraju i unapređuju u skladu sa zahtevima projekta. Takva fleksibilnost može olakšati postepeno prilagođavanje sistema, ali povećava značaj upravljanja verzijama i kompatibilnošću dodatnih biblioteka koje projekat koristi. Zbog toga dugoročno održavanje u većoj meri zavisi od pravila koja tim uspostavi za izbor, ažuriranje i zamenu pratećih rešenja.

Razlika se zato ne ogleda u tome da li je dugoročno održavanje moguće, već u načinu na koji se odgovornost za njegovu organizaciju raspodeljuje. Kod *Angular*-a veći deo pravila i procesa nadogradnje unapred je definisan kroz radni okvir i njegov ekosistem, dok kod *Vue.js*-a veća fleksibilnost zahteva da tim pažljivije upravlja izborom i usklađivanjem pratećih rešenja.

### Zaključna razmatranja

Analiza održavanja i dugoročne stabilnosti pokazuje da *Angular* i *Vue.js* pružaju mehanizme za upravljanje promenama tokom životnog ciklusa projekta, ali se razlikuju u stepenu formalizacije tih procesa.



*Angular* se oslanja na unapred definisanu politiku izdanja i podrške, dokumentovane zahteve kompatibilnosti i centralizovaniji postupak nadogradnje. Takav pristup omogućava predvidljivije planiranje prelaska između verzija, ali istovremeno zahteva redovno praćenje definisanog životnog ciklusa podrške. Kod *Vue.js*-a predvidljivost se u većoj meri zasniva na semantičkom verzionisanju, dokumentovanju promena i migracionim mehanizmima prilagođenim konkretnim verzijama, dok modularniji ekosistem zahteva pažljivije upravljanje pratećim zavisnostima.

Dugoročna stabilnost zato ne zavisi samo od učestalosti novih izdanja ili trajanja podrške, već i od toga koliko razvojni tim dosledno prati promene, planira nadogradnje i održava usklađenost korišćenih alata i biblioteka. Razlike između dva radna okvira prvenstveno se ogledaju u tome koliko su ovi procesi unapred definisani kroz sam ekosistem, a koliko njihova organizacija ostaje odgovornost razvojnog tima.

### 3.9 Zaključna razmatranja poglavlja

Uporedna analiza pokazuje da *Angular* i *Vue.js* omogućavaju realizaciju ključnih zahteva savremenih veb aplikacija, ali se razlikuju u načinu na koji organizuju razvoj i povezuju mehanizme dostupne programeru. Najizraženija razlika ne odnosi se zato na samu dostupnost određenih funkcionalnosti, već na stepen unapred definisane strukture, integracije alata i slobode koju razvojni tim ima pri organizovanju projekta.

*Angular* primenjuje integrisaniji i standardizovaniji pristup. Struktura aplikacije, ubrizgavanje zavisnosti, razvojni alati, upravljanje stanjem i veliki deo pratećeg ekosistema povezani su kroz zajedničke mehanizme i konvencije. Takva organizacija podrazumeva ranije usvajanje većeg broja povezanih koncepata, ali istovremeno pruža predvidljiviji razvojni tok i više unapred definisanih pravila. *Vue.js* primenjuje progresivniji i modularniji pristup, u kojem se osnovne mogućnosti mogu koristiti sa manjim početnim skupom koncepata, dok se dodatna rešenja uključuju u skladu sa rastom zahteva aplikacije. Time se ostavlja više prostora za prilagođavanje strukture i izbor pratećih alata, ali i veća odgovornost razvojnog timu da definiše sopstvene konvencije.

Na nivou pojedinačnih tehničkih mehanizama razlike se ispoljavaju na različite načine. Sintaksa šablona i organizacija komponenti koriste različite programske interfejse, dok sistemi reaktivnosti i renderovanja primenjuju različite strategije za praćenje promena i ažuriranje korisničkog interfejsa. Na osnovu tih razlika ne mo-

že se izdvojiti jedan radni okvir kao univerzalno efikasniji, jer performanse zavise i od karakteristika aplikacije i načina njene implementacije. Slično tome, upravljanje stanjem moguće je organizovati od lokalnog do centralizovanog nivoa u oba radna okvira, ali se razlikuju mehanizmi i stepen formalizacije kojima se tok promena stanja uređuje.

Praktični značaj ovih razlika zavisi od projektnog konteksta. Veći stepen standardizacije može biti koristan kada su važni zajednička pravila, predvidljiva organizacija i usklađivanje rada većeg broja programera, dok modularniji pristup može imati prednost kada su značajni brz početak, postepeno uvođenje tehnologije i prilagođavanje postojećem sistemu. Dugoročno održavanje dodatno zavisi od politike izdanja, načina migracije i upravljanja zavisnostima: *Angular* ove procese u većoj meri formalizuje kroz sopstveni razvojni tok, dok *Vue.js* ostavlja više prostora za modularno upravljanje pratećim delovima ekosistema.

Na osnovu analiziranih kriterijuma ne može se izdvojiti univerzalno pogodniji radni okvir. Izbor između *Angular*-a i *Vue.js*-a zavisi od zahteva projekta, iskustva i organizacije razvojnog tima, potrebnog stepena standardizacije i fleksibilnosti, kao i načina na koji se očekuje da se aplikacija razvija i održava tokom vremena. Teorijske razlike utvrđene u ovom poglavlju predstavljaju osnovu za praktično poređenje u narednom poglavlju, u kojem se funkcionalno ekvivalentna aplikacija implementira u oba radna okvira.

## Glava 4

# Praktična implementacija

Cilj praktičnog dela rada je da na konkretnom primeru prikaže kako se iste funkcionalnosti mogu implementirati u dva savremena *frontend* radna okvira – *Angular* i *Vue.js*. Za potrebe praktičnog poređenja razvijena je aplikacija za upravljanje zadacima, implementirana paralelno u oba radna okvira.

Obe implementacije zasnovane su na istom skupu funkcionalnih zahteva. Funkcionalnosti, ponašanje i korisnički interfejs obe aplikacije međusobno su usklađeni, dok se razlike odnose na način organizacije i implementacije kôda. Ovakav pristup omogućava neposredno poređenje načina na koji *Angular* i *Vue.js* rešavaju iste zahteve u razvoju veb aplikacija.

Kompletan kôd obe implementacije dostupan je u javnom repozitorijumu na platformi *GitHub*: <https://github.com/alettrailo/master-thesis-task-manager>.

Domen aplikacije odabran je tako da bude jednostavan, a opet dovoljno bogat da omogući prikaz ključnih funkcionalnosti:

- upravljanje zadacima - pregled, kreiranje, izmenu i brisanje;
- pretragu, filtriranje i sortiranje zadataka;
- rad sa formama i validaciju korisničkog unosa;
- autentifikaciju korisnika i kontrolu pristupa pojedinim rutama;
- navigaciju između stranica i obradu nepostojećih ruta.

Praktično poređenje usmereno je na one aspekte analizirane u prethodnim poglavljima koji se mogu neposredno sagledati kroz implementaciju funkcionalno ekvi-

valentnih aplikacija, pre svega na organizaciju komponenti, rad sa šablonima i korisničkim interakcijama, upravljanje stanjem i rutiranje.

### 4.1 Postavljanje projekata

Za potrebe praktične implementacije kreirana su dva odvojena projekta, jedan u radnom okviru *Angular*, a drugi u radnom okviru *Vue.js*. Projekti su inicijalizovani pomoću zvaničnih alata odgovarajućih ekosistema, pri čemu su zadržane savremene postavke karakteristične za korišćene verzije radnih okvira.

Za praktičnu implementaciju korišćeni su *Angular* 22.1.1 i *Vue.js* 3.5.41. Prva aplikacija razvijena je u jeziku *TypeScript*, a druga u jeziku *JavaScript*. Ovakav izbor prati način upotrebe oba radna okvira opisan u prethodnim poglavljima: *TypeScript* predstavlja standardni jezik razvoja aplikacija u *Angular*-u, dok *Vue.js* omogućava njegovu upotrebu, ali je ne zahteva.

U obe implementacije korišćena je biblioteka *Bootstrap* 5.3.8, kako bi se smanjio uticaj različitih pristupa stilizaciji i omogućilo formiranje funkcionalno i vizuelno usklađenog korisničkog interfejsa. Obe aplikacije razvijene su u istom razvojnom okruženju, uz *Node.js* 22.22.3.

#### Postavljanje *Vue.js* projekta

Za inicijalizaciju *Vue.js* projekta korišćen je zvanični alat *create-vue*. Projekat je kreiran naredbom:

```
npm create vue@latest
```

Tokom inicijalizacije uključeni su *Vue Router*, namenjen rutiranju unutar aplikacije, i *Pinia*, namenjena upravljanju zajedničkim stanjem. Projekat je realizovan u *JavaScript*-u, dok dodatne mogućnosti koje nisu potrebne za praktično poređenje, kao što su *JSX* i automatsko testiranje, nisu uključene. Kreiran je početni projekat bez demonstracionog koda, koji predstavlja osnovu za implementaciju funkcionalnosti opisanih u nastavku poglavlja.

Nakon generisanja početne strukture instalirane su zavisnosti projekta i biblioteka *Bootstrap*:

```
cd vue-aplikacija  
npm install
```

```
npm install bootstrap
```

Biblioteka *Bootstrap* uključena je globalno u ulaznoj datoteci `main.js`:

```
import 'bootstrap/dist/css/bootstrap.min.css'
```

Projekat kreiran pomoću alata *create-vue* zasnovan je na alatu *Vite*, koji se koristi za razvoj i izgradnju aplikacije. Razvojni server pokrenut je naredbom:

```
npm run dev
```

U generisanom projektu korišćeni su *Vue.js* 3.5.41, *Vue Router* 5.2.0, *Pinia* 4.0.2 i *Vite* 8.2.1. Nakon pokretanja razvojnog servera provereno je da se početna aplikacija uspešno izvršava u veb pregledaču.

### Postavljanje *Angular* projekta

*Angular* projekat kreiran je pomoću alata *Angular CLI* verzije 22. Za inicijalizaciju projekta korišćena je naredba:

```
npx @angular/cli@22 new angular-aplikacija  
  --routing  
  --style=scss  
  --skip-tests  
  --skip-git
```

Opcijom `--routing` uključena je početna podrška za rutiranje, dok je *SCSS* izabran kao format stilova. Generisanje testnih datoteka je izostavljeno, jer automatsko testiranje nije deo praktičnog poređenja sprovedenog u ovom poglavlju. Kreiranje zasebnog *Git* repozitorijuma unutar *Angular* projekta takođe je izostavljeno, budući da se obe implementacije nalaze u zajedničkom repozitorijumu praktičnog dela rada.

Prilikom inicijalizacije nije uključeno serversko renderovanje niti statičko generisanje stranica. Aplikacija je realizovana kao klijentska jednostranična aplikacija, što odgovara funkcionalnostima koje se razmatraju u praktičnom delu.

Generisana struktura *Angular* 22 projekta zasniva se na savremenom *standalone* pristupu. Pokretanje korenske komponente vrši se funkcijom `bootstrapApplication`, dok su konfiguracija aplikacije i definicije ruta izdvojene u datoteke `app.config.ts` i `app.routes.ts`. Na taj način nije potrebno definisanje korenskog modula `AppModule`.

Nakon kreiranja projekta instalirana je biblioteka *Bootstrap*:

```
cd angular-aplikacija  
npm install bootstrap
```

*Bootstrap* stilovi uključeni su globalno u datoteci `styles.scss`:

```
@import 'bootstrap/dist/css/bootstrap.min.css';
```

Razvojni server zatim je pokrenut naredbom:

```
npm run start
```

U *Angular* implementaciji korišćeni su *Angular* 22.1.1, *Angular CLI* 22.1.3, *TypeScript* 6.0.3 i *RxJS* 7.8.2. Generisana aplikacija koristi podrazumevani model *Angular*-a 22 bez korišćenja biblioteke *ZoneJS*, kao i podrazumevanu *OnPush* strategiju detekcije promena. Obe karakteristike opisane su u prethodnom poglavlju i nisu naknadno uvedene posebnom konfiguracijom, već predstavljaju deo načina rada korišćene verzije radnog okvira.

Nakon pokretanja razvojnog servera provereno je da se početna *Angular* aplikacija uspešno izvršava u veb pregledaču.

## 4.2 Implementacija funkcionalnosti i poređenje

U nastavku je prikazana implementacija funkcionalnosti aplikacije za upravljanje zadacima u radnim okvirima *Angular* i *Vue.js*. Prikaz je organizovan prema funkcionalnim celinama, tako da se za svaku od njih najpre opisuje njena uloga u aplikaciji, zatim način realizacije u oba radna okvira, a potom se razmatraju najznačajnije sličnosti i razlike između primenjenih pristupa.

Obe implementacije zasnovane su na istim funkcionalnim zahtevima i ostvaruju isto ponašanje sa stanovišta korisnika. Razlike se odnose na organizaciju koda, način upravljanja stanjem, komunikaciju između komponenti, rad sa formama, rutiranje i druge mehanizme karakteristične za svaki od posmatranih radnih okvira.

Radi preglednosti, u radu nisu prikazane kompletne izvorne datoteke obe aplikacije, već odabrani delovi koda koji su neposredno relevantni za implementaciju posmatrane funkcionalnosti i poređenje pristupa u dva radna okvira. Delovi koji se odnose prvenstveno na stilizovanje korisničkog interfejsa, ponavljajuću strukturu šablona ili druge detalje koji ne utiču značajno na poređenje izostavljeni su iz

prikazanih listinga. Na taj način pažnja je usmerena na mehanizme koji najbolje ilustruju razlike između *Angular*-a i *Vue.js*-a.

Korisnički interfejs je u obe implementacije usklađen, tako da iste funkcionalnosti imaju jednak vizuelni prikaz i način korišćenja. Zbog toga se slike ekrana koriste kao reprezentativni prikaz rezultata implementacije, dok se razlike između radnih okvira analiziraju prvenstveno na osnovu njihove unutrašnje organizacije i načina realizacije funkcionalnosti.

### Rutiranje i navigacija

Rutiranje u obe aplikacije organizovano je tako da korisniku obezbedi isti skup stranica i isti način prelaska između funkcionalnih celina. Definisane su početna stranica, stranica za prijavu, stranica sa listom zadataka, stranice za dodavanje, prikaz detalja i izmenu zadatka, kao i stranica koja se prikazuje u slučaju nepostojeće rute. Na taj način je struktura aplikacija sa stanovišta korisnika ujednačena, dok se razlike odnose na mehanizme koje radni okviri koriste za konfigurisanje ruta i prikaz odgovarajućih komponenti.

U obe implementacije korišćene su sledeće putanje:

- `/` – početna stranica,
- `/prijava` – prijava korisnika,
- `/zadaci` – pregled zadataka,
- `/zadaci/novi` – dodavanje novog zadatka,
- `/zadaci/:id` – prikaz detalja izabranog zadatka,
- `/zadaci/:id/izmeni` – izmena izabranog zadatka,
- nepostojeće putanje – stranica sa greškom 404.

Putanje koje se odnose na rad sa zadacima dostupne su samo prijavljenim korisnicima. Početna stranica i stranica za nepostojeće rute dostupne su svim korisnicima, dok je stranica za prijavu namenjena neprijavljenim korisnicima. Ukoliko prijavljeni korisnik pokuša da pristupi stranici za prijavu, preusmerava se na stranicu sa zadacima.

Pravila pristupa rutama realizovana su odgovarajućim mehanizmima za zaštitu ruta, koji su detaljnije opisani u odeljku posvećenom autentifikaciji i zaštiti ruta.

### Angular implementacija

U aplikaciji implementiranoj u radnom okviru *Angular*, rute su definisane u datoteci `app.routes.ts` korišćenjem tipa `Routes`. Svakoj putanji pridružena je komponenta koja se prikazuje kada korisnik pristupi odgovarajućoj adresi. Dinamički segment `:id` koristi se za identifikaciju zadatka pri prikazu njegovih detalja i izmeni, dok je putanja `**` namenjena presretanju svih nepostojećih ruta.

Reprezentativni deo konfiguracije prikazan je u kodu 4.1.

```
export const routes: Routes = [
  {
    path: '',
    component: Pocetna
  },
  {
    path: 'zadaci',
    component: Zadaci,
    canActivate: [autentifikacijaGuard]
  },
  {
    path: 'zadaci/:id',
    component: DetaljiZadatka,
    canActivate: [autentifikacijaGuard]
  },
  {
    path: '**',
    component: NijePronadjeno
  }
];
```

Kôd 4.1: Primer definisanja ruta u *Angular* aplikaciji

U korenskom šablonu aplikacije koristi se komponenta `router-outlet`, koja predstavlja mesto na kojem *Angular Router* prikazuje komponentu povezanu sa trenutno aktivnom rutom:

```
<app-navigacija />
<router-outlet />
```

### Vue.js implementacija

U *Vue.js* aplikaciji rutiranje je realizovano korišćenjem biblioteke *Vue Router*. Ruter se kreira funkcijom `createRouter`, a konfiguracija ruta prosleđuje se kroz niz `routes`. Pored putanje i odgovarajuće komponente, rutama su dodeljena i imena



koja se koriste pri programskoj navigaciji. Za dinamičke putanje takode se koristi parametar `:id`, dok je nepostojeća ruta definisana izrazom `/:pathMatch(.*)*`.

Odgovarajući deo konfiguracije prikazan je u kodu 4.2.

```
const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'pocetna',
      component: PocetnaView
    },
    {
      path: '/zadaci',
      name: 'zadaci',
      component: ZadaciView,
      meta: { requiresAuth: true }
    },
    {
      path: '/zadaci/:id',
      name: 'detalji-zadatka',
      component: DetaljiZadatkaView,
      meta: { requiresAuth: true }
    },
    {
      path: '/*',
      name: 'nije-pronadjeno',
      component: NijePronadjenoView
    }
  ]
});
```

Kôd 4.2: Primer definisanja ruta u *Vue.js* aplikaciji

Mesto za prikaz komponente aktivne rute u korenskoj `App.vue` komponenti predstavlja `RouterView`:

```
<Navigacija />
<RouterView />
```

Pored prelaska između stranica, obe aplikacije sadrže responzivnu navigacionu traku. Ona se prikazuje samo prijavljenom korisniku i omogućava prelazak na početnu stranicu i listu zadataka, prikaz e-mail adrese prijavljenog korisnika i odjavu. Za izgled navigacione trake korišćene su klase biblioteke *Bootstrap*, dok je stanje mobilnog menija kontrolisano neposredno kroz mehanizme reaktivnosti posmatranih

radnih okvira.

U *Angular* implementaciji lokalno stanje otvorenosti menija predstavlja signal:

```
readonly meniOtvoren = signal(false);

promeniMeni(): void {
  this.meniOtvoren.update(otvoren => !otvoren);
}
```

Njegova vrednost se u šablonu koristi za uslovno dodavanje klase **show**:

```
[class.show]="meniOtvoren()"
```

U *Vue.js* implementaciji ista lokalna informacija čuva se pomoću funkcije **ref**:

```
const meniOtvoren = ref(false)

function promeniMeni() {
  meniOtvoren.value = !meniOtvoren.value
}
```

U šablonu se odgovarajuća klasa vezuje direktivom **:class**:

```
:class="{ show: meniOtvoren }"
```

### Poređenje pristupa

Na ovaj način je ponašanje navigacije u obe aplikacije jednako, ali je vidljiva razlika u načinu izražavanja lokalnog reaktivnog stanja. *Angular* koristi signal čija se vrednost čita pozivom signala i menja metodama kao što su **set** i **update**, dok se u *Vue.js* za istu namenu koristi **ref**, čijoj se vrednosti u JavaScript delu komponente pristupa preko svojstva **value**. U samom *Vue.js* šablonu dodatno pristupanje svojstvu **value** nije potrebno.

Posmatrano na nivou rutiranja, obe implementacije omogućavaju statičke i dinamičke putanje, prikaz odgovarajuće komponente za aktivnu rutu, deklarativnu i programsku navigaciju, kao i obradu nepostojećih putanja. Glavna razlika je u organizaciji korišćenih mehanizama: *Angular Router* predstavlja deo integrisanog *Angular* ekosistema i konfiguracija je zasnovana na objektima tipa **Routes**, dok se u *Vue.js* koristi zvanična biblioteka *Vue Router*, koja se zasebno kreira i povezuje sa aplikacijom. Sa stanovišta korisnika, ove razlike nisu vidljive, jer obe implementacije ostvaruju jednaku navigacionu strukturu i ponašanje.

## Autentifikacija i zaštita ruta

Autentifikacija korisnika u obe aplikacije realizovana je na klijentskoj strani sa ciljem da se omogući demonstracija kontrole pristupa pojedinim delovima aplikacije. Implementacija ne predstavlja produkcionu autentifikacionu mehanizam, već služi za poređenje načina upravljanja stanjem prijavljenog korisnika, validacije podataka i zaštite ruta u posmatranim radnim okvirima.

Za potrebe demonstracije definisani su unapred određeni podaci za prijavu. Nakon uspešne prijave, e-mail adresa korisnika čuva se u objektu `localStorage` pod ključem `prijavljeniKorisnik`. Na taj način stanje prijave ostaje sačuvano i nakon ponovnog učitavanja stranice. Odjavljivanjem korisnika odgovarajući podatak se uklanja iz lokalnog skladišta, a reaktivno stanje aplikacije se ažurira.

### *Angular* implementacija

U *Angular* aplikaciji logika autentifikacije izdvojena je u poseban servis. Servis `ServisAutentifikacije` čuva e-mail adresu prijavljenog korisnika u signalu. Informacija o tome da li je korisnik prijavljen izvodi se pomoću funkcije `computed`, dok je ostatku aplikacije izložena samo verzija signala namenjena za čitanje.

Stranica za prijavu u *Angular* implementaciji koristi reaktivnu formu kreiranu pomoću `NonNullableFormBuilder`-a. Polje za e-mail adresu je obavezno i proverava se validatorom `email`, dok lozinka mora sadržati najmanje šest karaktera. Ukoliko forma nije ispravna, sva polja se označavaju kao dodirnuti kako bi se korisniku prikazale odgovarajuće poruke o greškama. Nakon uspešne prijave korisnik se preusmerava na prvobitno zahtevanu zaštićenu rutu ili, ukoliko ona nije zabeležena, na stranicu sa zadacima.

Reprezentativni deo servisa prikazan je u kodu 4.3.

```
const KLJUC_KORISNIKA = 'prijavljeniKorisnik';

@Service()
export class ServisAutentifikacije {
  private readonly _email = signal<string | null>(localStorage.getItem(KLJUC_KORISNIKA));

  readonly email = this._email.asReadonly();
  readonly prijavljen = computed(() => this._email() !== null);

  prijavi(email: string, lozinka: string): boolean {
    if (email !== 'korisnik@primer.rs' || lozinka !== 'master123') {
      return false;
    }

    localStorage.setItem(KLJUC_KORISNIKA, email);
    this._email.set(email);
    return true;
  }

  odjavi(): void {
    localStorage.removeItem(KLJUC_KORISNIKA);
    this._email.set(null);
  }
}
```

Kôd 4.3: Upravljanje stanjem autentifikacije u *Angular* aplikaciji

Ključni deo obrade prijave prikazan je u kodu 4.4.

```
prijavi(): void {
  this.greska.set('');

  if (this.forma.invalid) {
    this.forma.markAllAsTouched();
    return;
  }
  const podaci = this.forma.getRawValue();
  const uspesno = this.servisAutentifikacije.prijavi(podaci.email.trim(), podaci.lozinka);

  if (!uspesno) {
    this.greska.set('Prijava nije uspela. Proverite e-mail i lozinku.');
```

Kôd 4.4: Obrada prijave u *Angular* aplikaciji

Za kontrolu pristupa rutama koriste se dva funkcionalna čuvara ruta. Čuvar `autentifikacijaGuard` proverava da li je korisnik prijavljen pre pristupa stranici namenjenim radu sa zadacima. Ukoliko korisnik nije prijavljen, formira se preusmeravanje ka stranici za prijavu, pri čemu se prvobitno zahtevana putanja prosleđuje kroz parametar `redirect`.

```
export const autentifikacijaGuard: CanActivateFn = (_, state) => {
  const servisAutentifikacije = inject(ServisAutentifikacije);
  const router = inject(Router);

  if (servisAutentifikacije.prijavljen()) {
    return true;
  }

  return router.createUrlTree(['/prijava'], {
    queryParams: { redirect: state.url }
  });
};
```

Kôd 4.5: Zaštita ruta za prijavljene korisnike u *Angular* aplikaciji

Drugi čuvar, `neprijavljenGuard`, primenjuje obrnuto pravilo na stranicu za prijavu. Neprijavljenom korisniku pristup je dozvoljen, dok se već prijavljeni korisnik preusmerava na stranicu sa zadacima. Time se sprečava ponovno prikazivanje forme za prijavu korisniku koji već ima aktivno stanje prijave.

### **Vue.js implementacija**

U *Vue.js* aplikaciji zajedničko stanje autentifikacije organizovano je pomoću biblioteke *Pinia*. `Store useAutentifikacijaStore` definisan je korišćenjem kompozicionog pristupa. E-mail adresa korisnika čuva se pomoću funkcije `ref`, dok se svojstvo `prijavljen` izvodi pomoću funkcije `computed`. Operacije prijave i odjave menjaju i reaktivno stanje i vrednost sačuvanu u `localStorage-u`.

Za razliku od *Angular* implementacije, forma za prijavu u *Vue.js* aplikaciji nije zasnovana na posebnom *API*-ju za forme. Vrednosti polja čuvaju se u objektu kreiranom funkcijom `reactive`, dok se stanje validnosti izvodi pomoću `computed` vrednosti. Povezivanje polja sa stanjem forme realizovano je direktivom `v-model`. Posebno se prati i da li je korisnik već stupio u interakciju sa poljem, kako bi se poruke o neispravnom unosu prikazale u odgovarajućem trenutku.

Reprezentativni deo *store*-a prikazan je u kodu 4.6.

```
export const useAutentifikacijaStore = defineStore(
  'autentifikacija',
  () => {
    const email = ref(localStorage.getItem(KLJUC_KORISNIKA));
    const prijavljen = computed(() => email.value !== null);

    function prijavi(unetiEmail, lozinka) {
      if (unetiEmail !== 'korisnik@primer.rs' || lozinka !== 'master123') {
        return false;
      }

      localStorage.setItem(KLJUC_KORISNIKA, unetiEmail);
      email.value = unetiEmail;
      return true;
    }

    function odjavi() {
      localStorage.removeItem(KLJUC_KORISNIKA);
      email.value = null;
    }

    return { email, prijavljen, prijavi, odjavi };
  }
);
```

Kôd 4.6: Upravljanje stanjem autentifikacije u *Vue.js* aplikaciji

Deo validacije forme prikazan je u kodu 4.7.

```
const forma = reactive({
  email: '',
  lozinka: ''
});

const dodirnuto = reactive({
  email: false,
  lozinka: false
});

const emailNeispravan = computed(() => {
  const email = forma.email.trim();
  return email === '' || !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
});

const lozinkaNeispravna = computed(
  () => forma.lozinka.length < 6
);
```

Kôd 4.7: Validacija prijave u *Vue.js* aplikaciji

Zaštita ruta ostvarena je globalnim navigacionim čuvarom. Funkcija `beforeEach` proverava stanje autentifikacije pre svakog prelaska. Zaštićene rute označene su svojevremeno `meta.requiresAuth`, a kada pristup nije dozvoljen vraća se odgovarajuća ciljna ruta.

```
router.beforeEach((to) => {
  const autentifikacijaStore = useAutentifikacijaStore();

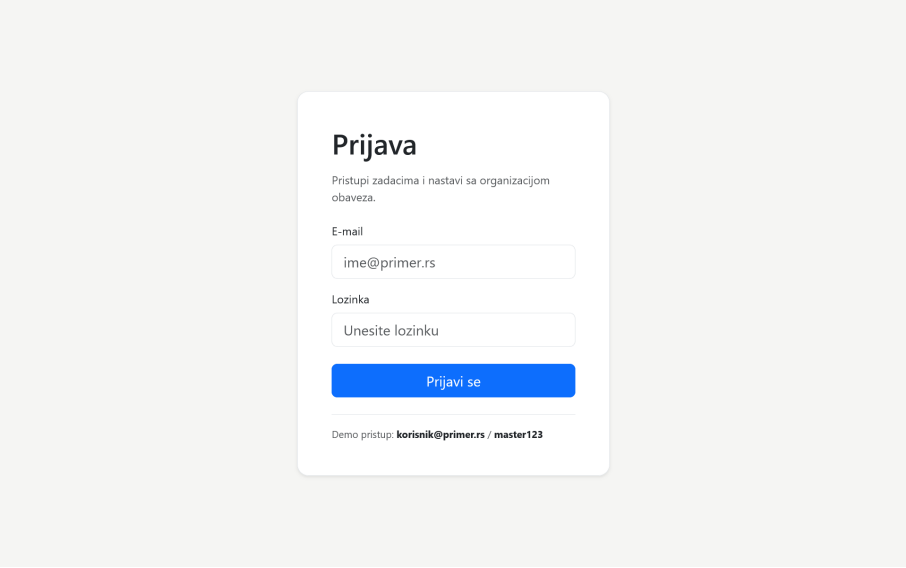
  if (to.name === 'prijava' && autentifikacijaStore.prijavljen) {
    return { name: 'zadaci' };
  }

  if (to.meta.requiresAuth && !autentifikacijaStore.prijavljen) {
    return {
      name: 'prijava',
      query: { redirect: to.fullPath }
    };
  }
});
```

Kôd 4.8: Zaštita ruta u *Vue.js* aplikaciji

Globalni čuvar u *Vue.js* aplikaciji objedinjuje pravila koja su u *Angular* implementaciji razdvojena između `autentifikacijaGuard`-a i `neprijavljenGuard`-a. Parametar `redirect` u obe implementacije omogućava povratak na prvobitno zahtevanu stranicu nakon uspešne prijave.

Jednak prikaz stranice za prijavu dat je na slici 4.1.



Slika 4.1: Stranica za prijavu korisnika

### Poređenje pristupa

Sa stanovišta funkcionalnosti, dve implementacije ostvaruju isto ponašanje: čuvaju stanje prijavljenog korisnika, omogućavaju prijavu i odjavu, sprečavaju neovlašćen pristup stranicama sa zadacima i preusmeravaju korisnika na prvobitno zahtevanu rutu nakon uspešne prijave. Razlika je prvenstveno u organizaciji mehanizama. *Angular* koristi servis ubrizgan u komponente i funkcionalne čuvare ruta tipa `CanActivateFn`, dok je u *Vue.js* zajedničko stanje izdvojeno u *Pinia store-u*, a pravila pristupa objedinjena su u globalnom čuvaru `beforeEach`.

Razlika je vidljiva i pri radu sa formom za prijavu. *Angular* obezbeđuje strukturisan *API* reaktivnih formi i ugrađene validatore, dok je u ovoj *Vue.js* implementaciji stanje forme i validacija eksplicitno organizovana korišćenjem reaktivnih vrednosti i izračunatih svojstava. Iako se unutrašnja realizacija razlikuje, obe implementacije sa stanovišta korisnika ostvaruju isto ponašanje i pružaju odgovarajuće povratne informacije pri neispravnom unosu.

### Upravljanje stanjem i podacima o zadacima

Centralni deo obe aplikacije predstavlja skup podataka o zadacima i operacije koje omogućavaju njihovo pronalaženje, dodavanje, izmenu i brisanje. Pošto je cilj praktične implementacije poređenje radnih okvira, podaci o zadacima nisu povezani sa udaljenim serverom ili bazom podataka, već se čuvaju u memoriji aplikacije. Ponovnim učitavanjem aplikacije skup zadataka se zato vraća na početno definisano stanje.

U obe implementacije svaki zadatak sadrži identifikator, naziv, opis i status. Status može imati jednu od dve vrednosti, koje predstavljaju aktivan ili završen zadatak. Iako je struktura podataka funkcionalno jednaka, način njenog definisanja i organizovanja zajedničkog stanja razlikuje se između dve implementacije.

#### *Angular* implementacija

U *Angular* aplikaciji struktura zadatka eksplicitno je definisana korišćenjem mogućnosti jezika *TypeScript*. Tip `StatusZadatak` ograničava dozvoljene vrednosti statusa, dok interfejs `Zadatak` određuje svojstva koja svaki objekat zadatka mora da sadrži.



```
export type StatusZadatka = 'aktivan' | 'završen';

export interface Zadatak {
  id: number;
  naziv: string;
  opis: string;
  status: StatusZadatka;
}
```

Kôd 4.9: Model zadatka u *Angular* aplikaciji

Zajedničko stanje zadataka izdvojeno je u servis **ServisZadataka**. Interni niz zadataka čuva se pomoću signala `_zadaci`, dok se ostatku aplikacije izlaže njegova verzija namenjena samo za čitanje. Na taj način komponente mogu reaktivno da koriste trenutno stanje, dok se njegove izmene obavljaju kroz metode samog servisa.

Pored prikazanih metoda, servis sadrži i operacije **izmeni** i **obrisi**. Prilikom izmene koristi se metoda `update` signala i kreira se novi niz u kojem se odgovarajući zadatak zamenjuje izmenjenim podacima. Brisanje se realizuje filtriranjem postojećeg niza prema identifikatoru zadatka.

Identifikator novog zadatka određuje se pronalaženjem najveće postojeće vrednosti atributa `id` i njenim uvećavanjem za jedan. Ovakvo rešenje odgovara potrebama aplikacije u kojoj se podaci nalaze isključivo u memoriji.

Servis se u komponentama i stranicama koristi putem *Angular* mehanizma ubrizgavanja zavisnosti. Na primer, stranica sa zadacima dobija instancu servisa funkcijom `inject`, nakon čega koristi javno izloženo stanje:

```
private readonly servisZadataka = inject(ServisZadataka);

readonly zadaci = this.servisZadataka.zadaci;
```

Na ovaj način zajednički podaci i operacije nad njima ostaju izdvojeni iz komponenti koje su odgovorne za njihov prikaz i korisničku interakciju.

Osnovna organizacija servisa prikazana je u kodu 4.10.

```
@Service()
export class ServisZadataka {
  private readonly _zadaci = signal<Zadatak[]>([
    {
      id: 1,
      naziv: 'Naučiti Angular',
      opis: 'Proći kroz osnovne koncepte i praktičnu implementaciju.',
      status: 'aktivan'
    },
    {
      id: 2,
      naziv: 'Naučiti Vue.js',
      opis: 'Implementirati iste funkcionalnosti u Vue.js aplikaciji.',
      status: 'završen'
    }
  ]);

  readonly zadaci = this._zadaci.asReadonly();

  pronadjiPoId(id: number): Zadatak | undefined {
    return this._zadaci().find(zadatak => zadatak.id === id);
  }

  dodaj(zadatak: Omit<Zadatak, 'id'>): void {
    const noviId =
      Math.max(0, ...this._zadaci().map(zadatak => zadatak.id)) + 1;

    this._zadaci.update(zadaci => [
      ...zadaci,
      { id: noviId, ...zadatak }
    ]);
  }
}
```

Kôd 4.10: Upravljanje stanjem zadataka u *Angular* aplikaciji

### **Vue.js implementacija**

U *Vue.js* aplikaciji zajedničko stanje zadataka izdvojeno je u *Pinia store*. *Store useZadaciStore* koristi se za organizovanje zajedničkog stanja i operacija nad zadacima. Budući da je ova implementacija napisana u jeziku *JavaScript*, ne postoji zaseban interfejs koji statički definiše strukturu objekta zadatka. Ipak, objekti koriste jednaka svojstva i jednake vrednosti statusa kao u *Angular* aplikaciji.

*Store* je definisan kompozicionim pristupom, pri čemu se niz zadataka čuva pomoću funkcije `ref`. Funkcije za rad sa zadacima nalaze se u istom *store*-u i zajedno sa stanjem vraćaju se kao njegov javni interfejs.

```
export const useZadaciStore = defineStore('zadaci', () => {
  const zadaci = ref([
    {
      id: 1,
      naziv: 'Naučiti Angular',
      opis: 'Proći kroz osnovne koncepte i praktičnu implementaciju.',
      status: 'aktivan'
    },
    {
      id: 2,
      naziv: 'Naučiti Vue.js',
      opis: 'Implementirati iste funkcionalnosti u Vue.js aplikaciji.',
      status: 'završen'
    }
  ]);

  function pronadjiPoId(id) {
    return zadaci.value.find((zadatak) => zadatak.id === id);
  }

  function dodaj(zadatak) {
    const noviId =
      Math.max(0, ...zadaci.value.map((zadatak) => zadatak.id)) + 1;

    zadaci.value.push({
      id: noviId,
      ...zadatak
    });
  }

  return { zadaci, pronadjiPoId, dodaj, izmeni, obrisi };
});
```

Kôd 4.11: Upravljanje stanjem zadataka u *Vue.js* aplikaciji

Kao i u *Angular* implementaciji, *store* sadrži operacije za pronalaženje, dodavanje, izmenu i brisanje zadataka. Pri radu unutar *store*-a vrednosti reaktivnog niza pristupa se preko svojstva `value`. Prilikom dodavanja novi zadatak se dodaje u postojeći niz metodom `push`, dok se za izmenu i brisanje vrednost reference zamenjuje rezultatom operacija `map`, odnosno `filter`.

Komponente koje koriste zajedničke podatke dobijaju pristup *store*-u pozivom funkcije `useZadaciStore`. Na primer:

```
const zadaciStore = useZadaciStore()
```

Nakon toga se stanje i operacije mogu koristiti na svim mestima na kojima su potrebni, bez prosleđivanja kompletnog skupa zadataka kroz hijerarhiju komponenti.

### Poređenje pristupa

U obe aplikacije zajedničko stanje zadataka izdvojeno je iz komponenti koje ga prikazuju, čime je postignuto razdvajanje podataka i operacija nad njima od korisničkog interfejsa. Obe implementacije definišu početni skup podataka i obezbeđuju iste operacije za pronalaženje, dodavanje, izmenu i brisanje zadataka.

U *Angular* aplikaciji ovo je realizovano servisom i mehanizmom ubrizgavanja zavisnosti, dok se reaktivno stanje čuva pomoću signala. Interni signal ostaje privatan, a komponentama se izlaže verzija namenjena samo za čitanje, pa su promene koncentrisane u metodama servisa.

U *Vue.js* aplikaciji ista odgovornost pripada *Pinia store*-u, u kojem se stanje definiše pomoću funkcije `ref`, a operacije kao funkcije u okviru istog *store*-a. Za korišćenje zajedničkog stanja nije potrebno ubrizgavanje zavisnosti, već komponenta direktno poziva odgovarajuću `use...Store` funkciju.

Razlika je prisutna i na nivou jezika. *TypeScript* implementacija u *Angular*-u omogućava eksplicitno definisanje tipa zadatka i ograničenje dozvoljenih vrednosti njegovog statusa, dok se u *JavaScript* implementaciji *Vue.js*-a ista struktura održava kroz način na koji su objekti kreirani i korišćeni u aplikaciji. Ova razlika proizlazi iz izbora jezika u praktičnoj implementaciji i ne predstavlja razliku u funkcionalnim zahtevima aplikacija.

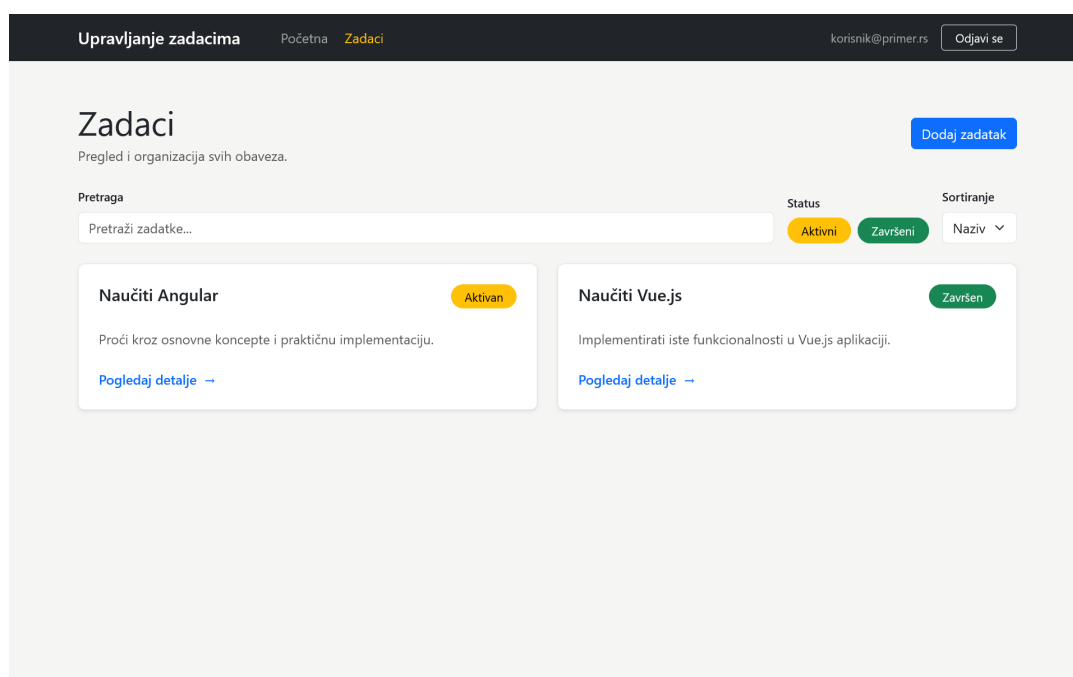
Sa stanovišta korisnika, oba pristupa daju isti rezultat. Razlike postaju vidljive prvenstveno u organizaciji zajedničkog stanja, načinu pristupa tom stanju i sintaksi kojom se njegove promene realizuju.

### Prikaz liste zadataka i komponentna organizacija

Stranica sa zadacima predstavlja centralni deo korisničkog interfejsa obe aplikacije. Na njoj se prikazuje skup trenutno dostupnih zadataka, omogućava prelazak na stranicu za dodavanje novog zadatka i pristup detaljima pojedinačnih zadataka. Pored toga, stranica sadrži kontrole za pretragu, filtriranje i sortiranje, koje su detaljnije razmotrene u narednoj celini.

U obe implementacije prikaz pojedinačnog zadatka izdvojen je u posebnu komponentu. Na taj način stranica odgovorna za prikaz liste ne sadrži kompletnu strukturu kartice za svaki zadatak, već prolazi kroz kolekciju podataka i za svaki element koristi komponentu `ZadatakKartica`. Ovakva organizacija omogućava izdvajanje odgovornosti i ponovnu upotrebu komponente, dok se podaci o konkretnom zadatku prosleđuju iz roditeljske u komponentu potomka.

Korisnički interfejs stranice sa zadacima jednak je u obe implementacije. Reprezentativni prikaz stranice dat je na slici 4.2.



Slika 4.2: Stranica sa prikazom zadataka

### Angular implementacija

U *Angular* aplikaciji komponenta `ZadatakKartica` prima podatke o zadatku preko signal ulaza. Funkcija `input.required` označava da je prosleđivanje objekta tipa `Zadatak` obavezno.

```
export class ZadatakKartica {  
  readonly zadatak = input.required<Zadatak>();  
}
```

Kôd 4.12: Ulazni podatak komponente `ZadatakKartica` u *Angular* aplikaciji

Vrednosti prosleđenog zadatka u šablonu se čitaju pozivom signala. Na osnovu statusa zadatka određuje se tekst i odgovarajuća klasa za vizuelno označavanje, dok se identifikator koristi za formiranje putanje ka stranici sa detaljima.

```
<h2>{{ zadatak().naziv }}</h2>

<span
  [class.text-bg-warning]="zadatak().status === 'aktivan'"
  [class.text-bg-success]="zadatak().status === 'završen'"
>
  {{ zadatak().status === 'aktivan' ? 'Aktivan' : 'Završen' }}
</span>

<a [routerLink]="['/zadaci', zadatak().id]">
  Pogledaj detalje
</a>
```

Kôd 4.13: Deo šablona kartice zadatka u *Angular* aplikaciji

Na stranici **Zadaci** rezultat koji treba prikazati prolazi se ugrađenim kontrolnim blokom `@for`. Identifikator zadatka koristi se u izrazu `track`, čime se svakom prikazanom elementu pridružuje stabilan identitet. Objekat zadatka zatim se prosleđuje komponenti `ZadatakKartica` preko njenog ulaza.

```
@for (zadatak of prikazaniZadaci(); track zadatak.id) {
  <div class="col-12 col-lg-6">
    <app-zadatak-kartica [zadatak]="zadatak" />
  </div>
} @empty {
  <div class="col-12">
    <div class="border-top py-3 text-muted">
      Nema zadataka koji odgovaraju izabranim kriterijumima.
    </div>
  </div>
}
```

Kôd 4.14: Prikaz kolekcije zadataka u *Angular* aplikaciji

Blok `@empty` omogućava da se u okviru iste strukture definiše sadržaj koji se prikazuje kada kolekcija nema nijedan element. U ovom slučaju korisniku se prikazuje poruka da nijedan zadatak ne odgovara trenutno izabranim kriterijumima.

### ***Vue.js* implementacija**

U *Vue.js* aplikaciji komponenta `ZadatakKartica` definisana je kao komponenta u jednoj datoteci i koristi sintaksu `<script setup>`. Podatak o zadatku prima se

preko svojstva komponente definisanog funkcijom `defineProps`. Svojstvo je označeno kao obavezno, čime se ostvaruje ista uloga koju u *Angular* implementaciji ima obavezni signal ulaz.

```
<script setup>
defineProps({
  zadatak: {
    type: Object,
    required: true,
  },
})
</script>
```

Kôd 4.15: Ulazni podatak komponente `ZadatakKartica` u *Vue.js* aplikaciji

Za razliku od signal ulaza u *Angular* šablonu, svojstvima prosleđenog objekta u *Vue.js* šablonu pristupa se neposredno. Dinamičke klase povezuju se direktivom `:class`, dok se za prelazak na detalje koristi komponenta `RouterLink`.

```
<h2>{{ zadatak.naziv }}</h2>

<span
  :class="{
    'text-bg-warning': zadatak.status === 'aktivan',
    'text-bg-success': zadatak.status === 'završen'
  }"
>
  {{ zadatak.status === 'aktivan' ? 'Aktivan' : 'Završen' }}
</span>

<RouterLink
  :to="{ name: 'detalji-zadatka', params: { id: zadatak.id } }"
>
  Pogledaj detalje
</RouterLink>
```

Kôd 4.16: Deo šablona kartice zadatka u *Vue.js* aplikaciji

Prikaz kolekcije realizovan je direktivom `v-for`. Kao i u *Angular* implementaciji, identifikator zadatka koristi se kao ključ svakog elementa, dok se objekat zadatka prosleđuje komponenti pomoću `:zadatak` vezivanja.

Za proveru da li postoje elementi za prikaz koriste se direktive `v-if` i `v-else`. Kada kolekcija nije prazna izvršava se `v-for`, dok se u suprotnom prikazuje odgovarajuća poruka korisniku.

Odgovarajući deo šablona prikazan je u kodu 4.17.

```
<div v-if="prikazaniZadaci.length" class="row g-4">
  <div
    v-for="zadatak in prikazaniZadaci"
    :key="zadatak.id"
    class="col-12 col-lg-6"
  >
    <ZadatakKartica :zadatak="zadatak" />
  </div>
</div>
<div v-else class="border-top py-3 text-muted">
  Nema zadataka koji odgovaraju izabranim kriterijumima.
</div>
```

Kôd 4.17: Prikaz kolekcije zadataka u *Vue.js* aplikaciji

## Poređenje pristupa

Komponentna organizacija je u ovom delu aplikacija konceptualno veoma slična. U oba slučaja stranica predstavlja roditeljsku komponentu koja raspolaže kolekcijom zadataka, dok je prikaz pojedinačnog elementa izdvojen u komponentu *ZadatakKartica*. Podatak se prosleđuje jednosmerno iz roditeljske komponente ka komponenti potomka, a identifikator zadatka koristi se prilikom prikaza kolekcije i navigacije ka detaljima.

Razlika je prvenstveno u sintaksi i mehanizmima kojima se ovaj odnos izražava. U *Angular* implementaciji obavezni ulaz definisan je funkcijom `input.required`, a njegova vrednost se u šablonu čita kao signal. U *Vue.js* implementaciji odgovarajući ulaz predstavlja *prop* definisan funkcijom `defineProps`, kojem se u šablonu pristupa neposredno.

Razlikuje se i način iterativnog i uslovnog prikaza. *Angular* koristi kontrolne blokove `@for` i `@empty`, pri čemu se identitet elementa definiše izrazom `track`. U *Vue.js* ista funkcionalnost ostvarena je kombinacijom direktiva `v-for`, `:key`, `v-if` i `v-else`.

Uprkos razlikama u sintaksi, rezultat je jednak: obe aplikacije prikazuju isti skup zadataka korišćenjem ponovo upotrebljive komponente i pružaju istu informaciju u slučaju kada nijedan zadatak ne odgovara kriterijumima prikaza.



### Pretraga, filtriranje i sortiranje

Pored osnovnog prikaza liste, obe aplikacije omogućavaju korisniku da suzi i organizuje skup prikazanih zadataka. Dostupna je pretraga prema nazivu i opisu, nezavisno uključivanje i isključivanje aktivnih i završenih zadataka, kao i sortiranje prema nazivu u rastućem ili opadajućem redosledu.

Ove funkcionalnosti ne menjaju osnovni skup zadataka, već određuju njegovu izvedenu verziju koja se prikazuje korisniku. Zbog toga se stanje pretrage, filtera i sortiranja čuva lokalno na stranici sa zadacima, dok zajedničko stanje samih zadataka ostaje u servisu, odnosno *Pinia store*-u. Na taj način privremeni kriterijumi prikaza nisu nepotrebno smešteni u globalno stanje aplikacije.

Kontrole za pretragu, filtriranje i sortiranje izdvojene su u posebnu komponentu **FilteriSortiranje**. Stranica sa zadacima zadržava stanje i izračunava rezultat, dok izdvojena komponenta prima trenutne vrednosti i obaveštava roditeljsku komponentu o promenama. Time je odgovornost za prikaz i korisničku interakciju odvojena od logike formiranja konačne liste.

### Angular implementacija

U *Angular* aplikaciji lokalno stanje definisano je pomoću signala. Dva logička signala određuju da li se prikazuju aktivni i završeni zadaci, dok posebni signali čuvaju izabrani način sortiranja i tekst pretrage. Dozvoljene vrednosti sortiranja dodatno su ograničene *TypeScript* tipom **SortiranjeZadataka**.

```
export type SortiranjeZadataka = 'naziv-asc' | 'naziv-desc';

readonly aktivniUkljuceni = signal(true);
readonly zavrсениUkljuceni = signal(true);
readonly sortiranje = signal<SortiranjeZadataka>('naziv-asc');
readonly pretraga = signal('');
```

Kôd 4.18: Lokalno stanje pretrage, filtriranja i sortiranja u *Angular* aplikaciji

Konačan skup elemenata za prikaz definisan je kao izračunati signal **prikazaniZadaci**. Njegova vrednost zavisi od zajedničkog skupa zadataka i svih lokalnih kriterijuma. Kada se promeni neka od tih vrednosti, rezultat se ponovo izračunava i prikaz se automatski usklađuje sa novim stanjem. Formiranje konačne liste prikazano je u kodu 4.19.

```
readonly prikazaniZadaci = computed(() => {
  const sortiranje = this.sortiranje();
  const pojam = this.pretraga().trim().toLocaleLowerCase('sr');
  let rezultat = this.zadaci();

  rezultat = rezultat.filter(zadatak =>
    (zadatak.status === 'aktivan' && this.aktivniUkljuceni()) ||
    (zadatak.status === 'završen' && this.završeniUkljuceni())
  );

  if (pojam) {
    rezultat = rezultat.filter(zadatak =>
      `${zadatak.naziv} ${zadatak.opis}`
        .toLocaleLowerCase('sr')
        .includes(pojam)
    );
  }

  return [...rezultat].sort((a, b) =>
    sortiranje === 'naziv-asc'
      ? a.naziv.localeCompare(b.naziv, 'sr')
      : b.naziv.localeCompare(a.naziv, 'sr')
  );
});
```

Kôd 4.19: Formiranje liste za prikaz u *Angular* aplikaciji

Filtriranje prema statusu zasnovano je na dva nezavisna kriterijuma. Ako su oba uključena, prikazuju se svi zadaci. Isključivanjem jednog od njih ostaju samo zadaci drugog statusa, dok isključivanje oba kriterijuma daje praznu listu.

Pretraga se primenjuje nad spojenim nazivom i opisom zadatka. Pre poređenja uneti tekst se uklanja od vodećih i završnih razmaka i prevodi u mala slova korišćenjem srpskog lokaliteta. Isti postupak primenjuje se nad tekстом zadatka, čime pretraga nije zavisna od veličine slova.

Na kraju se filtrirani niz kopira i sortira prema nazivu. Signal `sortiranje` određuje smer poređenja, dok metoda `localeCompare` koristi srpski lokalitet.

Komponenta `FilteriSortiranje` prima trenutno stanje preko signal ulaza, dok promene prosleđuje roditeljskoj komponenti pomoću izlaza:

```
readonly aktivniUkljuceni = input.required<boolean>();
readonly zavrсениUkljuceni = input.required<boolean>();
readonly sortiranje = input.required<SortiranjeZadataka>();
readonly pretraga = input.required<string>();

readonly aktivniPromenjeni = output<boolean>();
readonly zavrсениPromenjeni = output<boolean>();
readonly sortiranjePromenjeno = output<SortiranjeZadataka>();
readonly pretragaPromenjena = output<string>();
```

Kôd 4.20: Ulazi i izlazi komponente za filtriranje u *Angular* aplikaciji

Roditeljska komponenta reaguje na emitovane događaje promenom odgovarajućih signala. Na primer:

```
<app-filteri-sortiranje
  [aktivniUkljuceni]="aktivniUkljuceni()"
  [zavrсениUkljuceni]="zavrсениUkljuceni()"
  [sortiranje]="sortiranje()"
  [pretraga]="pretraga()"
  (aktivniPromenjeni)="aktivniUkljuceni.set($event)"
  (zavrсениPromenjeni)="zavrсениUkljuceni.set($event)"
  (sortiranjePromenjeno)="sortiranje.set($event)"
  (pretragaPromenjena)="pretraga.set($event)"
/>
```

Na ovaj način komponenta sa kontrolama ne menja direktno stanje roditeljske stranice, već samo emituje informaciju o korisničkoj akciji.

### ***Vue.js* implementacija**

U *Vue.js* aplikaciji ista lokalna stanja definisana su funkcijom `ref`. Početne vrednosti odgovaraju vrednostima u *Angular* implementaciji, pa su pri prvom prikazu uključena oba statusa, sortiranje je postavljeno od A do Z, a tekst pretrage je prazan.

```
const aktivniUkljuceni = ref(true)
const zavrсениUkljuceni = ref(true)
const sortiranje = ref('naziv-asc')
const pretraga = ref('')
```

Kôd 4.21: Lokalno stanje pretrage, filtriranja i sortiranja u *Vue.js* aplikaciji

Konačna lista takođe je izvedeno reaktivno stanje i formira se pomoću funkcije `computed`. Redosled operacija odgovara *Angular* implementaciji: najpre se primeњуje filter prema statusu, zatim pretraga, a potom sortiranje.

```
const prikazaniZadaci = computed(() => {
  let rezultat = zadaciStore.zadaci;

  rezultat = rezultat.filter((zadatak) =>
    (zadatak.status === 'aktivan' && aktivniUkljuceni.value) ||
    (zadatak.status === 'zavrsen' && zavrseniUkljuceni.value)
  );

  const pojam = pretraga.value.trim().toLocaleLowerCase('sr');

  if (pojam) {
    rezultat = rezultat.filter((zadatak) =>
      `${zadatak.naziv} ${zadatak.opis}`
        .toLocaleLowerCase('sr')
        .includes(pojam)
    );
  }

  return [...rezultat].sort((a, b) =>
    sortiranje.value === 'naziv-asc'
      ? a.naziv.localeCompare(b.naziv, 'sr')
      : b.naziv.localeCompare(a.naziv, 'sr')
  );
});
```

Kôd 4.22: Formiranje liste za prikaz u *Vue.js* aplikaciji

Logika filtriranja, pretrage i sortiranja namerno je jednaka logici *Angular* aplikacije, kako bi obe implementacije ostvarile isto ponašanje. Razlika je u načinu pristupa reaktivnim vrednostima: unutar *JavaScript* dela *Vue.js* komponente vrednosti kreirane funkcijom `ref` čitaju se preko svojstva `value`.

Komponenta `FilteriSortiranje` prima trenutno stanje preko svojstava definisanih funkcijom `defineProps`, dok se promene prosleđuju roditeljskoj komponenti događajima definisanim pomoću `defineEmits`.

Na primer, promena teksta za pretragu prosleđuje se događajem neposredno iz šablona:

```
<input type="search" :value="pretraga"
  @input="emit('pretraga-promenjena', $event.target.value)">
```

Roditeljska stranica zatim menja lokalno reaktivno stanje. U šablonima *Vue.js* komponente vrednosti kreirane funkcijom `ref` automatski se raspakuju, zbog čega nije potrebno eksplicitno pristupati svojstvu `value`.

Definicija svojstava i događaja komponente prikazana je u kodu 4.23.

```
defineProps({
  aktivniUkljuceni: { type: Boolean, required: true },
  završeniUkljuceni: { type: Boolean, required: true },
  sortiranje: { type: String, required: true },
  pretraga: { type: String, required: true }
});
const emit = defineEmits([
  'aktivni-promenjeni', 'završeni-promenjeni',
  'sortiranje-promenjeno', 'pretraga-promenjena'
]);
```

Kôd 4.23: Svojstva i događaji komponente za filtriranje u *Vue.js* aplikaciji

### Poređenje pristupa

U obe implementacije pretraga, filtriranje i sortiranje predstavljaju lokalno stanje stranice, dok osnovni skup zadataka ostaje u zajedničkom servisu, odnosno *store*-u. Takva organizacija razdvaja trajanje i odgovornost dva tipa stanja: podaci o zadacima dele se između više delova aplikacije, dok su kriterijumi prikaza relevantni samo za konkretnu stranicu.

Mehanizam izvedenog stanja je konceptualno gotovo jednak. *Angular* koristi `signal` za promenljive vrednosti i `computed` za formiranje liste, dok *Vue.js* koristi `ref` i `computed`. U oba slučaja rezultat zavisi od reaktivnih izvora i automatski se menja kada korisnik promeni neki od kriterijuma.

Razlika je izraženija u komunikaciji između roditeljske komponente i komponente sa kontrolama. U *Angular* implementaciji koriste se `signal` ulazi definisani funkcijom `input` i izlazi definisani funkcijom `output`. U *Vue.js* aplikaciji istu ulogu imaju *props* i emitovani događaji definisani funkcijama `defineProps` i `defineEmits`.

Dodatna razlika proizlazi iz korišćenih jezika. Vrednost sortiranja u *Angular* implementaciji ograničena je *TypeScript* unijom na dve dozvoljene vrednosti, dok je u *JavaScript* implementaciji *Vue.js*-a predstavljena običnim tekstualnim podatkom. Sama logika obrade i rezultat za korisnika, međutim, ostaju jednaki.

Ovaj deo implementacije posebno pokazuje sličnost reaktivnog modela dva radna okvira: iako se razlikuju sintaksa i način deklarisanja komunikacije između kompo-

nenti, oba pristupa omogućavaju da se izvedeni prikaz zadataka definiše deklarativno na osnovu trenutnog stanja, bez ručnog osvežavanja korisničkog interfejsa.

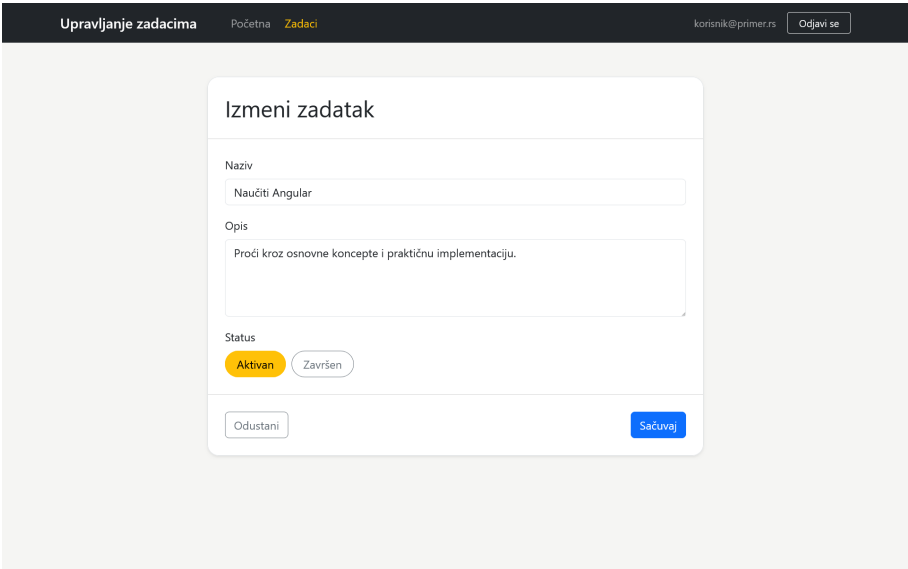
### Dodavanje i izmena zadataka i validacija formi

Dodavanje i izmena zadataka u obe aplikacije realizovani su korišćenjem iste ponovo upotrebljive komponente forme. Na taj način zajednički deo korisničkog interfejsa i logike unosa nije dupliran između stranice za dodavanje novog zadatka i stranice za izmenu postojećeg zadatka.

Forma sadrži polja za naziv i opis zadatka, kao i izbor jednog od dva moguća statusa. Naziv mora sadržati najmanje tri karaktera, dok opis mora sadržati najmanje pet karaktera. Provera minimalne dužine vrši se nad tekstom nakon uklanjanja vodećih i završnih razmaka, čime se sprečava da ti razmaci utiču na ispunjenost uslova minimalne dužine.

Pri dodavanju se forma prikazuje sa početnim statusom *Aktivan*, dok se pri izmeni njena polja popunjavaju postojećim podacima izabranog zadatka. Nakon uspešnog čuvanja odgovarajuća stranica poziva operaciju nad zajedničkim stanjem zadataka i korisnika vraća na listu zadataka.

Korisnički interfejs forme jednak je u obe implementacije. Reprezentativni prikaz forme dat je na slici 4.3.



Slika 4.3: Forma za unos i izmenu zadatka

### Angular implementacija

U *Angular* aplikaciji komponenta `ZadatakForma` zasnovana je na mehanizmu *Reactive Forms*. Instanca forme kreira se pomoću klase `NonNullableFormBuilder`, čime su vrednosti kontrola definisane kao nenulte vrednosti odgovarajućih tipova.

Za proveru minimalne dužine naziva i opisa definisan je poseban validator koji najpre uklanja vodeće i završne razmake, a zatim proverava dužinu preostalog teksta.

```
function minimalnaTrimovanaDuzina(duzina: number): ValidatorFn {  
  return (control: AbstractControl): ValidationErrors | null =>  
    String(control.value).trim().length >= duzina  
      ? null  
      : { minimalnaTrimovanaDuzina: true };  
}
```

Kôd 4.24: Validator minimalne dužine nakon uklanjanja razmaka

Forma se zatim definiše kao grupa od tri kontrole. Za naziv i opis koriste se ugrađeni validator `required` i prethodno definisani validator, dok je status ograničen tipom svojstva `status` interfejsa `Zadatak`.

```
readonly forma = this.formBuilder.group({  
  naziv: ['', [Validators.required, minimalnaTrimovanaDuzina(3)]],  
  opis: ['', [Validators.required, minimalnaTrimovanaDuzina(5)]],  
  status: this.formBuilder.control<Zadatak['status']>('aktivan')  
});
```

Kôd 4.25: Definisanje forme zadatka u *Angular* aplikaciji

Komponenta prima naslov forme i opcione početne podatke preko signal ulaza. Rezultat uspešnog unosa prosleđuje roditeljskoj komponenti preko izlaza `sacuvano`.

```
readonly naslov = input.required<string>();  
readonly pocetniPodaci = input<Omit<Zadatak, 'id'> | null>(null);  
readonly sacuvano = output<Omit<Zadatak, 'id'>>();
```

Kôd 4.26: Ulazi i izlaz forme u *Angular* aplikaciji

Pri izmeni zadatka potrebno je popuniti formu postojećim vrednostima. Promena ulaznih podataka prati se pomoću funkcije `effect`, a kada su početni podaci dostupni, vrednosti odgovarajućih kontrola postavljaju se metodom `setValue`. Ovaj postupak prikazan je u kodu 4.27.

```
constructor() {  
  effect(() => {  
    const podaci = this.pocetniPodaci();  
  
    if (podaci) {  
      this.forma.setValue({  
        naziv: podaci.naziv,  
        opis: podaci.opis,  
        status: podaci.status  
      });  
    }  
  });  
}
```

Kôd 4.27: Popunjavanje forme pri izmeni zadatka u *Angular* aplikaciji

Prilikom slanja forme najpre se proverava njena validnost. Ukoliko forma nije ispravna, sve kontrole se označavaju kao dodirnite kako bi korisniku bile prikazane poruke o greškama. U suprotnom se naziv i opis dodatno normalizuju metodom `trim`, a rezultat se emituje roditeljskoj komponenti.

```
sacuvaj(): void {  
  if (this.forma.invalid) {  
    this.forma.markAllAsTouched();  
    return;  
  }  
  
  const podaci = this.forma.getRawValue();  
  
  this.sacuvano.emit({  
    naziv: podaci.naziv.trim(),  
    opis: podaci.opis.trim(),  
    status: podaci.status  
  });  
}
```

Kôd 4.28: Čuvanje podataka iz *Angular* forme

Status je u korisničkom interfejsu predstavljen sa dva dugmeta koja imaju ulogu radio-kontrola. Promena statusa vrši se izmenom vrednosti odgovarajuće kontrole forme, pa u svakom trenutku može biti izabran tačno jedan status.

Stranice za dodavanje i izmenu koriste istu komponentu forme, ali događaj `sacuvano` obrađuju različito: prva poziva metodu `dodaj`, a druga metodu `izmeni` sa identifikatorom izabranog zadatka.



```
dodaj(podaci: Omit<Zadatak, 'id'>): void {  
  this.servisZadataka.dodaj(podaci);  
  this.router.navigate(['/zadaci']);  
}
```

Kôd 4.29: Korišćenje forme pri dodavanju zadatka u *Angular* aplikaciji

Pri izmeni se identifikator preuzima iz parametra rute, a postojeći zadatak prosleđuje komponenti kao početni podatak.

### *Vue.js* implementacija

U *Vue.js* aplikaciji forma je takođe izdvojena u komponentu `ZadatakForma`, ali njeno stanje nije zasnovano na posebnom *API*-ju za forme. Podaci se čuvaju u objektu kreiranom funkcijom `reactive`.

```
const forma = reactive({  
  naziv: '',  
  opis: '',  
  status: 'aktivan',  
})  
  
const dodirnuto = reactive({  
  naziv: false,  
  opis: false,  
})
```

Kôd 4.30: Stanje forme zadatka u *Vue.js* aplikaciji

Validnost naziva i opisa predstavljena je izračunatim vrednostima. Kao i u *Angular* implementaciji, proverava se dužina teksta nakon uklanjanja vodećih i završnih razmaka.

```
const nazivNeispravan = computed(() => forma.naziv.trim().length < 3);  
const opisNeispravan = computed(() => forma.opis.trim().length < 5);
```

Kôd 4.31: Validacija forme zadatka u *Vue.js* aplikaciji

Vrednosti polja povezuju se sa reaktivnim stanjem direktivom `v-model`. Dodatno se prati da li je korisnik stupio u interakciju sa poljem, kako se poruka o grešci ne bi prikazivala pre nego što je unos započet.

Komponenta prima naslov i opcione početne podatke preko svojstava, dok se rezultat uspešnog unosa prosleđuje događajem `sacuvano`.

```
const props = defineProps({
  naslov: {
    type: String,
    required: true,
  },
  pocetniPodaci: {
    type: Object,
    default: null,
  },
})

const emit = defineEmits(['sacuvano'])
```

Kôd 4.32: Svojstva i događaj forme u *Vue.js* aplikaciji

Pri izmeni zadatka potrebno je reagovati na početne podatke koje roditeljska komponenta prosleđuje formi. Za tu namenu koristi se funkcija `watch`. Opcija `immediate` omogućava izvršavanje funkcije i pri inicijalnom postavljanju komponente, tako da se postojeće vrednosti odmah upišu u formu.

```
watch(
  () => props.pocetniPodaci,
  (podaci) => {
    if (podaci) {
      forma.naziv = podaci.naziv
      forma.opis = podaci.opis
      forma.status = podaci.status
    }
  },
  { immediate: true },
)
```

Kôd 4.33: Popunjavanje forme pri izmeni zadatka u *Vue.js* aplikaciji

Izbor statusa realizovan je neposrednom promenom svojstva `forma.status`. Kao i u *Angular* aplikaciji, korisnički interfejs prikazuje dve međusobno isključive opcije, tako da zadatak u jednom trenutku može imati samo jedan status.

Pri slanju forme oba polja se označavaju kao dodirnuta. Ukoliko neka od izračunatih vrednosti ukazuje na neispravan unos, obrada se prekida. U suprotnom se normalizovani podaci emituju roditeljskoj komponenti.

```
function sacuvaj() {
  dodirnuto.naziv = true;
  dodirnuto.opis = true;

  if (nazivNeispravan.value || opisNeispravan.value) {
    return;
  }

  emit('sacuvano', {
    naziv: forma.naziv.trim(),
    opis: forma.opis.trim(),
    status: forma.status
  });
}
```

Kôd 4.34: Čuvanje podataka iz *Vue.js* forme

Stranice za dodavanje i izmenu koriste istu komponentu forme. Pri dodavanju nije prosleđena početna vrednost zadatka, dok stranica za izmenu prosleđuje postojeći objekat preko svojstva `pocetniPodaci`. Događaj `sacuvano` se zatim obrađuje pozivom odgovarajuće metode *Pinia store*-a.

```
<ZadatakForma
  v-if="zadatak"
  naslov="Izmeni zadatak"
  :pocetni-podaci="zadatak"
  @sacuvano="izmeni"
/>
```

Kôd 4.35: Korišćenje forme pri izmeni zadatka u *Vue.js* aplikaciji

Nakon dodavanja ili izmene korisnik se programskom navigacijom vraća na stranicu sa zadacima.

### Poređenje pristupa

U obe implementacije forma je izdvojena u ponovo upotrebljivu komponentu koju koriste i stranica za dodavanje i stranica za izmenu. Komponenta prima početne podatke kada se koristi za izmenu, emituje validirane podatke roditeljskoj komponenti i ne obavlja direktno promene nad zajedničkim stanjem zadataka. Time je sama forma odvojena od odluke da li se izvršava operacija dodavanja ili izmene.

Najizraženija razlika odnosi se na način organizacije forme i validacije. *Angular* koristi strukturisan *API Reactive Forms*, u okviru kojeg su kontrole grupisane u objekat forme, a pravila validacije pridružena samim kontrolama. Validnost forme

dostupna je kroz njen *API*, dok se poruke o greškama prikazuju na osnovu stanja pojedinačnih kontrola.

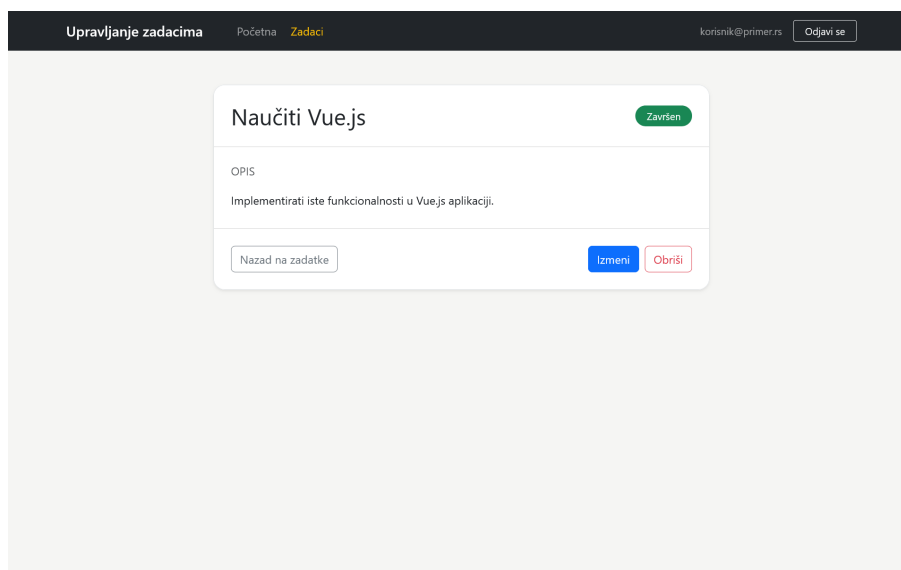
U *Vue.js* implementaciji stanje forme je običan reaktivan objekat. Validacija nije izdvojena u poseban sistem formi, već je eksplicitno realizovana pomoću funkcija `reactive` i `computed`. Povezivanje polja i njihovih vrednosti dodatno je pojednostavljeno direktivom `v-model`.

Razlika je vidljiva i pri inicijalizaciji forme za izmenu. *Angular* reaguje na signal ulaz pomoću funkcije `effect` i postavlja vrednosti kontrola forme, dok *Vue.js* prati promenu svojstva funkcijom `watch` i neposredno menja svojstva reaktivnog objekta.

Komunikacija sa roditeljskom komponentom prati obrasce već uočene u prethodnim delovima rada. *Angular* koristi `input` i `output`, dok *Vue.js* koristi `defineProps` i `defineEmits`. Uprkos različitim mehanizmima, rezultat je jednak: korisnik dobija istu formu, ista pravila validacije i jednako ponašanje pri dodavanju i izmeni zadatka.

### Prikaz detalja i brisanje zadataka

Stranica sa detaljima prikazuje podatke o izabranom zadatku i omogućava njegovu izmenu i brisanje. Zadatak se određuje na osnovu dinamičkog parametra `:id` iz putanje, nakon čega se odgovarajući objekat pronalazi u zajedničkom stanju zadataka. Jednak korisnički interfejs u obe implementacije prikazan je na slici 4.4.



Slika 4.4: Stranica sa detaljima zadatka

Ukoliko zadatak sa prosleđenim identifikatorom ne postoji, korisniku se umesto detalja prikazuje odgovarajuća poruka i omogućava povratak na stranicu sa zadacima. Brisanje zahteva potvrdu korisnika, a nakon potvrđenog brisanja korisnik se programski vraća na listu zadataka.

### Angular implementacija

U *Angular* aplikaciji pristup parametrima trenutno aktivne rute omogućen je servisom `ActivatedRoute`. Vrednost parametra `id` preuzima se iz objekta `paramMap` i pretvara u broj, nakon čega se metodom `pronadjiPoId` servisa `ServisZadataka` pronalazi odgovarajući zadatak.

```
private readonly servisZadataka = inject(ServisZadataka);
private readonly route = inject(ActivatedRoute);
private readonly router = inject(Router);

readonly id = Number(this.route.snapshot.paramMap.get('id'));
readonly zadatak = this.servisZadataka.pronadjiPoId(this.id);
```

Kôd 4.36: Preuzimanje parametra rute i pronalaženje zadatka u *Angular* aplikaciji

U šablonu se postojanje zadatka proverava kontrolnim blokom `@if`. Ako je zadatak pronađen, prikazuju se njegov naziv, opis i status, zajedno sa akcijama za izmenu i brisanje. Za prelazak na stranicu za izmenu koristi se dinamički `routerLink` koji u putanju uključuje identifikator zadatka.

```
@if (zadatak; as trenutniZadatak) {
  <h1>{{ trenutniZadatak.naziv }}</h1>

  <p>{{ trenutniZadatak.opis }}</p>

  <a [routerLink]="['/zadaci', trenutniZadatak.id, 'izmeni']">
    Izmeni
  </a>
} @else {
  <div class="alert alert-warning">Zadatak nije pronađen.</div>
}
```

Kôd 4.37: Uslovni prikaz i navigacija ka izmeni u *Angular* aplikaciji

Akcija brisanja realizovana je metodom `obrisi`. Pre promene zajedničkog stanja prikazuje se standardni dijalog za potvrdu pomoću funkcije `window.confirm`. Ukoliko korisnik odustane, metoda se završava bez promene podataka. Nakon potvrde poziva se odgovarajuća metoda servisa, a zatim se pomoću servisa `Router` izvršava

programska navigacija na stranicu sa zadacima.

```
obrisi(): void {
  if (!this.zadatak) {
    return;
  }

  const potvrda = window.confirm(
    'Da li ste sigurni da želite da obrišete zadatak "${this.zadatak.naziv}"?'
  );

  if (!potvrda) {
    return;
  }

  this.servisZadataka.obrisi(this.id);
  this.router.navigate(['/zadaci']);
}
```

Kôd 4.38: Brisanje zadatka u *Angular* aplikaciji

Sama promena niza zadataka ostaje odgovornost servisa **ServisZadataka**, dok stranica sa detaljima samo inicira operaciju i upravlja daljom navigacijom. Time je i u ovom slučaju očuvano razdvajanje između logike upravljanja zajedničkim stanjem i logike korisničkog interfejsa.

### **Vue.js** implementacija

U *Vue.js* aplikaciji podacima o aktivnoj ruti pristupa se funkcijom `useRoute`. Dinamički parametar `id` dostupan je kroz objekat `route.params`, nakon čega se pretvara u broj i koristi za pronalaženje zadatka u *Pinia store*-u.

```
const zadaciStore = useZadaciStore();
const route = useRoute();
const router = useRouter();

const id = Number(route.params.id);
const zadatak = zadaciStore.pronadjiPoId(id);
```

Kôd 4.39: Preuzimanje parametra rute i pronalaženje zadatka u *Vue.js* aplikaciji

Postojanje zadatka u šablonu proverava se direktivom `v-if`. Kada je zadatak dostupan, prikazuju se njegovi podaci i akcije, dok se u suprotnom prikazuje poruka o neuspešnom pronalaženju. Za prelazak na stranicu za izmenu koristi se komponenta **RouterLink**, pri čemu se odredište definiše imenom rute i odgovarajućim parametrom.

```
<template v-if="zadatak">
  <h1>{{ zadatak.naziv }}</h1>

  <p>{{ zadatak.opis }}</p>

  <RouterLink :to="{ name: 'izmeni-zadatak', params: { id: zadatak.id } }">
    Izmeni
  </RouterLink>
</template>

<template v-else>
  <div class="alert alert-warning">
    Zadatak nije pronađen.
  </div>
</template>
```

Kôd 4.40: Uslovni prikaz i navigacija ka izmeni u *Vue.js* aplikaciji

Postupak brisanja odgovara postupku u *Angular* aplikaciji. Nakon potvrde korisnika poziva se metoda `obrisi` *store*-a, a programska navigacija vrši se metodom `push` objekta dobijenog funkcijom `useRouter`.

```
function obrisi() {
  if (!zadatak) {
    return;
  }

  const potvrda = window.confirm(
    'Da li ste sigurni da želite da obrišete zadatak "${zadatak.naziv}"?'
  );

  if (!potvrda) {
    return;
  }

  zadaciStore.obrisi(id);
  router.push({ name: 'zadaci' });
}
```

Kôd 4.41: Brisanje zadatka u *Vue.js* aplikaciji

Kao i u prethodnim delovima aplikacije, stranica koristi funkcije *Pinia store*-a za promenu zajedničkog stanja, dok sama ostaje odgovorna za obradu korisničke akcije i navigaciju.

### Poređenje pristupa

Obe implementacije koriste isti osnovni postupak: identifikator zadatka preuzima se iz dinamičkog segmenta rute, na osnovu njega se pronalazi odgovarajući objekat u zajedničkom stanju, a rezultat određuje sadržaj koji će biti prikazan korisniku. Ukoliko objekat ne postoji, obe aplikacije prikazuju poruku da zadatak nije pronađen.

Razlike se pre svega odnose na *API* i sintaksu korišćenih rutera. *Angular* parametrima rute pristupa preko servisa `ActivatedRoute` i objekta `paramMap`, dok *Vue.js* koristi funkciju `useRoute` i objekat `params`. Za programsku navigaciju *Angular* koristi servis `Router` i metodu `navigate`, dok se u *Vue.js* koristi rezultat funkcije `useRouter` i metoda `push`.

Slična razlika postoji i u šablonima. Uslovni prikaz u *Angular* implementaciji realizovan je kontrolnim blokom `@if`, dok se u *Vue.js* koristi direktiva `v-if`. Deklarativna navigacija ka stranici za izmenu ostvarena je direktivom `routerLink`, odnosno komponentom `RouterLink`.

Sam postupak brisanja je gotovo jednak u obe aplikacije: korisnik najpre potvrđuje akciju, zatim servis odnosno *store* menja zajedničko stanje, a nakon toga se izvršava programsko preusmeravanje na listu zadataka. Time je funkcionalna ekvivalentnost zadržana i u poslednjem delu *CRUD* operacija, dok su razlike ograničene na mehanizme i sintaksu karakteristične za posmatrane radne okvire.

### 4.3 Završna razmatranja

Praktična implementacija pokazala je da je isti skup funkcionalnih zahteva moguće realizovati u radnim okvirima *Angular* i *Vue.js* uz jednako ponašanje aplikacije sa stanovišta korisnika. Obe implementacije obuhvataju rutiranje, autentifikaciju, upravljanje zajedničkim stanjem, komponentni prikaz zadataka, pretragu, filtriranje i sortiranje, rad sa formama, kao i operacije dodavanja, izmene i brisanja zadataka. Razlike koje su uočene tokom implementacije zato se ne odnose na dostupne funkcionalnosti, već prvenstveno na način njihove organizacije i izražavanja u okviru dva radna okvira.

U pogledu organizacije aplikacije, *Angular* se u praktičnom primeru oslanja na mehanizme koji su neposredno deo njegovog ekosistema. Rutiranje, ubrizgavanje zavisnosti, reaktivne forme i upravljanje stanjem pomoću signala realizuju se kori-



šćenjem odgovarajućih *Angular API*-ja. Zajedničko stanje zadataka i autentifikacije izdvojeno je u servise, kojima komponente pristupaju kroz mehanizam ubrizgavanja zavisnosti. Zaštita ruta takođe je realizovana posebnim funkcionalnim čuvarima ruta.

U *Vue.js* implementaciji iste odgovornosti raspoređene su između osnovnih mehanizama radnog okvira i zvaničnih biblioteka njegovog ekosistema. Rutiranje je realizovano pomoću biblioteke *Vue Router*, dok je zajedničko stanje organizovano korišćenjem biblioteke *Pinia*. Komponente zajedničkim podacima pristupaju neposrednim pozivom funkcija odgovarajućeg *store*-a, dok je kontrola pristupa rutama objedinjena u globalnom navigacionom čuvaru. Time se ostvaruje funkcionalno isto ponašanje kao u *Angular* aplikaciji, ali uz drugačiji način organizacije mehanizama.

Na nivou reaktivnosti dva pristupa pokazuju značajnu konceptualnu sličnost. U *Angular* implementaciji promenljivo stanje predstavljeno je signalima, dok se izvedene vrednosti formiraju funkcijom `computed`. U *Vue.js* aplikaciji istu ulogu imaju `ref` i `computed`. Ovo je posebno vidljivo pri implementaciji pretrage, filtriranja i sortiranja, gde se u oba slučaja konačna lista zadataka deklarativno izvodi iz zajedničkog stanja i trenutno izabranih kriterijuma. Razlike su prvenstveno sintaksne i odnose se na način pristupa i menjanja reaktivnih vrednosti.

Sličnosti su izražene i u komponentnoj organizaciji. U obe aplikacije ponavljajući delovi korisničkog interfejsa izdvojeni su u posebne komponente, dok se podaci prosleđuju iz roditeljskih komponenti ka komponentama potomcima, a korisničke akcije vraćaju roditeljskoj komponenti preko odgovarajućih događaja. *Angular* za ovu komunikaciju koristi funkcije `input` i `output`, dok *Vue.js* koristi `defineProps` i `defineEmits`. Iako je sintaksa različita, organizacija odgovornosti između komponenti ostaje veoma slična.

Jedna od izraženijih razlika u praktičnoj implementaciji uočena je pri radu sa formama. *Angular* pruža strukturisan mehanizam *Reactive Forms*, sa kontrolama forme, validatorima i zajedničkim stanjem validnosti. U implementaciji *Vue.js* aplikacije ista funkcionalnost realizovana je neposrednim korišćenjem reaktivnog objekta, izračunatih vrednosti i direktive `v-model`. Ovakav primer pokazuje da se isti zahtev može ostvariti različitim nivoom ugrađene strukture: u *Angular*-u je veći deo organizacije forme definisan posebnim *API*-jem, dok je u korišćenoj *Vue.js* implementaciji logika validacije eksplicitnije organizovana unutar same komponente.

Razlika u korišćenim jezicima takođe utiče na izgled praktične implementacije. *Angular* aplikacija napisana je u jeziku *TypeScript*, što je omogućilo eksplicitno de-

finisanje tipova podataka, kao što su model zadatka i dozvoljene vrednosti njegovog statusa. *Vue.js* aplikacija realizovana je u jeziku *JavaScript*, pa ista ograničenja nisu izražena kroz statički sistem tipova. Ova razlika proizlazi iz izbora jezika za praktičnu implementaciju i ne treba je tumačiti kao neposrednu razliku u mogućnostima samih radnih okvira.

Na osnovu realizovanog primera može se zaključiti da oba radna okvira pružaju mehanizme potrebne za razvoj funkcionalno ekvivalentne jednostranične veb aplikacije. *Angular* u posmatranoj implementaciji pruža izraženije integrisan skup mehanizama i unapred definisanih obrazaca, dok *Vue.js* kombinuje osnovne mogućnosti radnog okvira sa zvaničnim bibliotekama i omogućava da se odgovarajuća logika organizuje uz manju količinu unapred nametnute strukture. Izbor između ova dva pristupa zato ne može biti zasnovan samo na mogućnosti realizacije posmatranih funkcionalnosti, već zavisi i od željenog načina organizacije projekta, nivoa eksplicitnosti i razvojnog pristupa koji više odgovara konkretnom projektu i razvojnog timu.

## Glava 5

# Zaključak

Cilj ovog rada bio je da se izvrši uporedna analiza radnih okvira *Angular* i *Vue.js* kroz teorijsko razmatranje njihovih ključnih koncepata i praktičnu implementaciju dve međusobno funkcionalno ekvivalentne aplikacije. Poređenje je sprovedeno kroz više tehničkih i projektnih kriterijuma, uključujući arhitekturu i razvojnu filozofiju, složenost usvajanja, sintaksu i rad sa šablonima, performanse i renderovanje, upravljanje stanjem, ekosistem i razvojne alate, pogodnost u različitim projektnim kontekstima, kao i održavanje i dugoročnu stabilnost.

Teorijska analiza pokazala je da oba radna okvira pružaju mehanizme potrebne za razvoj savremenih komponentno zasnovanih veb aplikacija, ali da se razlikuju u načinu na koji su ti mehanizmi organizovani i povezani. Jedna od najizraženijih razlika ogleda se u stepenu unapred definisane strukture. *Angular* primenjuje integrisaniji i standardizovaniji pristup, u kojem su brojni mehanizmi i razvojni alati povezani kroz zajedničke konvencije. *Vue.js*, sa druge strane, primenjuje progresivniji i modularniji pristup, koji omogućava postepeno uključivanje dodatnih mogućnosti i ostavlja više prostora za izbor načina organizovanja aplikacije i pratećih alata.

Ova razlika odražava se i na proces usvajanja radnih okvira. Početak rada sa *Angular*-om podrazumeva upoznavanje sa većim brojem međusobno povezanih koncepata i mehanizama, dok *Vue.js* omogućava njihovo postepeno uvođenje u skladu sa zahtevima aplikacije. To, međutim, ne znači da je jedan radni okvir jednostavan, a drugi složen u apsolutnom smislu, već da se složenost njihovog usvajanja i organizovanja razvojnog procesa raspoređuje na različite načine.

Na nivou sintakse i komponentnog modela oba radna okvira omogućavaju deklarativno definisanje korisničkog interfejsa, povezivanje podataka, obradu događaja i kontrolu toka, ali koriste različite sintaksičke konvencije i načine organizovanja kom-

ponenti. Razlike postoje i u mehanizmima reaktivnosti, renderovanja i upravljanja stanjem, ali na osnovu njih nije moguće izdvojiti jedan radni okvir kao univerzalno efikasniji ili pogodniji. Performanse zavise od karakteristika konkretne aplikacije i načina njene implementacije, dok se zajedničko stanje u oba radna okvira može organizovati na različitim nivoima složenosti.

Razlike u organizaciji posebno dolaze do izražaja pri posmatranju ekosistema i dugoročnog održavanja. *Angular* u većoj meri standardizuje razvojni tok, kompatibilnost ključnih zavisnosti i postupke nadogradnje, dok modularniji ekosistem radnog okvira *Vue.js* ostavlja razvojnom timu više slobode pri izboru i usklađivanju dodatnih rešenja. Zbog toga dugoročna održivost projekta ne zavisi samo od osobina izabranog radnog okvira, već i od načina na koji razvojni tim definiše projektne konvencije, upravlja zavisnostima i planira nadogradnje.

Teorijski utvrđene razlike potvrđene su i praktičnim delom rada. Za potrebe poređenja razvijene su dve funkcionalno ekvivalentne aplikacije za upravljanje zadacima, sa međusobno usklađenim funkcionalnostima, ponašanjem i korisničkim interfejsom. U obe implementacije realizovani su rutiranje, autentifikacija i zaštita ruta, upravljanje zajedničkim stanjem, komponentni prikaz podataka, pretraga, filtriranje i sortiranje, rad sa formama, kao i dodavanje, izmena, prikaz i brisanje zadataka. Time je omogućeno da se poređenje usmeri prvenstveno na način implementacije istih zahteva, a ne na razlike u funkcionalnostima aplikacija.

U praktičnoj implementaciji *Angular* se oslanja na integrisan skup mehanizama, među kojima su ubrizgavanje zavisnosti, servisi, signali, funkcionalni čuvari ruta i *Reactive Forms*. U implementaciji zasnovanoj na radnom okviru *Vue.js* iste odgovornosti raspoređene su između osnovnih mehanizama radnog okvira i zvaničnih biblioteka njegovog ekosistema, pre svega *Vue Router*-a i *Pinia*-e. Na nivou reaktivnosti uočena je i značajna konceptualna sličnost: u obe implementacije stanje i izvedene vrednosti definisani su deklarativno i automatski utiču na prikaz, iako se konkretni programski interfejsi i sintaksa razlikuju.

Jedna od izraženijih praktičnih razlika uočena je pri radu sa formama. *Angular* pruža strukturisan mehanizam reaktivnih formi sa unapred definisanim kontrolama, validatorima i stanjem validnosti, dok je u korišćenoj *Vue.js* implementaciji ista funkcionalnost organizovana neposrednijim korišćenjem reaktivnog stanja, izvedenih vrednosti i direktive `v-model`. Ovaj primer pokazuje širu razliku između posmatranih pristupa: *Angular* veći deo strukture definiše kroz sopstvene mehanizme i konvencije, dok *Vue.js* ostavlja više prostora da način organizovanja odgovarajuće

logike bude određen na nivou konkretne implementacije.

Na izgled praktičnih rešenja uticao je i izbor programskih jezika. *Angular* aplikacija implementirana je u jeziku *TypeScript*, dok je *Vue.js* aplikacija implementirana u jeziku *JavaScript*. Zbog toga se razlike koje neposredno proizlaze iz statičkog sistema tipova ne mogu tumačiti kao razlike u mogućnostima samih radnih okvira. Ova činjenica je važna pri interpretaciji praktičnog poređenja, jer *Vue.js* takođe podržava razvoj aplikacija u jeziku *TypeScript*.

Praktični deo rada imao je jasno definisan obim. Podaci o zadacima čuvani su u memoriji aplikacije, autentifikacija je realizovana na klijentskoj strani u demonstracione svrhe, a automatsko testiranje i serversko renderovanje nisu bili deo praktičnog poređenja. Zbog toga praktična implementacija nije namenjena poređenju produkcijskih autentifikacionih sistema, rada sa udaljenim izvorima podataka ili neposrednom merenju performansi, već poređenju načina na koji dva radna okvira organizuju i realizuju isti skup funkcionalnosti. Dalji rad mogao bi da proširi poređenje uključivanjem serverske aplikacije i trajnog skladištenja podataka, automatizovanih testova i kvantitativnog merenja performansi u kontrolisanim uslovima.

Na osnovu teorijske analize i praktične implementacije ne može se izdvojiti univerzalno pogodniji radni okvir. Kada su za projekat posebno značajni standardizovan razvojni tok, unapred definisane konvencije i usklađivanje rada većeg broja programera, karakteristike radnog okvira *Angular* mogu predstavljati prednost. Kada su važni postepeno uvođenje tehnologije, prilagodljivost postojećem sistemu i veća sloboda pri izboru i organizaciji pratećih rešenja, karakteristike radnog okvira *Vue.js* mogu biti pogodnije. Ove smernice ne predstavljaju strogu podelu prema veličini ili vrsti projekta, već ukazuju na značaj usklađivanja karakteristika radnog okvira sa zahtevima konkretnog sistema i načinom rada razvojnog tima.

Konačni izbor između radnih okvira *Angular* i *Vue.js* zato treba posmatrati kao projektantsku odluku zasnovanu na skupu međusobno povezanih kriterijuma. Rezultati ovog rada pokazuju da oba radna okvira omogućavaju realizaciju uporedivih funkcionalnosti, dok se njihova najvažnija razlika ogleda u razvojnem pristupu, stepenu integracije i načinu raspodele odgovornosti između samog radnog okvira, njegovog ekosistema i razvojnog tima. Izbor odgovarajućeg rešenja zavisi od toga koji od ovih pristupa bolje odgovara zahtevima, ograničenjima i očekivanom razvoju konkretnog projekta.

# Bibliografija

- [1] Angular Team. Advanced component configuration. <https://angular.dev/guide/components/advanced-configuration>.
- [2] Angular Team. Angular - Introduction to Components. <https://angular.dev/guide/components>.
- [3] Angular Team. Angular - Signals. <https://angular.dev/guide/signals>.
- [4] Angular Team. Angular - Template Control Flow. <https://angular.dev/guide/templates/control-flow>.
- [5] Angular Team. Angular - Template Syntax. <https://angular.dev/guide/templates>.
- [6] Angular Team. Angular Component Dev Kit (CDK). <https://material.angular.dev/cdk>.
- [7] Angular Team. Angular DevTools. <https://angular.dev/tools/devtools>.
- [8] Angular Team. Angular Essentials. <https://angular.dev/essentials>.
- [9] Angular Team. Angular Internationalization (i18n). <https://angular.dev/guide/i18n>.
- [10] Angular Team. Angular Material. <https://material.angular.dev/>.
- [11] Angular Team. Angular Router. <https://angular.dev/guide/routing>.
- [12] Angular Team. Angular unit testing. <https://angular.dev/guide/testing>.
- [13] Angular Team. Angular update guide. <https://angular.dev/update>.
- [14] Angular Team. Angular Versioning and Releases. <https://angular.dev/reference/releases>.

- [15] Angular Team. Async reactivity with resources. <https://angular.dev/guide/signals/resource>.
- [16] Angular Team. AsyncPipe. <https://angular.dev/api/common/AsyncPipe>.
- [17] Angular Team. Building Angular Apps. <https://angular.dev/tools/cli/build>.
- [18] Angular Team. Custom Elements. <https://angular.dev/guide/elements>.
- [19] Angular Team. Dependency injection in angular. <https://angular.dev/guide/di>.
- [20] Angular Team. Expression syntax. <https://angular.dev/guide/templates/expression-syntax>.
- [21] Angular Team. Hierarchical injectors. <https://angular.dev/guide/di/hierarchical-dependency-injection>.
- [22] Angular Team. Injectors. <https://angular.dev/tools/devtools/injectors>.
- [23] Angular Team. Introduction to Angular. <https://angular.dev/overview>.
- [24] Angular Team. Migrating to New Build System. <https://angular.dev/tools/cli/build-system-migration>.
- [25] Angular Team. ng generate. <https://angular.dev/cli/generate>.
- [26] Angular Team. ng lint. <https://angular.dev/cli/lint>.
- [27] Angular Team. ng new. <https://angular.dev/cli/new>.
- [28] Angular Team. NgModules. <https://angular.dev/guide/ngmodules/overview>.
- [29] Angular Team. Profiler. <https://angular.dev/tools/devtools/profiler>.
- [30] Angular Team. Router Tree. <https://angular.dev/tools/devtools/router>.
- [31] Angular Team. RxJS interop with Angular signals. <https://angular.dev/ecosystem/rxjs-interop>.

- [32] Angular Team. Server-side and Hybrid Rendering. <https://angular.dev/guide/prerendering>.
- [33] Angular Team. Serving Angular Apps for Development. <https://angular.dev/tools/cli/serve>.
- [34] Angular Team. Testing Utility APIs. <https://angular.dev/guide/testing/utility-apis>.
- [35] Angular Team. Two-way binding. <https://angular.dev/guide/templates/two-way-binding>.
- [36] Angular Team. Version Compatibility. <https://angular.dev/reference/versions>.
- [37] Angular Team. Zoneless. <https://angular.dev/guide/zoneless>.
- [38] eslint-plugin-vue Team. eslint-plugin-vue. <https://eslint.vuejs.org/>.
- [39] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [40] Google Search Central. Understand JavaScript SEO Basics. <https://developers.google.com/search/docs/crawling-indexing/javascript/javascript-seo-basics>.
- [41] John Gossman. Introduction to model/view/viewmodel pattern for building wpf apps. Microsoft Developer Blogs (archived).
- [42] Intlify Team. Vue I18n. <https://vue-i18n.intlify.dev/>.
- [43] Piotr Lipski, Jarosław Kyć, and Beata Pańczyk. Comparative analysis of the angular 10 and vue 3.0 frameworks. *Journal of Computer Sciences Institute*, 20:205–209, 2021. DOI: <https://doi.org/10.35784/jcsi.2688>.
- [44] MDN Web Docs. CSS: Cascading Style Sheets. <https://developer.mozilla.org/en-US/docs/Web/CSS>.
- [45] MDN Web Docs. Fetch API. [https://developer.mozilla.org/en-US/docs/Web/API/Fetch\\_API](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API).



- [46] MDN Web Docs. Introduction to client-side frameworks. [https://developer.mozilla.org/en-US/docs/Learn\\_web\\_development/Core/Frameworks\\_libraries/Introduction](https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Frameworks_libraries/Introduction).
- [47] MDN Web Docs. Introduction to the dom. [https://developer.mozilla.org/en-US/docs/Web/API/Document\\_Object\\_Model/Introduction](https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction).
- [48] MDN Web Docs. JavaScript – Origins and History. <https://developer.mozilla.org/en-US/docs/Glossary/JavaScript>.
- [49] MDN Web Docs. JavaScript overview. [https://developer.mozilla.org/en-US/docs/Learn/JavaScript/First\\_steps/What\\_is\\_JavaScript](https://developer.mozilla.org/en-US/docs/Learn/JavaScript/First_steps/What_is_JavaScript).
- [50] MDN Web Docs. Making network requests in JavaScript. [https://developer.mozilla.org/en-US/docs/Learn\\_web\\_development/Core/Scripting/Network\\_requests](https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Network_requests).
- [51] MDN Web Docs. Manipulating the browser history. [https://developer.mozilla.org/en-US/docs/Web/API/History\\_API](https://developer.mozilla.org/en-US/docs/Web/API/History_API).
- [52] MDN Web Docs. Model-View-Controller (MVC). <https://developer.mozilla.org/en-US/docs/Glossary/MVC>.
- [53] MDN Web Docs. Structuring content with HTML. [https://developer.mozilla.org/en-US/docs/Learn\\_web\\_development/Core/Structuring\\_content](https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content).
- [54] Tommi Mikkonen and Antero Taivalsaari. Web applications – spaghetti code for the 21st century. In *Sixth International Conference on Software Engineering Research, Management and Applications (SERA 2008)*, pages 319–328. IEEE, 2008. DOI: <https://doi.org/10.1109/SERA.2008.16>.
- [55] Michael S. Mikowski and Josh C. Powell. *Single Page Web Applications: JavaScript End-to-End*. Manning Publications, 2013.
- [56] NgRx Team. Effects. <https://ngrx.io/guide/effects>.
- [57] NgRx Team. NgRx Store. <https://ngrx.io/guide/store>.
- [58] NgRx Team. SignalStore. <https://ngrx.io/guide/signals/signal-store>.

- [59] Nuxt Core Team. Rendering Modes – Nuxt 4. <https://nuxt.com/docs/4.x/guide/concepts/rendering>.
- [60] Risto Ollila, Niko Mäkitalo, and Tommi Mikkonen. Modern web frameworks: A comparison of rendering performance. *Journal of Web Engineering*, 21(3):789–814, 2022. DOI: <https://doi.org/10.13052/jwe1540-9589.21311>.
- [61] OpenJS Foundation. jQuery. <https://jquery.com/>.
- [62] Amantia Pano, Daniel Graziotin, and Pekka Abrahamsson. Factors and actors leading to the adoption of a javascript framework. *Empirical Software Engineering*, 23(6):3503–3534, 2018. DOI: <https://doi.org/10.1007/s10664-018-9613-x>.
- [63] Pinia Team. Actions. <https://pinia.vuejs.org/core-concepts/actions.html>.
- [64] Pinia Team. Pinia Documentation. <https://pinia.vuejs.org/>.
- [65] PrimeTek. PrimeVue. <https://primevue.dev/>.
- [66] Trygve Reenskaug. Models–views–controllers. Technical report, Xerox PARC. DOI: <https://doi.org/10.5281/zenodo.3676092>.
- [67] RxJS Core Team. RxJS - Observable. <https://rxjs.dev/guide/observable>.
- [68] Josh Smith. Wpf apps with the model-view-viewmodel design pattern. *MSDN Magazine*, 24(2), 2009.
- [69] Clemens Szyperski, Dominik Gruntz, and Stephan Murer. *Component Software: Beyond Object-Oriented Programming*. ACM Press/Addison-Wesley, 2 edition, 2002.
- [70] TypeScript Team. TypeScript Documentation. <https://www.typescriptlang.org/docs/>.
- [71] Vite Team. Getting Started. <https://vite.dev/guide/>.
- [72] Vite Team. Why Vite. <https://vite.dev/guide/why.html>.

- [73] Vitest Team. Vitest Documentation. <https://vitest.dev/>.
- [74] Vue.js Core Team. Composables. <https://vuejs.org/guide/reusability/composables.html>.
- [75] Vue.js Core Team. Composition API FAQ. <https://vuejs.org/guide/extras/composition-api-faq.html>.
- [76] Vue.js Core Team. Introduction — What is Vue.js? <https://vuejs.org/guide/introduction.html>.
- [77] Vue.js Core Team. List Rendering. <https://vuejs.org/guide/essentials/list>.
- [78] Vue.js Core Team. Migration Build. <https://v3-migration.vuejs.org/migration-build>.
- [79] Vue.js Core Team. Quick Start. <https://vuejs.org/guide/quick-start.html>.
- [80] Vue.js Core Team. Reactivity in Depth. <https://vuejs.org/guide/extras/reactivity-in-depth.html>.
- [81] Vue.js Core Team. Releases. <https://vuejs.org/about/releases>.
- [82] Vue.js Core Team. Rendering Mechanism. <https://vuejs.org/guide/extras/rendering-mechanism>.
- [83] Vue.js Core Team. Server-Side Rendering (SSR). <https://vuejs.org/guide/scaling-up/ssr.html>.
- [84] Vue.js Core Team. State Management. <https://vuejs.org/guide/scaling-up/state-management.html>.
- [85] Vue.js Core Team. Testing. <https://vuejs.org/guide/scaling-up/testing.html>.
- [86] Vue.js Core Team. Tooling. <https://vuejs.org/guide/scaling-up/tooling>.
- [87] Vue.js Core Team. Vue 3 Migration Guide. <https://v3-migration.vuejs.org/>.

- [88] Vue.js Core Team. Vue CLI. <https://cli.vuejs.org/>.
- [89] Vue.js Core Team. Vue DevTools - Browser extension for debugging Vue.js applications. <https://devtools.vuejs.org/>.
- [90] Vue.js Core Team. Vue DevTools - Features. <https://devtools.vuejs.org/getting-started/features>.
- [91] Vue.js Core Team. Vue Router. <https://router.vuejs.org/>.
- [92] Vue.js Core Team. Vue Test Utils. <https://test-utils.vuejs.org/>.
- [93] Vue.js Core Team. Vue.js - Components Basics. <https://vuejs.org/guide/essentials/component-basics.html>.
- [94] Vue.js Core Team. Vue.js - Form Input Bindings (v-model). <https://vuejs.org/guide/essentials/forms.html>.
- [95] Vue.js Core Team. Vue.js - Provide/Inject. <https://vuejs.org/guide/components/provide-inject.html>.
- [96] Vue.js Core Team. Vue.js - <script setup>. <https://vuejs.org/api/sfc-script-setup.html>.
- [97] Vue.js Core Team. Vue.js - Single File Components. <https://vuejs.org/guide/scaling-up/sfc.html>.
- [98] Vue.js Core Team. Vue.js - Template Syntax. <https://vuejs.org/guide/essentials/template-syntax.html>.
- [99] Vue.js Core Team. Vue.js and TypeScript Support. <https://vuejs.org/guide/typescript/overview.html>.
- [100] Vue.js Core Team. Vue.js Core Releases. <https://github.com/vuejs/core/releases>.
- [101] Vue.js Core Team. Watchers. <https://vuejs.org/guide/essentials/watchers.html>.
- [102] Vuetify Team. Vuetify. <https://vuetifyjs.com/>.

# Biografija autora

**Aleksandra A. Trailović** rođena je 20. aprila 1997. godine u Požarevcu. Osnovnu školu „Ugrin Branković” završila je u Kučevu kao dobitnica diplome „Vuk Karadžić”. Srednje obrazovanje stekla je u Ekonomsko-trgovinskoj i mašinskoj školi u Kučevu, na smeru ekonomski tehničar, takođe kao dobitnica diplome „Vuk Karadžić”.

Osnovne akademske studije upisala je 2016. godine na Matematičkom fakultetu Univerziteta u Beogradu, na studijskom programu Matematika, modul Računarstvo i informatika, a završila ih je 2022. godine. Iste godine upisala je master akademske studije na istom fakultetu, na studijskom programu Matematika, modul Računarstvo i matematika.

Tokom praktičnog rada u oblasti razvoja softvera susretala se sa radnim okvirima *Angular* i *Vue.js*, što je predstavljalo jedan od glavnih podsticaja za izbor teme ovog master rada.